

分类号: (按中国图书分类法, 学位办网上可查)

单位代码: 10335

密 级: 公开

学 号: 22141009

浙江大学

硕士学位论文



中文论文题目: 基于 SRAM 的高能效比全数字存内计算架构
及电路设计研究

英文论文题目: Research on All-Digital SRAM-Based
Computing-In-Memory Architecture and
Circuit Design with High Energy Efficiency

申请人姓名: 陈业宽

指导教师: 朱晓雷

专业名称: 电子科学与技术

研究方向: 存内计算; 模拟集成电路设计

所在学院: 集成电路学院

论文提交日期: 2024 年 1 月 10 日

中文论文题目

基于 SRAM 的高能效比全数字存内计算架构
及电路设计研究



论文作者签名: _____

指导教师签名: _____

论文评阅人 1: _____

评阅人 2: _____

评阅人 3: _____

评阅人 4: _____

评阅人 5: _____

答辩委员会主席: 虞小鹏\教授\浙江大学

委员 1: 黄科杰\研究员\浙江大学

委员 2: 葛海通\高级工程师\杭州宙其科技有限公司

委员 3: _____

委员 4: _____

委员 5: _____

答辩日期: 2024 年 3 月 12 日

摘要

近年来,伴随着人工智能技术的快速发展及广泛应用,与人工智能相关的算法需要处理的数据量越来越庞大,传统冯诺依曼架构中中央处理器与存储器之间频繁的数据搬移产生了巨大的传输功耗。同时,存储器带宽与处理器性能之间的不匹配也限制了计算硬件的算力性能。存内计算(Computing-in-Memory, CIM)架构通过赋予存储器一定的计算功能,将大量数据在存储器内部直接进行计算,减少了存储器与中央处理器之间的总线交互频率,降低了数据传输的功耗,从而大幅提高了系统的能耗效率,因此在近几年其受到了工业界与学术界的广泛关注。

本文以基于 6T SRAM 的存内计算设计为研究内容,首先对 6T SRAM 的保持操作、写操作及读操作进行了具体的介绍,分别就模拟与数字两种存内计算电路架构的实现方式及优缺点进行了分析和对比,然后重点围绕基于 6T SRAM 的全精度数字存内计算架构进行了研究,提出了两种高吞吐率、高能效比的全精度数字存内计算架构。首先设计了一种基于 6T SRAM 的串行单比特存内计算电路架构,通过两级流水线的方式架构在多周期内完成多比特卷积运算并实现高并行度运算,系统支持最高输入数据位宽为 8-bit,最高权重数据位宽为 16-bit,并且支持输入与权重值的符号位配置。采用台积电 28nm 标准 CMOS 工艺对上述存内计算架构进行了电路设计与仿真,在处理 4-bit 输入与 4-bit 权重并输出 16-bit 卷积计算结果时,该存内计算电路可实现 3300GOPS 的吞吐率与 56.39TOPS/W 的能耗效率。此外,为了进一步提高系统的能耗效率,在第一种存内计算架构的基础上,引入 Booth 乘法的原理,提出了一种基于复用 Booth 计算单元与 6T SRAM 的串行单比特可重构存内计算电路架构,该架构每个操作周期内支持 3-bit 的输入数据位宽,输入数据先通过 Booth 编码处理,编码出 Booth 计算的驱动控制信号,再输入到存内计算阵列中进行运算,简化了多比特乘法的硬件实现复杂度。同时架构采取分时复用的策略并设计了一种可配置移位周期的移位累加器,使系统可以配置卷积运算的规模从而灵活应对不同规模的神经网络算法。该存内计算架构支持最高输入位宽为 8-bit,最高权重位宽为 8-bit,并且支持输入与权重的符号配置。同样采用台积电 28nm 标准 CMOS 工艺对提出的存算架构进行了电路设计与仿真,在处理 4-bit 输入与 4-bit 权重并输出 16-bit 卷积计算结果时,该存内计算电路可以获得 1820GOPS 的吞吐率与 61.745TOPS/W 的能耗效率。

本文提出的两种全精度数字存内计算设计突破了模拟存内计算设计中的技术瓶颈，实现了较高的吞吐率与良好的能耗效率，并且支持数据位宽、卷积运算规模等参数的灵活配置，可应用于多比特卷积运算的加速场景。相关技术有望在后摩尔时代突破计算设备的算力与能效瓶颈。

关键词：人工智能；冯诺依曼架构；存内计算；SRAM；卷积神经网络

ABSTRACT

In recent years, the rapid development and widespread application of AI technology have led to a significant increase in the volume of data processed by AI-related algorithms. The frequent data relocation between the central processor and memory in the conventional Von Neumann architecture results in substantial transmission power consumption. Simultaneously, the mismatch between memory bandwidth and processor performance imposes limitations on the computational capabilities of hardware. The Computing-in-Memory (CIM) architecture addresses these challenges by reducing the frequency of bus interactions between memory and the central processor. By endowing the memory with specific computational functions and enabling the direct processing of large data sets within the memory itself, CIM effectively minimizes power consumption during data transfer. This innovation greatly enhances the energy efficiency of the system, garnering considerable attention from both industry and academia in recent years.

In this thesis, we delve into the design of a 6T SRAM-based in-store computation system. Initially, we present an overview of the hold, write, and read operations of 6T SRAM. Subsequently, we conduct a detailed analysis and comparison of the implementation, advantages, and disadvantages of both analogue and digital in-store computation circuits. Our research then narrows down to the exploration of a full-precision digital in-store computation architecture based on 6T SRAM, introducing two high-throughput and high-energy-efficiency digital in-store computation architectures. Turning our attention to the 6T SRAM-based full-precision digital in-store computing architecture, we propose two high-throughput and high-efficiency alternatives. Firstly, we design a serial single-bit in-store computing circuit architecture based on 6T SRAM, completing multi-bit convolutional operations across multiple cycles. This achieves substantial parallelism through a two-stage pipelined architecture. The system supports an input data bit-width of up to 8 bits and a weight data bit-width of up to 16 bits, accommodating various sign-bit configurations for both inputs and weight values. The circuit design and simulation utilize TSMC's 28nm standard CMOS process, demonstrating a throughput rate of 3300GOPS and an energy efficiency of 56.39TOPS/W when processing 4-bit inputs with 4-bit weights, and producing 16-bit convolutional computation results. Additionally, to further enhance energy efficiency, we introduce the principle of Booth multiplication. Building upon the first in-store computing architecture, we propose a serial single-bit reconfigurable in-store computing circuit

architecture. This design incorporates multiplexed Booth computing units and 6T SRAM, supporting 3-bit input data bit-widths in each operation cycle. Booth encoding simplifies the hardware implementation of multi-bit multiplication by encoding drive control signals for Booth computation. The architecture employs a time-sharing multiplexing strategy and integrates a configurable shift accumulator, allowing flexibility in configuring the scale of convolutional operations to meet the demands of various neural network algorithms. This in-store computing architecture supports a maximum input bit-width of 8 bits and a maximum weight bit-width of 8 bits, with symbolic configuration options for both inputs and weights. Simulation using TSMC's 28nm standard CMOS process showcases a throughput rate of 1820 GOPS and an energy efficiency of 61.745 TOPS/W when processing 4-bit inputs with 4-bit weights, resulting in 16-bit convolutional computation outputs.

The two full-precision digital in-store computing designs presented in this paper represent a significant breakthrough in addressing the technical challenges posed by analogue in-store computing. These designs not only achieve a high throughput rate and commendable energy efficiency but also provide support for the flexible configuration of parameters such as data bit-width and convolutional operation scale. This adaptability makes them well-suited for acceleration scenarios involving multi-bit convolutional operations. The implications of these advancements extend beyond the current landscape, as the related technology holds the potential to surmount the impending bottleneck in computing power and energy efficiency faced by computing devices in the post-Moore era.

KEY WORDS: artificial intelligence; von Neumann architecture; Computing-in-Memory; SRAM; convolutional neural network

目 录

第 1 章 绪论.....	1
1.1 研究工作的背景和意义.....	1
1.2 存内计算研究现状.....	2
1.3 论文内容与结构安排.....	4
第 2 章 存内计算电路设计基础	7
2.1 6 管 SRAM 单元概述	7
2.1.1 基本结构	7
2.1.2 6 管 SRAM 的保持操作	7
2.1.3 6 管 SRAM 的写操作	8
2.1.4 6 管 SRAM 的读操作	9
2.2 模拟存内计算设计综述.....	11
2.2.1 模拟存内计算介绍.....	11
2.2.2 模拟存内计算电路设计	12
2.2.3 模拟存内计算设计中的挑战.....	16
2.3 数字存内计算设计综述.....	17
2.3.1 数字存内计算电路设计	17
2.3.2 面向多比特卷积计算的数字存内计算实现方法	19
2.4 存内计算电路设计指标.....	21
2.5 本章小结.....	21
第 3 章 基于 6T SRAM 的串行单比特输入存内计算设计	22
3.1 串行单比特输入数字存内计算设计架构	22
3.2 串行单比特输入存内计算电路设计	24
3.2.1 基于 6T SRAM 的存内计算单元设计	24
3.2.2 并行加法器树电路设计	25
3.2.3 累加器电路设计	28
3.2.4 读入读出驱动电路设计	29
3.2.5 时钟控制电路设计	33
3.2.6 字线译码器电路设计	37
3.3 电路性能仿真与分析.....	38
3.3.1 存内计算单元性能仿真.....	38
3.3.2 系统性能仿真与分析.....	43
3.3.3 串行单比特输入存内计算架构接口时序	48
3.4 本章小结.....	51
第 4 章 基于复用 Booth 计算单元和 6T SRAM 的可重构存内计算	

设计	52
4.1 复用 Booth 计算单元和 6T SRAM 存内计算设计架构.....	52
4.2 架构分时复用策略与可重构卷积运算时序	53
4.3 复用 Booth 计算单元和 6T SRAM 存内计算电路设计.....	56
4.3.1 Booth 乘法编码电路设计	56
4.3.2 Booth 乘法计算单元电路设计	58
4.3.3 位扩展单元设计	59
4.3.4 时钟控制电路与地址译码器电路设计	60
4.4 电路性能仿真与分析.....	62
4.4.1 Booth 乘法存内计算数据块性能仿真	62
4.4.2 系统性能仿真与分析	64
4.5 本章小结.....	69
第 5 章 总结与展望	71
5.1 全文总结.....	71
5.2 未来研究展望.....	72
参考文献.....	73

图目录

图 1-1 冯诺伊曼计算架构与存内计算架构的对比图	2
图 2-1 6T SRAM 单元电路结构图	7
图 2-2 6T SRAM 单元的保持操作	8
图 2-3 (a) 6T SRAM 单元的写操作 (b)写操作时序图	9
图 2-4 (a) 6T SRAM 单元的读操作 (b)读操作时序图	10
图 2-5 基于 SRAM 的模拟存内计算设计	11
图 2-6 10T SRAM 计算单元	13
图 2-7 分裂式字线 6T SRAM 单元	14
图 2-8 12T SRAM 计算单元	15
图 2-9 分比特乘法计算单元	16
图 2-10 基于双字线 6 管 SRAM 单元的数字存内计算 Cache 结构	18
图 2-11 面向大规模并行加法运算的数字存内计算 Cache 结构	19
图 2-12 位并行输入卷积计算流程图	20
图 2-13 位串行单比特输入卷积计算流程图	20
图 3-1 串行单比特输入存内计算设计架构	22
图 3-2 基于 6T SRAM 的存内计算比特单元	24
图 3-3 并行加法器数电路示意图	25
图 3-4 两种全加器电路结构: (a)28 管全加器电路 (b)11 管全加器电路	26
图 3-5 11T 与 28T 交替运行的加法器树结构	27
图 3-6 符号位扩展电路	28
图 3-7 累加器电路	28
图 3-8 卷积计算时序图	29
图 3-9 读入读出驱动电路结构示意图	30
图 3-10 位线驱动电路	31
图 3-11 (a)输入多路选择器电路 (b)输出多路选择器电路	32
图 3-12 电压锁存型灵敏放大器	33
图 3-13 时钟控制电路结构示意图	34

图 3-14 RSTN 模块内部电路示意图.....	35
图 3-15 TRKWL 模块内部电路示意图.....	36
图 3-16 两种译码器电路: (a)2-4 译码器 (b)3-8 译码器 (编译 Y[7:0]为例)	37
图 3-17 字线译码器电路.....	38
图 3-18 存内计算单元版图.....	39
图 3-19 HSNM 仿真分析: (a)蝴蝶曲线图 (b)0.9V 电压下蒙特卡洛仿真结果	40
图 3-20 RSNM 仿真分析: (a)蝴蝶曲线图 (b)0.9V 电压下蒙特卡洛仿真结果.....	41
图 3-21 RSNM 随电压变化情况	42
图 3-22 WSNM 仿真分析: (a)QN 随 Q 变化曲线图 (b)0.9V 电压下蒙特卡洛仿真结果	43
图 3-23 存内计算单元计算性能仿真分析: (a)计算延时对比 (b)计算功耗对比.....	43
图 3-24 存内计算处理器 (CIM Macro) 版图.....	44
图 3-25 串行单比特存内计算架构顶层版图.....	44
图 3-26 系统的瞬态仿真: (a)读写操作过程 (b)计算过程.....	45
图 3-27 系统的读取延时与计算延时对比.....	46
图 3-28 系统的三种操作功耗对比.....	46
图 3-29 系统内各模块的计算功耗占比.....	47
图 3-30 系统的能耗效率随工作条件的变化趋势.....	47
图 3-31 存内计算芯片版图.....	51
图 4-1 基于复用 Booth 计算单元和 6T SRAM 的存内计算设计架构.....	53
图 4-2 存内计算阵列布局.....	54
图 4-3 串行双比特输入卷积计算架构.....	54
图 4-4 Booth 乘法存内计算处理器内部结构示意图	55
图 4-5 可重构卷积计算操作时序图.....	56
图 4-6 Booth 乘法编码电路	57
图 4-7 Booth 乘法编码电路的操作真值表	58
图 4-8 Booth 乘法存内计算数据块电路示意图	59
图 4-9 位扩展单元工作流程.....	60
图 4-10 添加计算标志位的时钟控制电路.....	61
图 4-11 列地址译码器.....	62

图 4-12 BMCDB 电路的瞬态仿真	63
图 4-13 BMCDB 性能仿真分析: (a)计算功耗对比 (b)计算速度对比	63
图 4-14 系统写操作过程的瞬态仿真.....	64
图 4-15 系统计算操作过程的瞬态仿真.....	65
图 4-16 (a) 不同电压下系统的功耗对比 (b) 不同电压下系统的计算速度对比.....	66
图 4-17 系统内各模块的计算功耗占比.....	66
图 4-18 系统的吞吐率与能耗效率随电源电压的变化趋势.....	67
图 4-19 系统的吞吐率与能耗效率随卷积计算组数的变化.....	68

表 目 录

表 3-1 存内计算单元的操作真值表	25
表 3-2 符号位扩展电路的操作真值表	27
表 3-3 串行单比特存内计算架构性能汇总	48
表 3-4 串行单比特存内计算架构接口功能汇总	49
表 3-5 串行单比特存内计算架构接口时序汇总	50
表 4-1 基于复用 Booth 计算单元和 6T SRAM 存内计算架构的性能总结与对比	69

第1章 绪论

1.1 研究工作的背景和意义

随着大数据、云计算（Cloud Computing）、物联网（Internet of Things, IOT）和人工智能（Artificial Intelligent, AI）等领域的迅速发展^[1-3]，我们正步入一个数据驱动的时代。这个时代特征是数据量的爆炸式增长以及对高效、高速数据处理的迫切需求。据研究所知，自 2012 年起，全世界每天产生的数据量约为 2.5×10^{18} Byte，且该数据量仍以每 40 个月翻倍的速度在持续增长^[6]。处理这些大规模数据集对传统的计算架构提出了巨大的挑战。

传统的冯诺依曼计算机架构中共有五个组成部分：处理器、控制器、存储器、输入设备和输出设备。其中，处理器与存储器在物理上是相互分离的，数据通过总线在两者之间来回传输^[4]。在面对人工智能领域中的图像识别、语音处理等大规模数据集的应用中，运算器与存储器之间需要通过有限的总线带宽进行频繁且大量的数据传输，这就导致数据传输所消耗的能量远远大于数据运算所消耗的能量^[18]，这一现象被称为“功耗墙”问题。例如根据英伟达的研究报告，在 22nm 工艺节点下，进行浮点数运算时所需的数据传输功耗约为数据处理功耗的 200 倍^[12]；根据英特尔的研究表明，在 7nm 工艺节点下，数据搬运功耗高达 35pJ/bit，占总功耗的 63.7%。此外，存储器与处理器性能之间的不匹配带来了“存储墙（Memory Wall）”问题^[35]。在过去的 20 年里，处理器的性能根据摩尔定律呈指数型增长，运算设备的算力提高了近 90000 倍，而存储器虽然从 DDR 发展到 GDDR6x，接口标准从 PCIe1.0a 升级到 NVLink3.0，但是存储器的传输带宽仅增长了 30 倍。这就导致在面对神经网络算法等数据交互频繁的应用时，受限于存储器有限的传输带宽，处理器不得不频繁地进入等待周期，导致处理器的实际算力远低于理论算力，严重限制了深度神经网络算法的推理速度。“功耗墙”问题与“存储墙”问题并称为冯诺依曼架构瓶颈^[10,11]。

为了缓解冯诺依曼架构瓶颈，一种被称为近存储计算（Near Memory Computing）的方案被提出^[8]。近存储计算核心思路是缩短存储器和处理器之间的数据传输距离，可以在一定程度上减少数据传输的功耗。目前产业界主要是采用 3 维堆叠技术把更多的存储器通过在垂直方向上堆叠，提高数据带宽，并缩短存储器与处理器之间的距离，

从而减小数据搬运功耗与数据通信延迟^[7]。但是近存储计算本质上还是基于冯诺依曼架构，数据传输依然要通过总线才能实现，并不能从根本上解决冯诺依曼架构瓶颈^[4]。

近年来，存内计算（Computing-in-Memory, CIM）架构受到了广泛的研究与关注^[5]。存内计算架构是在存储器内嵌入一部分计算单元，从而让存储器本身不仅具有存储功能同时还可以支持特定的运算操作，如乘累加（Multiplication and Accumulation, MAC）运算、同或累加（XNOR and Accumulation, XAC）运算等。如图 1-1 所示，存内计算架构中存储器内部的数据可以直接通过内部的计算单元进行数据运算，只将运算结果通过总线输出到处理中。相比于传统的冯诺依曼架构，其大大减少了处理器和存储器之间频繁的总线交互，减少了数据的搬移量，从而降低了因数据搬移所产生的大量功耗，提升了系统的能耗效率^[52]。同时总线交互的减少也缓解了因总线传输带宽限制所带来的处理器算力降低的问题。综上，存内计算作为一种新型计算架构，有望突破冯诺依曼架构“存储墙”与“功耗墙”的瓶颈，成为后摩尔时代集成电路领域的重点研究方向之一^[4]。

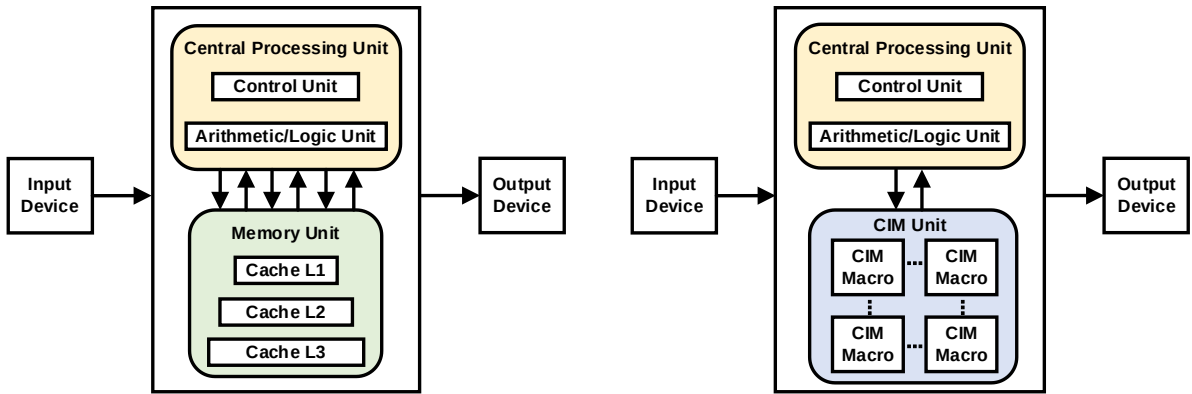


图 1-1 冯诺伊曼计算架构与存内计算架构的对比图

1.2 存内计算研究现状

存内计算技术概念的形成，最早可以追溯到上个世纪 70 年代^[54]。1969 年，斯坦福研究所的 Kautz 等人提出了一种存储器内单元逻辑（Cellular Logic-in-Memory）的概念^[36,37]。但由于当时的芯片制造技术的落后和算力需求较小，那时存内计算技术仅仅停留在理论研究阶段，并未得到实际应用。

近年来，随着半导体制造技术的飞速发展，以及 AI Edge、元宇宙等对算力需求巨大的应用场景的兴起，为存算一体技术提供了崭新的制造平台和强大的产业驱动力^[54]。传统的芯片大厂正在积极开展存内计算芯片的研发，包括英特尔、SK 海力士、

IBM、美光、三星、台积电、Mythic 等传统芯片大厂^[50]。2021 年初，三星发布基于 HBM 的新型内存^[40]，里面集成了 AI 处理器，该处理器可以实现高达 1.2TFLOPS 的计算能力，新型 HBM-PIM（存内计算）芯片将 AI 引擎引入每个存储库，从而将处理操作转移到 HBM，可以减轻在内存和处理器之间搬运数据的负担。三星表示新型 HBM-PIM 芯片，可以提供 2 倍的系统性能，同时能耗降低了 70% 以上。Mythic 于 2021 年推出基于 NOR Flash 的存内计算量产芯片 M1076，可支持 80MB 神经网络权重，单个芯片算力达到 25TOPS，主要面向边缘侧智能场景^[4]。2022 年 2 月，SK 海力士发布了一项最新的基于 GDDR 接口的 DRAM 存内计算研究成果，并展示了首个采用存内计算技术的产品——GDDR6-AiM^[53]。该产品将计算功能整合到数据传输速度达 16Gbps 的 GDDR6 内存产品中。相对于传统的 DRAM，将 GDDR6-AiM 与 CPU、GPU 结合的系统在特定计算环境中可实现 16 倍的计算速度提升。此外，由于存内计算显著减少了内存与 CPU、GPU 之间的数据传输频次，GDDR6-AiM 的功耗降低了 80%，这一技术突破有望在对高性能计算和低功耗要求的特定应用领域中产生显著影响。

国内众多新兴 AI 和存储企业也在陆续融资，推进存内计算技术的研发，比如亿铸科技、千芯科技、后摩智能、闪易半导体、苹芯科技、知存科技、智芯科、九天睿芯、恒烁半导体等。2021 年，知存科技发布了基于 NOR Flash 的存内计算 SoC 芯片 WTM2101^[4]。该芯片的算力性能相比市场上的同类产品高出两个数量级，同时功耗低于 1 mW。其主要定位于端侧低功耗和低成本的应用场景。亿铸科技成立于 2021 年 9 月，主要基于 ReRAM 路线来实现大算力的存内计算芯片，亿铸科技主要为数据中心和自动驾驶等领域打造能效比十倍于现有技术的解决方案，研发能力覆盖从存内计算底层器件，芯片设计到 AI 软件栈。目前亿铸科技已完成过亿元天使轮融资。后摩智能成立于 2020 年底，其专注于存内计算技术的研发，致力于打造大算力 AI 芯片，以提供高效能、低功耗的解决方案。这些芯片及解决方案广泛应用于智能驾驶、泛机器人等边缘端以及云端推理场景^[55]。该公司第一代芯片是基于 SRAM 的智能计算芯片，并且于 2021 年 8 月后摩智能宣布完成首款芯片验证流片，于 2022 年 4 月完成数亿元人民币 Pre-A+轮融资。

此外，存内计算技术近年来在学术界也是热门的研究课题之一。2016 年，Supreet Jeloka 等在文章^[47]中提出基于双字线 6 管存储单元的存内计算设计，实现了与运算及或非运算，引起了学术界的广泛关注。2017 年，Jintao Zhang 等在文章^[26]中提出基于

SRAM 存内计算的分类器设计, 采用多个弱分类器来构成一个强分类器的方法来提高分类的精度。2018 年, S. K. Gonugondla 等在文章[46]中提出一种支持向量机 (support vector machine, SVM) 算法中乘加运算的存内计算电路设计, 系统能耗效率达到 3.125TOPS/W。同年, Win-San Khwa 等在文章[19]中提出了一种基于分裂式字线的 6T SRAM 存内计算设计, 可以实现对二值神经网络全连接层的计算加速, 系统的能耗效率达到 55.8TOPS/W。2019 年, Avishek Biswas 等在文章[25]中提出一种基于 10 管存储单元的存内计算设计, 该设计实现了较大位宽的数据计算, 但由于 10 管存储单元面积开销较大, 同时设计引入 DAC 与 ADC 电路, 导致整体设计面积较为庞大。同年, Daichi Fujiki 等在文章[38]中提出一种基于 Cache 的存内计算设计, 该设计可以在 Cache 内部实现浮点数运算。2020 年, Xin Si 等在文章[22]中提出一种基于双生 8T SRAM 的多比特存内计算设计, 设计了一种奇偶双通道多比特输入 IN 的配置架构, 可以支持多比特 (不超过 5-bit) 的乘加计算, 系统可以实现 18.37TOPS/W 到 72.03TOPS/W 的能耗效率。同年, Shihui Yin 在文章[28]中提出一种基于 12 管 SRAM 计算单元的存内计算设计, 在基于手写数字的 MNIST 数据集上达到了 98.84% 的识别准确率, 在 CIFAR-10 数据集上实现了 88.78% 的准确率, 系统的能耗效率高达 403TOPS/W。2021 年, Xin Si 等在文章[21]中提出了一种基于局部计算单元与 6 管 SRAM 的存内计算设计, 该设计采用分比特权重乘加的思路, 进一步提高读取容限并且支持高达 8-bit 数据位宽的乘加运算, 系统达到了 11.5TOPS/W~68.4TOPS/W 的能耗效率。同年, Liu Qi 等在文章[29]中提出了一种基于三值 DRAM 的存算一体架构实现神经网络运算的加速。

综上所述可以看出不论是在学术界还是产业界, 存内计算架构近年来都备受关注, 并且衍生出了各种不同的架构类型。这主要是由于存内计算架构具有以下优点: (1) 具有极高的能耗效率和吞吐率; (2) 数据传输功耗显著降低, 节省了系统整体能耗开销; (3) 存储模块与计算单元相融合, 节省了系统的面积开销。因此, 研究存内计算设计有望在后摩尔时代突破计算机的算力与功耗瓶颈^[52]。

1.3 论文内容与结构安排

本文以全精度、高吞吐率与高能效比的 SRAM 存内计算设计为重点研究内容, 分别提出基于 6T SRAM 的串行单比特数字存内计算设计以及基于复用 Booth 计算单元和

6T SRAM 的可重构数字存内计算设计。基于 6T SRAM 的串行单比特数字存内计算设计通过采取串行单比特输入的数据输入方式，在传统的 6 管 SRAM 单元内嵌入了或非门单元实现单比特的逻辑乘法，并通过流水线的方式在多周期内实现最高 8 比特输入与 8bit 权重的卷积运算，同时每一个操作周期内系统共支持 256×64 组数据并行进行计算。系统的计算精度达到 100%，吞吐率达到 917GOPS~3300GOPS，同时获得了 15.67TOPS/W~56.39TOPS/W 的能耗效率。基于复用 Booth 计算单元和 6T SRAM 的可重构数字存内计算设计采取串行多轨输入的数据输入方式，通过预先对多比特输入进行 Booth 编码处理，然后将得到的操作信号驱动架构中设计的 Booth 计算单元与位扩展单元进行操作，降低了计算单元的面积和功耗开销。同时架构采取分时复用的策略，每四行 SRAM 共用一个 Booth 计算单元，虽然减少了每个操作周期内并行计算的数据组数，但是降低了加法器树的规模，减少了系统的功耗，并且由于 Booth 计算带来的运算所需周期数的减少，系统达到了更高的能耗效率。并且系统可配置卷积计算的规模，从而可以灵活应用不同规模的神经网络算法。系统在单周期内可支持并行运算组数为 $64N \times 64$ (N 最大为 4)，系统的计算精度达到 100%，吞吐率达到 482GOPS~1820GOPS，同时获得了 21.64TOPS/W~61.745TOPS/W 的能耗效率。

本文分为 5 个章节，具体章节安排如下：

第 1 章为本文的绪论，首先介绍了存内计算的研究背景和研究意义，然后调研了近几年存内计算技术在学术界与产业界的研究现状，最后对本文的内容与框架做了简要概括。

第 2 章为存内计算的设计基础。首先详细阐述了传统 6 管 SRAM 的基本结构与工作原理，接着引出了模拟存内计算设计的概念，并根据调研描述了几种模拟存内计算单元的工作原理并分析了它们的优缺点，同时指出了模拟存内计算设计的优势以及遇到的一些挑战。最后提出数字存内计算设计的概念与实现方法，并简单对比了其模拟存内计算设计的优劣势，同时简要概括了进行存内计算电路设计时需要考虑的一些指标。

第 3 章介绍基于 6T SRAM 的串行单比特数字存内计算设计。从整个设计的架构出发，介绍了系统的工作原理。接着详细阐述了设计所采用的存内计算单元的工作原理并且介绍了混合并行加法器树与累加器的电路结构。然后对外围电路模块做了概述，依次介绍了位线驱动电路、时钟控制电路与字线译码器的电路结构与原理。最后基于

台积电 28nm 标准 CMOS 工艺搭建了该架构，对计算单元的功耗与稳定性做了详细仿真，同时对整个系统的性能与功耗进行了联合后仿真，最后列举了芯片的版图与存内计算设计的接口功能及时序。

第 4 章介绍了基于复用 Booth 计算单元和 6T SRAM 的可重构数字存内计算设计。首先从系统的架构出发，介绍了整个架构的组成及工作原理。接着详细阐述了系统的分时复用策略与可重构卷积运算的操作过程。然后介绍了关键的输入编码电路及 Booth 计算单元的电路原理。最后基于台积电 28nm 标准 CMOS 工艺对该架构进行了电路设计，对计算单元及整个系统的功能与性能进行了仿真，并比对了不同的工作条件下系统的能耗效率与吞吐率的变化。

第 5 章对本文的工作进行总结，分析了本文提出两种架构的优势及不足，并对未来的研究方向做出了展望。

第2章 存内计算电路设计基础

本章主要介绍存内计算电路设计所涉及到的基础知识、工作原理与设计指标，并总结了近几年已经提出的一些模拟存内计算设计的优点与缺陷，最后引出了数字存内计算的概念与实现方式，并简要地将模拟存内计算与数字存内计算的优缺点进行对比。

2.1 6 管 SRAM 单元概述

2.1.1 基本结构

如图 2-1 展示了 6 管 SRAM 单元的电路结构。它是由一个互锁的双稳态器（Latch）与两个负责传输数据的 NMOS（TNL 与 TNR）组成。Latch 由两对互补晶体管构成：上拉 PMOS 晶体管（PUL 和 PUR）以及下拉 NMOS 晶体管（PDL 和 PDR）。其中上拉 PMOS（PUL 和 PUR）负责写入逻辑“1”，下拉 NMOS（PDL 和 PDR）负责写入逻辑“0”。整个 Latch 结构负责稳定节点 Q 与 Q_N 存储的数值，其中节点 Q 是写入或读出的值，Q_N 是写入或读出的值的反向。TNL 与 TNR 负责选通需要操作的 SRAM 单元，其导通受字线 WL 控制。当 SRAM 单元进行读写操作时，字线 WL 开启，TNL 与 TNR 被导通，位线 BL 与 BLB 与 Latch 连通，进而通过位线传递要写和读的值。

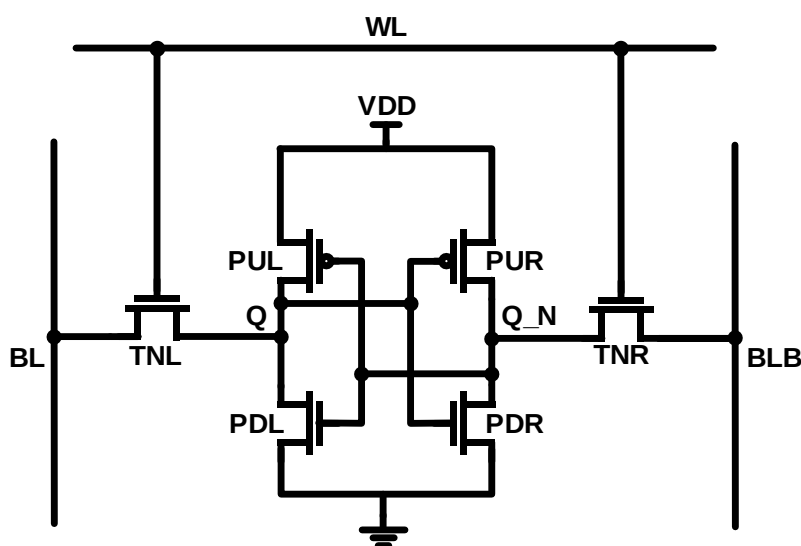


图 2-1 6T SRAM 单元电路结构图

2.1.2 6 管 SRAM 的保持操作

6 管 SRAM 处于保持状态时电路如图 2-2 所示，此时字线 WL 始终为逻辑低电平“0”，关闭 TNL 与 TNR 从而将位线与 Latch 隔断，位线 BL 与 BLB 预充电到逻辑高电平“1”。假设存储节点 Q 的值为“1”，Q_N 的值为“0”，Q_N 值导通左侧上拉管

PUL，对节点 Q 进行充电，Q 值导通右侧下拉管 PDR，对节点 Q_N 进行放电，从而保证了存储数据的稳定。

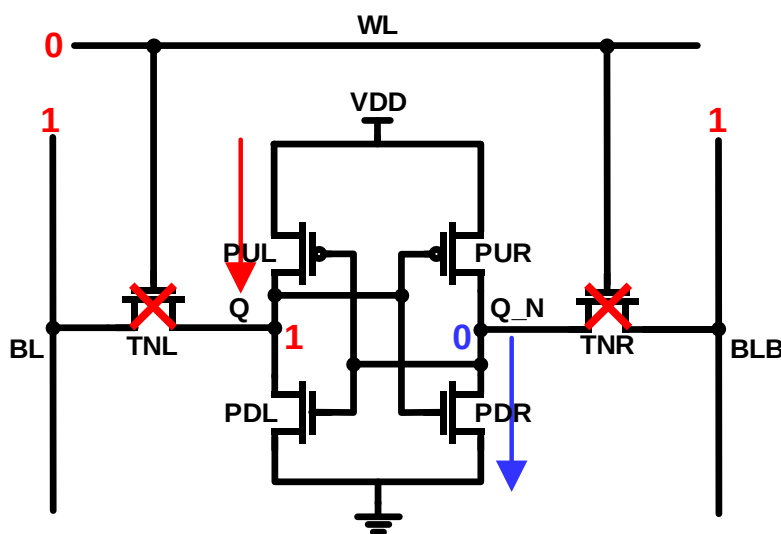


图 2-2 6T SRAM 单元的保持操作

2.1.3 6 管 SRAM 的写操作

如图 2-3(a)与 2-3(b)所示，分别是 6T SRAM 单元的写操作过程与写操作波形。假设 SRAM 单元在执行写操作之前内部存储的值为“1”，即 Q 的值为“1”，Q_N 的值为“0”。此时如果我们想往存储单元内写入“0”值，则将位线 BL 上的电平放电到 VSS，位线 BLB 上的电平充电到 VDD，反之亦然。随后开启字线 WL，导通传输管 TNL 与 TNR，两根位线会驱动 Latch 保存要写入的数据。在写“0”过程中，Q 点的电平由 TNL 与 PUL 之间的驱动能力强弱决定，Q_N 点的电平由 TNR 与 PDR 之间的驱动能力强弱决定。一般标准 CMOS 工艺下，相同尺寸的 PMOS 驱动能力要弱于 NMOS，因此 Q 点电平容易从“1”翻转到“0”，当 Q 的值变为“0”时，关闭右侧下拉管 PDR，导通右侧上拉管 PUR，对 Q_N 充电，因此 Q_N 从“0”翻转到“1”，SRAM 单元完成一次写“0”操作。

写容限（Write Margin）是衡量 SRAM 单元写入能力的重要指标^[52]，它定义为 BL 上可以施加的对应逻辑“0”的最大电压，当 BL 上的电平高于这一电压时，SRAM 单元无法完成写“0”操作。

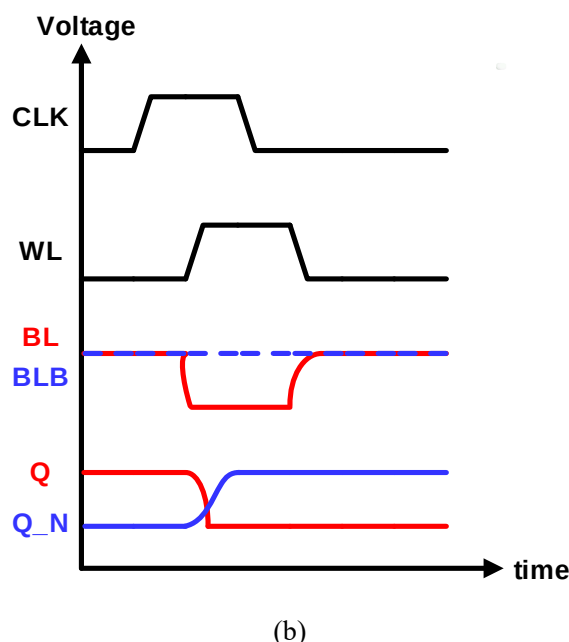
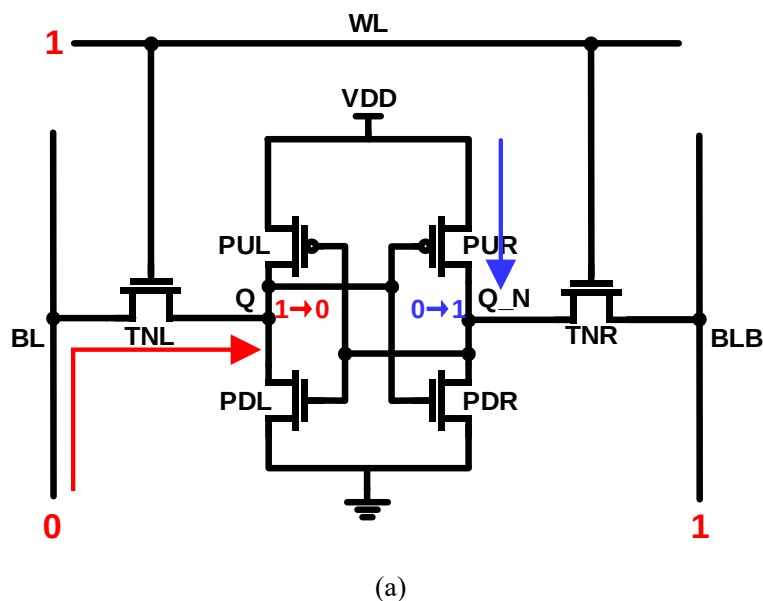
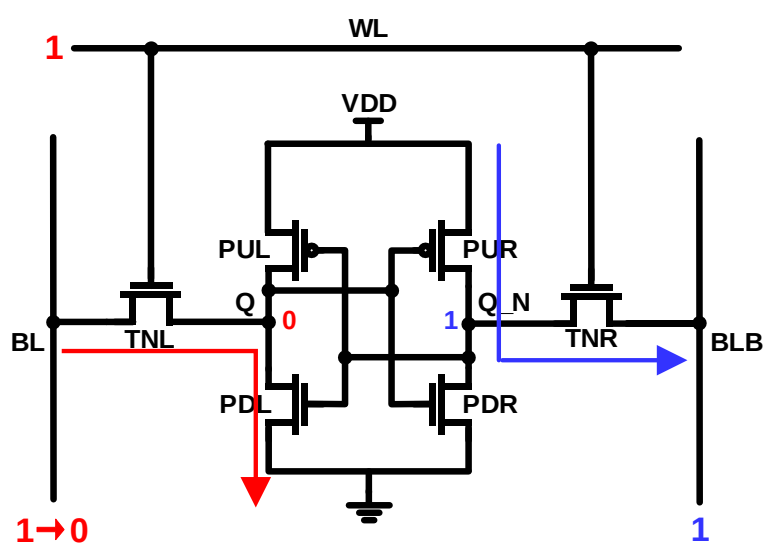


图 2-3 (a) 6T SRAM 单元的写操作 (b) 写操作时序图

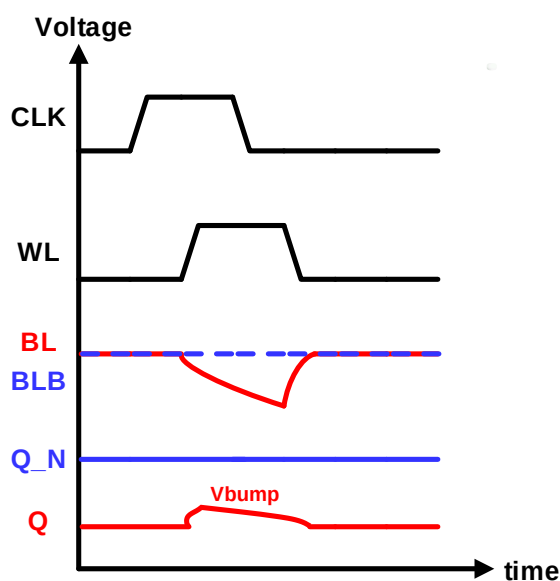
2.1.4 6 管 SRAM 的读操作

如图 2-4(a)与 2-4(b)所示, 分别是 6T SRAM 单元的读操作过程与读操作波形。假设 SRAM 内部存储的值为“0”, 即 $Q=0$, $Q_N=1$ 。在进行读操作之前, 我们预先将位线 BL 与 BLB 均预充电到 VDD, 当字线 WL 开启时, 传输管 TNL 与 TNR 被导通。由于 $Q_N=1$, 左侧下拉管 PDL 导通, BL 上的充电电荷经由 $TNL \rightarrow PDL \rightarrow VSS$ 的路径放电, 导致 BL 上的电平逐渐降低。由于 $Q=0$, 右侧上拉管 PUR 导通, BLB 上的电平经由 $VDD \rightarrow PUR \rightarrow TNR$ 的路径保持在 VDD。因此 BL 与 BLB 之间产生一个电压差, 后续可以通过灵敏放大器放大这个电压差从而读取 SRAM 单元存储的值。

SRAM 执行读“0”操作的过程中，Q 点的电平会随着 BL 上电荷对地的放电产生一个电压抬升（Voltage Bump），这个电压抬升过高会导致 SRAM 单元原本存储的“0”翻转到“1”，从而导致存储的数据被破坏。这一现象被称为读干扰（Read Disturb）^[52]。衡量 SRAM 单元在读过程中的抗干扰能力指标被称为静态噪声容限（Static Noise Margin）^[41]，其定义为在 SRAM 单元存储的值不被破坏的情况下能够在 SRAM 单元中的 Latch 上所能施加的最大噪声电压。要使 SRAM 单元抗干扰能力强，在设计时我们一般让 PDL 的驱动能力大于 TNL 且 PDR 驱动能力大于 TNR，这样在进行读操作过程时 Q 或 Q_N 节点的电压抬升较小，不会使存储节点的值产生翻转。



(a)



(b)

图 2-4 (a) 6T SRAM 单元的读操作 (b) 读操作时序图

2.2 模拟存内计算设计综述

2.2.1 模拟存内计算介绍

近年来已提出的存内计算架构多数采用模拟的方法实现。模拟存内计算设计按照存储器类型的不同又可以分为基于静态随机存储器（Static Random Access Memory, SRAM）的存内计算设计^[14,19-28]、基于动态随机存储器（Dynamic Random Access Memory, DRAM）的存内计算设计^[9]、基于忆阻器（Resistive Random Access Memory, ReRAM）的存内计算设计^[29-31]及基于磁随机存储器（Magnetic Random Access Memory, STT-MRAM）的存内计算设计^[44]。得益于 SRAM 存储器高读写速度的特点^[5]，其更加适用于神经网络计算这类高速应用场景，并且 SRAM 的工艺成熟度较高，其可以兼容更先进的工艺制程，从而让系统实现更高的集成度与性能，因此基于 SRAM 的模拟存内计算架构被广泛采取。

图 2-5 展示了基于 SRAM 的模拟存内计算设计，权重数据存储存储在 SRAM 阵列中，输入计算数据通过数模转换器（Digital to Analog Converter, DAC）将二进制的数字信号转换成模拟电压信号后，通过字线 WL 或位线 BL 输入到 SRAM 阵列中。SRAM 阵列开启多个单元执行乘法操作，乘法的结果反映在位线 BL 的电流上，通过 BL 的连接，每个单元产生的乘法电流累加在一起，最终在 BL 上可以得到乘加的结果。该乘累加结果经由模数转换器（Analog to Digital Converter, ADC）可以得到二进制数字域的计算结果。

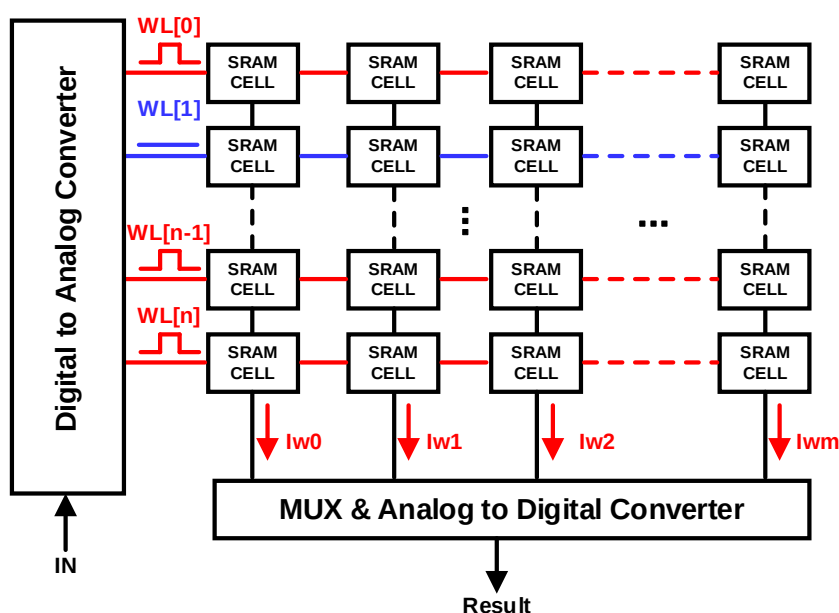


图 2-5 基于 SRAM 的模拟存内计算设计

2.2.2 模拟存内计算电路设计

存内计算电路的核心是存内计算单元，其主要由存储单元和计算单元组成。不同的存内计算单元所支持的计算类型也不同，导致整个存内计算电路设计适用的神经网络应用不同。目前已经提出的基于 SRAM 的模拟存内计算单元主要有：10T SRAM 计算单元^[25]、分列式字线 6T SRAM (Dual-split 6T SRAM)^[20]、12T SRAM 计算单元^[28]、分比特乘法计算单元 (Bit-wise multiply unit, BMU)^[21]等。接下来对以上列举的存内计算单元做简要介绍。

如图 2-6 所示，为 10T SRAM 计算单元电路图，其适用于单比特权重神经网络的应用。计算单元包含一个 6T SRAM 单元与 4 个 NMOS 管组成的两个读出端口，SRAM 单元负责存储 1 比特权重值，两个读出端口负责执行乘法运算。单比特权重神经网络内权重分为 +1 和 -1 两种状态，分别对于 6T SRAM 单元内存储的逻辑电平“1”与逻辑电平“0”。整个计算单元具有两种工作模式：存储模式与计算模式。在存储模式下，对 6T SRAM 进行读写操作，将权重数据写入到 6T SRAM 单元内并读出验证，这个过程与 2.1.3 与 2.1.4 节中提到的 6T SRAM 单元的写操作与读操作相同。在计算模式下，输入计算数据经由 DAC 转换成模拟电压量，并同时传送到两根读取位线 RBL 与 RBLB 上。当读取字线 RWL 被激活时，如果 SRAM 单元内存储的数据为“1”，则 RBL 上的电压会放电到 VSS，RBLB 上电压则保持不变，如果存储的数据为“0”，则情况相反。这样 RBL 和 RBLB 之间的电压差代表了输入计算数据与 1 比特权重的乘法结果。文献[25]基于 10T SRAM 单元搭建的存内计算架构可实现 BWN 并在基于手写数字的 MNIST 数据集上得到了 98.3% 的识别准确率，系统的能耗效率达到 28.1TOPS/W。但是架构具有以下局限性：(1) 10T SRAM 单元面积开销较大，降低了系统存储密度的同时提高了系统的功耗；(2) 应用局限于 BWN，不适用于多比特输入与多比特权重的卷积计算应用场景；(3) 输出数据精度高达 13bit，量化读出电路 ADC 的误差会影响计算精度。



13

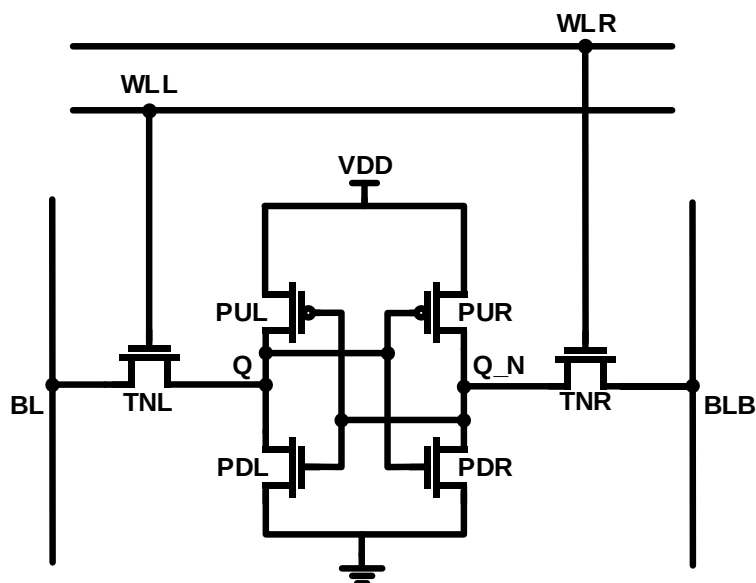


图 2-7 分裂式字线 6T SRAM 单元

如图 2-8 所示，为 12T SRAM 计算单元电路图，其适用于二元/三元深度神经网络计算加速的应用。12T SRAM 计算单元主要包含一个 6T SRAM 单元与 6T 计算单元。在存储模式下，6T SRAM 单元进行常规读写操作。在计算模式下，输入数据有三种情况：+1、0、-1，其转换成计算单元四根字线 RWL_P、RWLB_P、RWL_N、RWLB_N 上的电平，存储的权重值有 +1 与 -1 两种情况，通过导通相应的 MOS 管，在输出位线上产生充电电流、关断、放电电流三种状态，从而实现 XNOR 计算。文章 [28] 提出基于 12T SRAM 单元的存内计算架构，计算时所有计算单元高度并行进行计算使得整个系统的能耗效率高达 403TOPS/W，在基于手写数字的 MNIST 数据集上达到了 98.84% 的识别准确率，而在 CIFAR-10 数据集上达到了 88.78% 的准确率。但是该架构有以下局限性，如（1）12T SRAM 单元所用 MOS 管数目是 6T SRAM 单元的两倍，这导致系统的面积和功耗开销很大，降低了存储密度；（2）计算结果范围达到 -256~256，导致 ADC 无法满足精度要求，造成一定的计算误差；（3）过于复杂的计算单元提高了控制逻辑的复杂度，为系统带来了额外的面积与功耗开销。

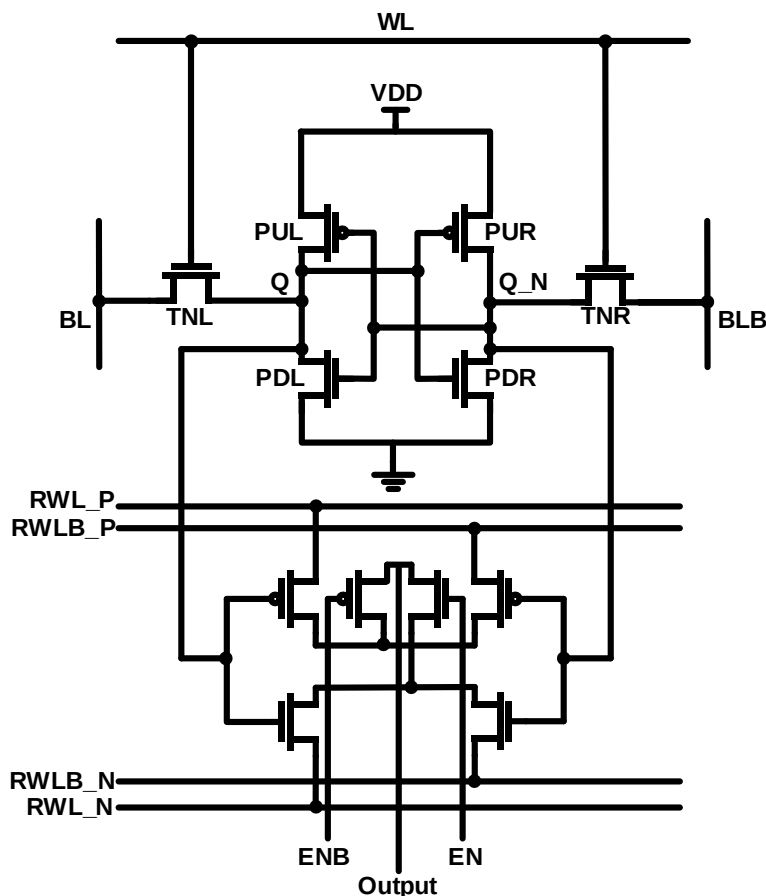


图 2-8 12T SRAM 计算单元

如图 2-9 所示，为分比特乘法计算单元（Bit-wise multiply unit, BMU）的电路图，其适用于多比特卷积神经网络计算加速的应用。BMU 由 16 个 6T SRAM 单元和一个局部计算单元（Local Computing Cell, LCC）组成。在存储模式下，它的读写操作与 6T SRAM 的读写操作一致，权重值从竖直方向的全局位线 VGBL 与 VGBLB 输入，通过横向字线 HWL 打开传输管 M1 与 M2，全局位线上的值经由传输管驱动输入到 SRAM 阵列的位线 BL 与 BLB 上，再通过开启字线 WL 将权重值写入相应的 SRAM 单元内。在计算模式下，HWL 关闭，关闭 M1 与 M2，字线 WL 开启，将进行运算的权重值通过 BL 输入到 LCC 单元中。4 比特输入数据被拆分为两组，分别转换成相应的电压送到全局位线 VGBL 与 VGBLB 上。VGBL 代表高两位输入数据，VGBLB 代表低两位输入数据。这样 M3 与 M5 上就会产生权重与高两位输入数据的乘法电流，M4 与 M6 上就会产生权重与低两位输入数据的乘法电流，并通过两条水平方向的全局位线 HGBL 与 HGBLB 与多个 BMU 的连接产生乘累加电流值。文章[21]提出基于 LCC 和 6T SRAM 单元的存内计算架构，可支持三种不同的数据运算精度：（1） $4b \times 4b$ ；（2） $4b \times 8b$ ；（3） $8b \times 8b$ 。系统达到了 $11.54\text{TOPS/W} \sim 68.55\text{TOPS/W}$ 的能耗效率，在对 CIFAR-10

数据集上可以达到最高 92.02% 的准确率，在对 CIFAR-100 数据集上可以达到最高 67.89% 的准确率。该架构通过分比特处理输入数据，降低了量化读出电路 ADC 的精度要求，极大地提高了系统的计算精度。

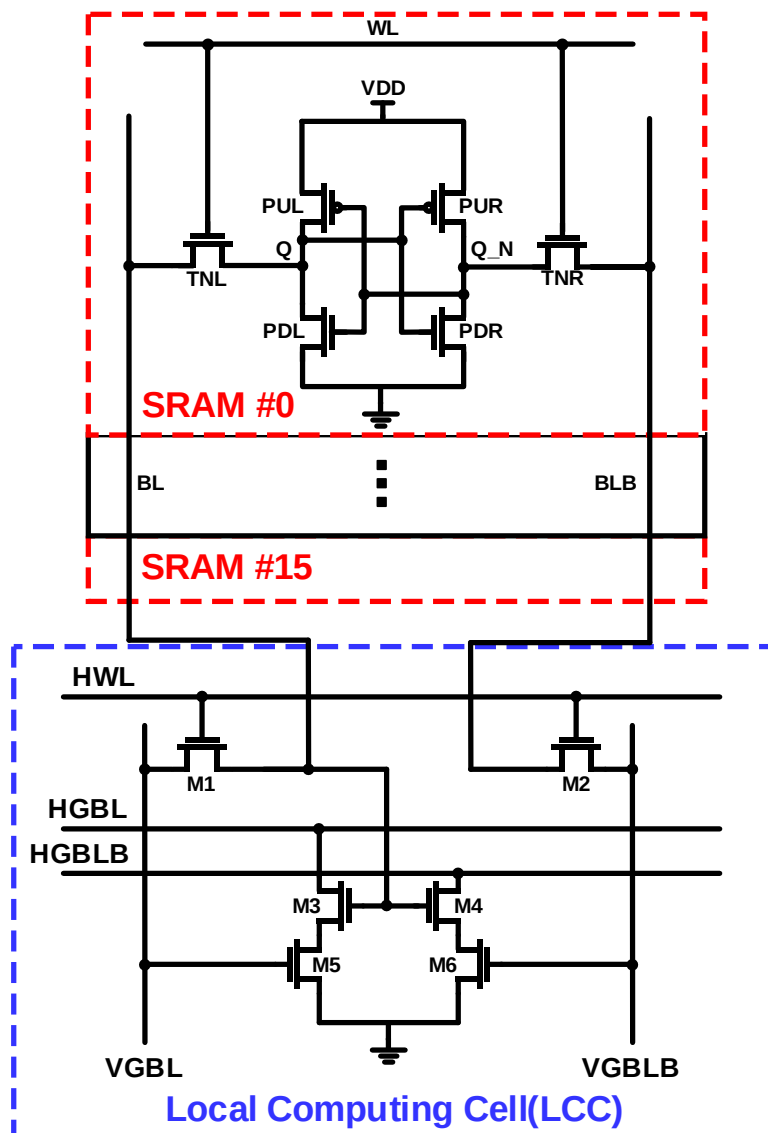


图 2-9 分比特乘法计算单元

2.2.3 模拟存内计算设计中的挑战

得益于在模拟信号域进行运算，模拟存内计算设计可以达到较高的计算速度，系统的性能与能耗效率表现良好^[52]。但是随着神经网络规模的扩大，在实际应用中需要处理的数据量也越来越大，这就要求存内计算设计支持更高的运算数据量。同时现在应用场景算法迭代速度快，要求存内计算设计可以根据应用灵活配置运算规模或运算类型^[39]。面对以上两种需求，模拟存内计算设计会碰到以下一些问题：（1）提高系统的运算规模要求存内计算阵列在运算时开启的单元数量更多，可能会产生读干扰的

问题从而覆盖存储单元内的数据^[42]；(2) 若输出数据的位宽过高，会导致量化读出电路造成过大的面积与功耗开销，限制系统的能耗效率^[42]；(3) 量化读出电路 ADC 的输出精度有限，当执行更高精度的运算时容易产生计算误差^[42]；(4) 量化读出电路 ADC 的精度易受 PVT 及噪声的影响^[43]；(5) 模拟存内计算一般不支持配置，电路设计由应用场景和运算规模所决定，系统的灵活性较差^[41]。综上，模拟存内计算虽然可以实现很高的运算速度与较好的能耗效率，但是面对较大的运算数据量时，其需要额外的电路与架构设计去解决存在的一些问题。同时模拟存内运算设计的运算精度始终受限，并且系统的可重构性较差，其不适用于运算精度要求较高或者算法迭代频繁的应用场景。

2.3 数字存内计算设计综述

2.3.1 数字存内计算电路设计

数字存内计算设计是在数字域内进行运算操作^[13,15-18]，权重值仍然预先存储在 SRAM 阵列内，存内计算宏电路通常采用逻辑运算单元，并引入额外的运算电路对数据进行处理从而输出运算结果。数字存内计算设计不受 PVT 及噪声的影响，可以达到 100% 的运算精度。近年来，有关数字存内计算设计的研究工作也在不断推动中。

文章^[56]提出一种基于双字线 6 管 SRAM 单元的数字存内计算 Cache 结构，可以在 Cache 内实现与 (AND) 运算、或非 (NOR) 运算、异或 (XOR) 运算及清零等操作。如图 2-10 所示，在运算开始前将所有位线 BL 与 BLB 预充电到逻辑高电平“1”。进行运算时选择激活 BL 或 BLB 可以实现不同的逻辑运算。开启任意两行的字线 WLL 来激活 BL，若开启的两行存储单元内存储值均为“1”，则 BL 上的电压保持不变，通过采样输出逻辑高电平“1”，若其中一个存储值为“0”，则 BL 上的电压会被拉低到参考电压 V_{ref} 以下，则采样输出逻辑低电平“0”。综上，激活位线 BL 可以实现逻辑与 (AND) 运算，同理激活位线 BLB 可以实现逻辑或非 (NOR) 运算。通过组合中间运算结果还可以实现异或 (XOR) 等逻辑运算。提出的数字存内计算 Cache 结构减少了 CPU 与 Cache 之间的数据传输开销，并且可以实现简单的逻辑运算，但是由于 Cache 内部的运算类型较为简单，在实际应用中的效果并不显著。

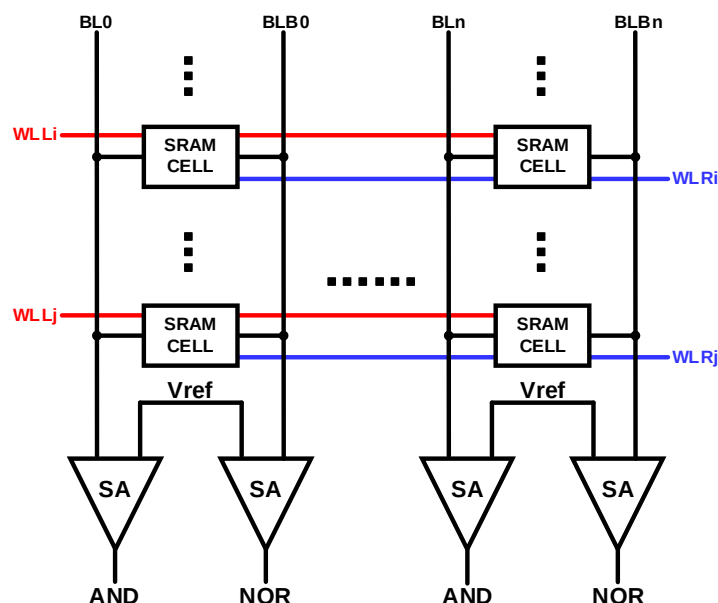


图 2-10 基于双字线 6 管 SRAM 单元的数字存内计算 Cache 结构

为了面向更复杂的应用场景，文章[57]在文章[56]的基础上提出一种可应用于神经网络的大规模并行计算数字存内计算 Cache 结构，通过位串行的方式在 Cache 内实现加法运算。如图 2-11 所示，采样模块输出的逻辑与（AND）运算结果和逻辑或非（NOR）运算结果输出到逻辑运算单元中，依据进位输入 C_{in} 的值，输出运算和结果 SUM 及输出进位结果 C_{out} ，并将进位寄存并传递给高位算子继续进行累加运算。通过配置合理的数据存储映射方案，该 Cache 结构可以运算大规模网络的加法运算。这种数字存内计算结构进一步扩展了 Cache 内部可进行的运算类型，但是由于逐位递进的运算模式，导致整体运算的速率较低，且该结构仅支持加法运算，无法应用于主流的卷积神经网络加速中。

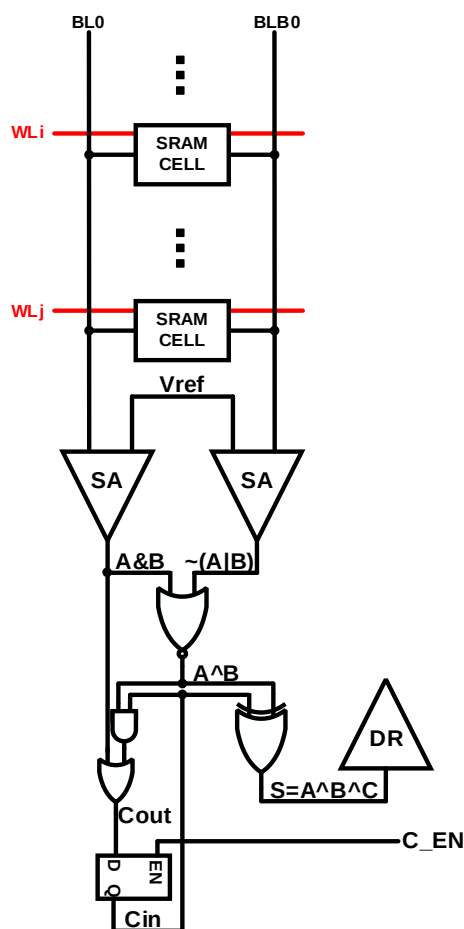


图 2-11 面向大规模并行加法运算的数字存内计算 Cache 结构

2.3.2 面向多比特卷积计算的数字存内计算实现方法

为了在存储模块内实现多比特卷积运算，网络中的权重值仍需预先存储在 SRAM 阵列内，然后将多组输入计算数据同时并行输入到 SRAM 阵列中的每一行与权重值相乘得到部分积，最后通过加法器树将这些部分积累加后得到卷积运算结果。如果每一组输入数据采取位并行的方式输入^[33]，如图 2-12 所示，以 4 比特输入和 4 比特权重举例，要实现乘法需要搭建 4×4 规模的乘法器，这会导致系统的面积和功耗开销迅速增大，从而严重限制了系统的能耗效率。为了尽量降低乘法运算实现的复杂度，在数字存内计算设计中输入数据一般采用位串行输入的方式^[32-34]，如图 2-13 所示，每个操作周期内每一组输入数据从高位到低位每次输入 1 比特与权重值相乘，这个乘法操作可以通过逻辑电路的方式实现，然后将多组相乘得到的部分积通过加法器树求和，最后输入到累加器中做移位累加，在多个操作周期内得到卷积计算的结果。

数字存内运算设计通过在数字域内实现卷积运算操作，运算精度可以达到 100%，且运算精度不受 PVT 与噪声等因素的影响。同时数字存内计算通过电路设计支持灵活

配置卷积运算规模或卷积运算类型^[45]，系统的可重构性较高。但是数字存内计算由于使用并行加法器树来处理中间积，当运算规模较大时，并行运算的组数增加，会导致加法器树的面积与功耗增大，限制系统的能耗效率，而且由于系统采取输入数据位串行输入，导致系统完成一次完整的卷积运算所花费的时间较长，速度方面也存在一定的劣势。因此在实际设计中，如何提高数字存内计算设计的能耗效率需要综合考量多方面因素，例如（1）加法器树的规模与速度；（2）系统并行运算的数量；（3）存内计算阵列的功耗等。

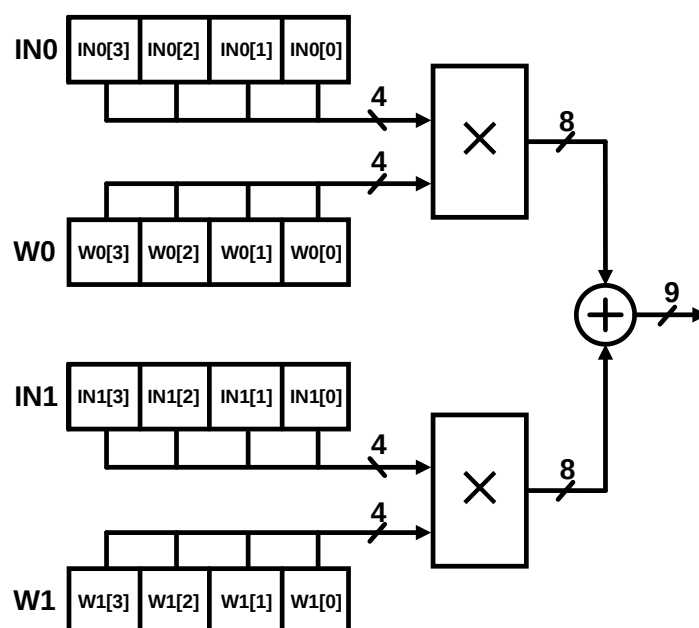


图 2-12 位并行输入卷积计算流程图

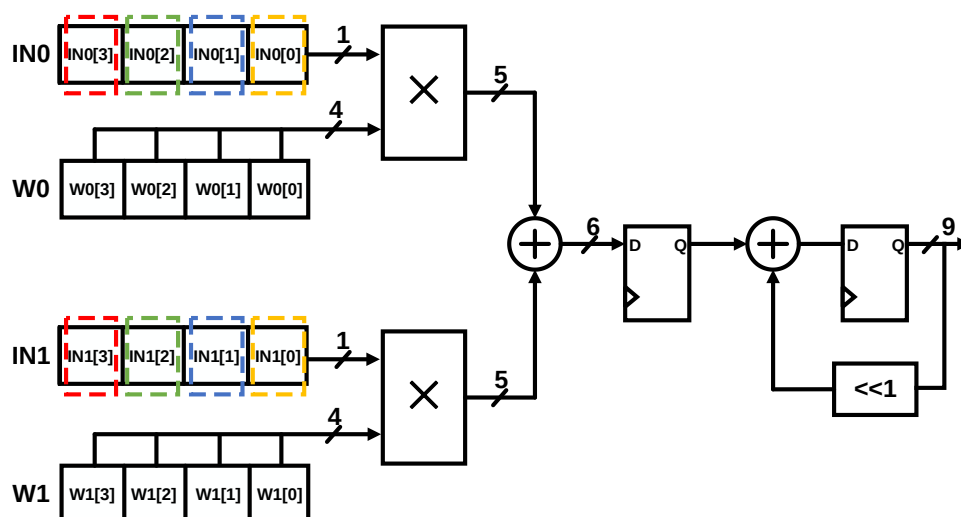


图 2-13 位串行单比特输入卷积计算流程图

2.4 存内计算电路设计指标

存内计算电路设计的评价指标主要有以下几个：

1.吞吐率：吞吐率（Throughput），用来衡量系统的计算速度，定义为每秒内系统所能执行的运算次数，单位通常为 GOPS（每秒进行 10 亿次操作）与 TOPS（每秒进行 10000 亿次操作）。

2.系统所支持的运算精度，模拟存内计算设计中运算精度受量化读出电路 ADC 的精度影响，数字存内计算设计中运算精度能达到 100%。

3.系统所支持的运算类型，例如同或运算、单比特乘累加、多比特乘累加（卷积运算）等。

4.能耗效率：能耗效率（Energy Efficiency）是综合衡量存内计算电路设计的参数，其定义为系统单位时间内所能执行的运算次数跟所消耗的能耗的比值，即吞吐率比上系统功耗，能耗效率公式如下：

$$\text{Energy Efficiency} = \frac{\text{Throughput}}{\text{power consumption}} \quad (2-1)$$

能耗效率同时与系统的吞吐率与功耗有关，系统计算速度越快且消耗的能耗越低，能耗效率的值就越高，其单位为 TOPS/W 或 GOPS/W。

2.5 本章小结

本章首先对基本的 6 管 SRAM 单元结构和工作原理进行了阐述，介绍了 6 管 SRAM 单元的保持操作、读操作与写操作。紧接着对模拟存内计算设计做了简单的综述汇报，整理了近年来的一些模拟存内计算单元并阐述了他们的原理，同时指出了模拟存内计算设计中会遇到的一些挑战。然后对数字存内计算设计做了简要的概述，提出了数字存内计算设计的实现方法与其优缺点。最后整理了存内计算电路需要考虑的一些设计指标。

第3章 基于 6T SRAM 的串行单比特输入存内计算设计

根据第 2 章提出的数字存内计算实现方法, 本章提出一种基于 6T SRAM 的串行单比特输入全数字存内计算设计, 适用于卷积神经网络计算应用场景。它主要采用了 (1) 串行单比特输入的数据输入模式 (2) 由 6T SRAM 与 NOR 组成的存内计算单元完成相乘运算; (3) 11T 全加器与 28T 全加器交替的加法器树来降低加法器树的面积与功耗; (4) 移位累加器电路完成流水线式卷积运算。设计基于台积电 28nm 工艺, 系统内存算阵列大小为 64Kb, 系统支持输入数据与权重值的符号位配置, 其工作频率达到 500MHZ, 计算精度为 100%, 同时获得了 917GOPS 到 3300GOPS 的吞吐率与 15.67TOPS/W 到 56.39TOPS/W 能耗效率。

3.1 串行单比特输入数字存内计算设计架构

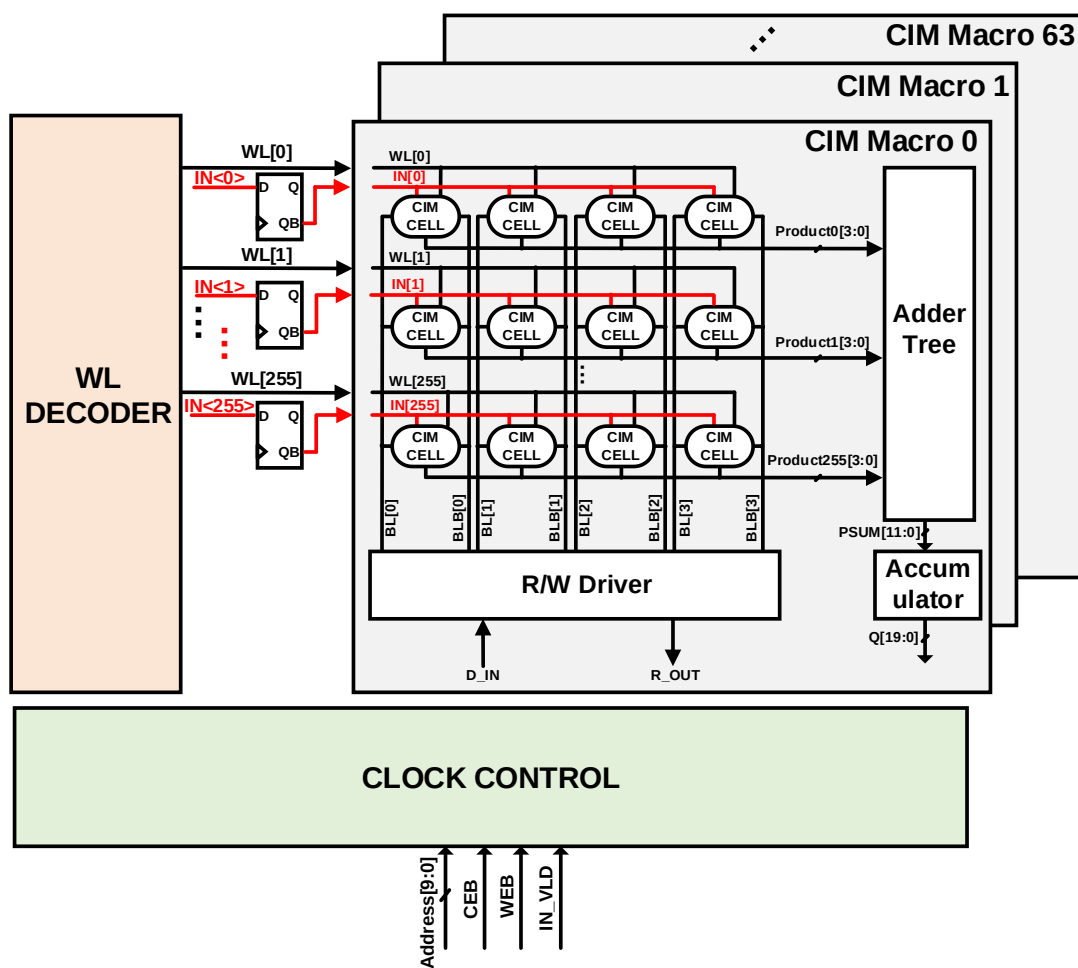


图 3-1 串行单比特输入存内计算设计架构

数字存内计算设计一般是通过流水线的方式将多比特输入与多比特权重的卷积运

算在多个周期内实现。图 3-1 展示了串行单比特输入数字存内计算设计整体架构，整个架构主要由三个模块组成：存内计算处理器（Computing in memory Macro, CIM Macro）、字线译码器（WL Decoder）与时钟控制模块（Clock control）。

系统内共有 64 个存内计算处理器，每个存内计算处理器内包含 1Kb 大小的存内计算阵列、阵列读入读出驱动电路、加法器树电路以及累加器电路。存内计算阵列由 256 行×4 列规模的存内计算比特单元组成，这个计算单元由标准的 6 管 SRAM 单元与 1 个或非门单元构成，负责完成单比特数的相乘操作。加法器树电路负责累加存内计算阵列产生的中间积从而得到部分和 PSUM，累加器电路负责对部分和 PSUM 做移位累加处理最终输出 20 位的卷积结果 Q。

时钟控制模块负责通过处理外围的输入信号从而输出相关控制信号驱动字线编码模块与存内计算处理器进行相应的操作。重要的外围信号分别有：（1）Address[9:0]，10bits 地址信号，其中高 8 位为行地址信号，低 2 位为列地址信号；（2）CEB，片选信号，低电平有效，CEB 为高时，存内计算阵列处于保持模式，其不进行任何操作，CEB 为低时，存内计算阵列被激活，可进行权重的读写；（3）WEB，读写控制信号，WEB 为低时，存内计算阵列进入写模式，WEB 为高时，存内计算阵列进入读模式；（4）IN_VLD，计算控制信号，高电平有效，当 IN_VLD 为低电平时，系统处于存储模式，当 IN_VLD 为高电平时，系统处于计算模式。

根据外围的输入信号，系统可在两种工作模式（存储模式与计算模式）间进行切换。当系统处于存储模式时，所有输入 IN 始终被置位高电平，每一行存内计算阵列输出的中间积 Product 均为零，加法器树与累加器均不进行翻转。此时存内计算阵列与正常 SRAM 阵列功能一样具有写权重值、读权重值、保持权重值三种模式。在写模式下，读写驱动电路输入权重值 D_IN，字线译码电路解译相关地址信号开启存内计算阵列某一行的字线 WL，将权重值刷新到存内计算单元中。在读模式下，存内计算阵列的位线（BL 与 BLB）首先预充电到高电平，字线的开启方式与写模式下相同，然后将存内计算单元中的权重值通过位线读入到读写驱动电路里从而输出权重值 R_OUT。在保持模式下，保证 256 根字线 WL 始终关闭并且位线上始终预充电到高电平，从而保证存内计算单元中的数据稳定。而当系统处于计算模式时，存内计算阵列始终处于保持模式从而确保存储权重值的稳定，触发器被激活，对 256 组输入 IN 进行采样，每一组输入 IN 的值是 8 比特卷积计算数据的某一位。在一次完整卷积计算开始

时, 8bits 的计算数据从高位到低位每次输入 1 比特依次逐位采样输入到存内计算架构中, 与存内计算阵列存储的权重值进行相乘, 得到 256 组 4bits 的中间积, 然后这些中间积输入到并行加法器树电路中进行累加, 得到 12bits 的部分和 PSUM, 保存在寄存器内, 完成卷积运算流水线的的第一级操作。流水线的第二级操作是在累加器模块中对前一周期的部分和进行移位并与当前周期产生的部分和进行累加。由此, 架构通过二级流水线的方式, 在多个周期内输出了 256 组输入计算数据 $IN[7:0]$ 与权重 $W[3:0]$ 的卷积运算结果 $Q[19:0]$, 简化了卷积操作在硬件上实现的复杂度, Q 的表达式为:

$$Q = \sum_{i=0}^{256} (IN_i[7:0] \times W_i[3:0]) \quad (3-1)$$

3.2 串行单比特输入存内计算电路设计

本节主要介绍架构中各个模块的具体电路设计, 从存内计算处理器内部的电路开始阐述, 详细描述了存内计算比特单元、并行加法器树、累加器电路的工作原理, 然后着重介绍了时钟控制模块内部的电路设计, 解释了系统的时序控制方法, 最后对字线译码器的电路结构做了简述。

3.2.1 基于 6T SRAM 的存内计算单元设计

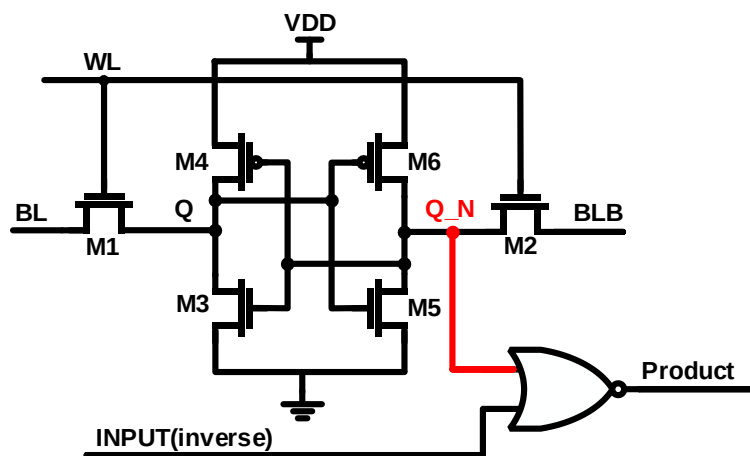


图 3-2 基于 6T SRAM 的存内计算比特单元

如图所示, 存内计算比特单元由 6 管 SRAM 单元与双输入或非门单元组成。6 管 SRAM 单元由两个传输数据的 NMOS (M1 与 M2) 以及一对交叉耦合的反相器 (M3、M4 与 M5、M6) 组成, 根据 SRAM 读写操作的原理, 设计管子的尺寸为 $M3 > M1 > M4$ ($M5 > M2 > M6$)。单元节点 Q 与 Q_N 为 SRAM 存储的数据, 其中 Q_N 为双输入或非门的其中一端输入, 或非门输入的另一端接到外部输入 INPUT 上, INPUT 为计算

数据的取反值，由布尔代数可以得到 $(IN_B+Q_N)'=(IN_B)' \cdot (Q_N)'=IN \cdot Q$ ，从而实现输入与存储值的相乘并输出中间积 Product。

当整个系统处于存储模式时，由于 INPUT 采样电路不使能，INPUT 始终保持在高电平，所以存内计算单元内或非门的输出 Product 始终为 0，不受存储节点 Q_N 的影响。此时存内计算比特单元可以看成是一个节点 Q 与 Q_N 阻抗不匹配的标准 6 管 SRAM 单元。由于或非门单元在设计时选取最小尺寸，其对 SRAM 单元的读写噪声容限几乎不产生影响，所以可以通过 SRAM 常规的写操作将卷积神经网络的权重值写入存内计算比特单元。

当系统进入计算模式后，首先外围的电路会保证存内计算单元的字线 WL 始终关闭并且位线 BL 与 BLB 均预充电到高电平，从而保证存储节点 Q 与 Q_N 值的稳定。然后 INPUT 采样电路使能，将输入信号采样到计算单元中进行相乘运算，表 3-1 展示了存内计算单元进行计算操作时的真值表。

表 3-1 存内计算单元的操作真值表

INPUT	Q_N	Product
0	0	1
0	1	0
1	0	0
	1	

3.2.2 并行加法器树电路设计

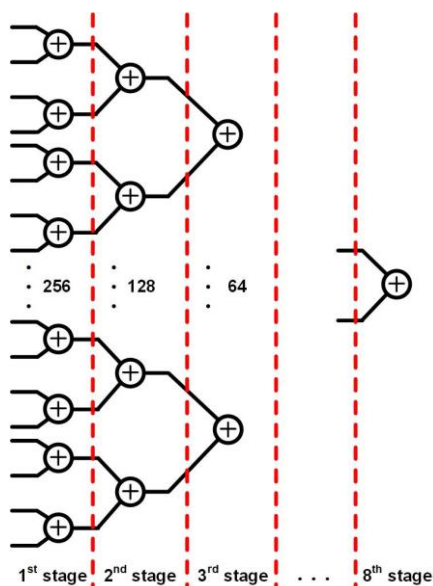


图 3-3 并行加法器数电路示意图

图 3-3 展示了并行加法器树电路的架构，256 组数据并行输入到第一级加法器中两两进行求和运算，得到的 128 组计算结果紧接着输入到下一级加法器中再一次两两求和，逐级递进，直到输出最终的累加值。由于每一级运算可以减少一半的运算组数，所以整个加法器树需要经过 $\log_2 256=8$ 级才能输出累加值。

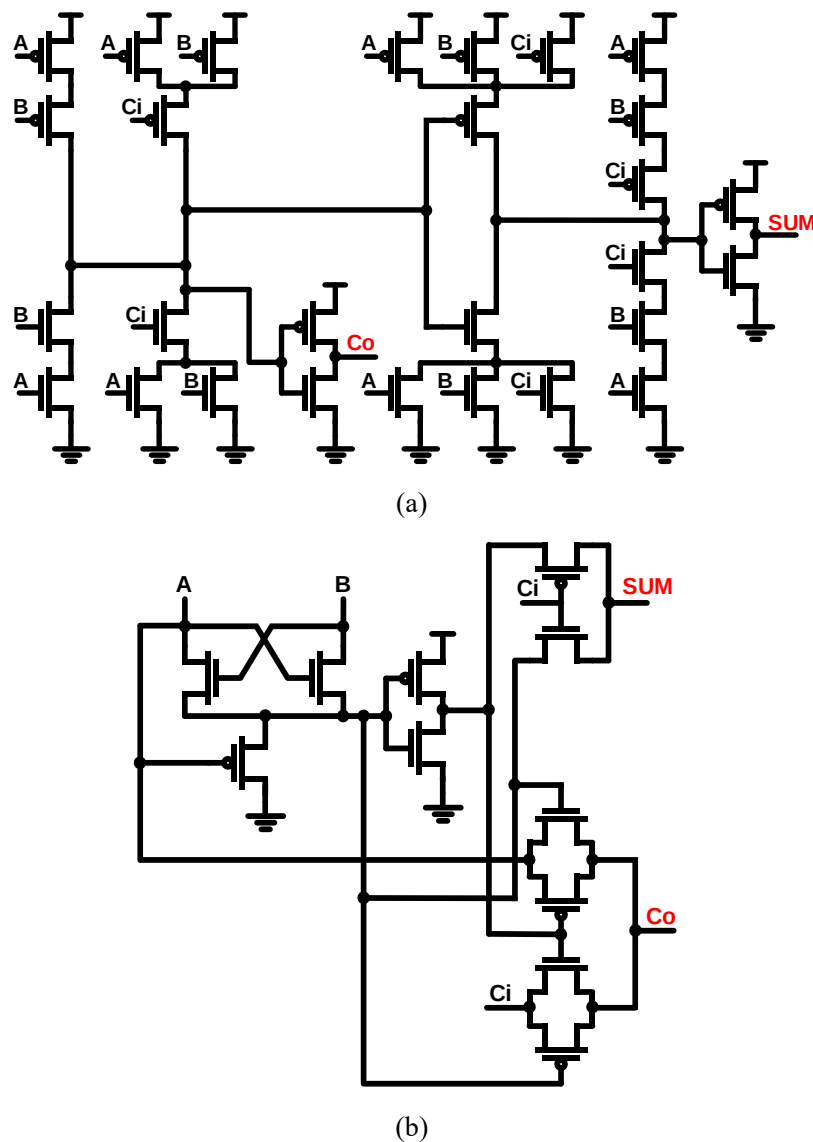


图 3-4 两种全加器电路结构：(a)28 管全加器电路 (b)11 管全加器电路

由于并行加法器树的规模较大，如果内部的全加器单元全部采用基本的 28 管全加器来搭建，会导致加法器数的功耗和面积变得很大，直接影响了存内计算系统的功耗与能耗效率，因此需要对加法器单元进行设计优化。本设计采用 11 管全加器^[48]来替代一部分 28 管全加器的方法来节约电路的面积与功耗，两种全加器的电路结构如图 3-4(a)与图 3-4(b)所示。可以看出，相比于传统的 28 管全加器，11 管全加器对一部分的计算逻辑进行了简化，使其不仅面积较小，而且还具有延迟低、功耗低等优点，非常

适用于高能效比加法器树的设计。但是 11 管全加器也具有阈值损失的缺点，如果全采用 11 管全加器搭建并行加法器树，在加法器单元多级级联的情况下，可能会导致后级加法器单元的输出电压产生较大的阈值损失，从而发生计算结果错误的情况。因此，最终采用 11 管全加器与 28 管全加器交替运作的方式实现并行加法器树，如图 3-5 所示为加法器树中的某两级，不仅在横向上采用 2 种全加器交替的方式实现加法器单元，也在纵向上采用交替的方式实现加法器树内的递进，充分降低加法器树的面积与功耗，提高存内计算架构的能效比。

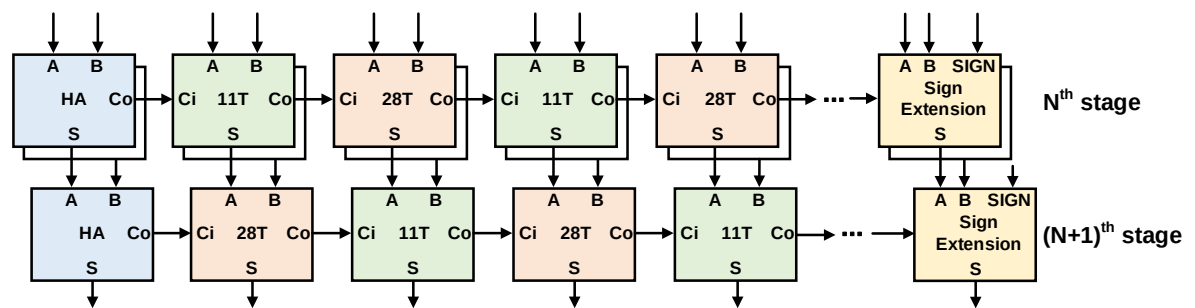


图 3-5 11T 与 28T 交替运行的加法器树结构

图 3-5 中多比特加法器链的开头均是一个半加器，而末尾均是一个符号位扩展电路。如图 3-6 所示，符号位扩展电路是一个由符号标志位 **SIGN** 控制的全加器电路，输入 **A** 与 **B** 分别是加数的最高位，**Ci** 是最高位求和产生的进位。当 **SIGN=0** 时，加法器树进行无符号数的求和运算，此时图 3-6 中红框内的反相器使能，扩展位即为进位信号 **Ci**，当 **SIGN=1** 时，图中蓝框内的反相器使能，与前级电路构成全加器，将 **A**、**B**、**Ci** 进行相加，来产生符号位扩展值，符号位扩展电路的操作真值表如表 3-2 所示。

表 3-2 符号位扩展电路的操作真值表

A	B	Ci	SUM(SIGN=0)	SUM(SIGN=1)
0	0	0	0	0
0	0	1	1	X
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	0	X
1	1	1	1	1

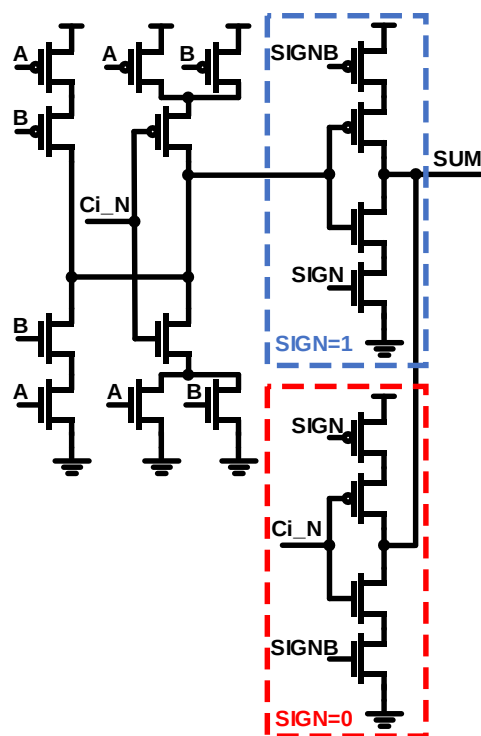


图 3-6 符号位扩展电路

3.2.3 累加器电路设计

累加器电路是实现流水线型卷积运算的关键电路模块，其需要完成三个操作：（1）采样当前周期加法器树的输出 PSUM，并对 PSUM 进行位扩展；（2）对上一周期的得到的累加值 SUM 向左移一位；（3）将位扩展后的 PSUM 与移位后的 SUM 相加，并在下一周期采样得到最终输出 Q。综上，本文设计的累加器电路结构如图 3-7 所示。

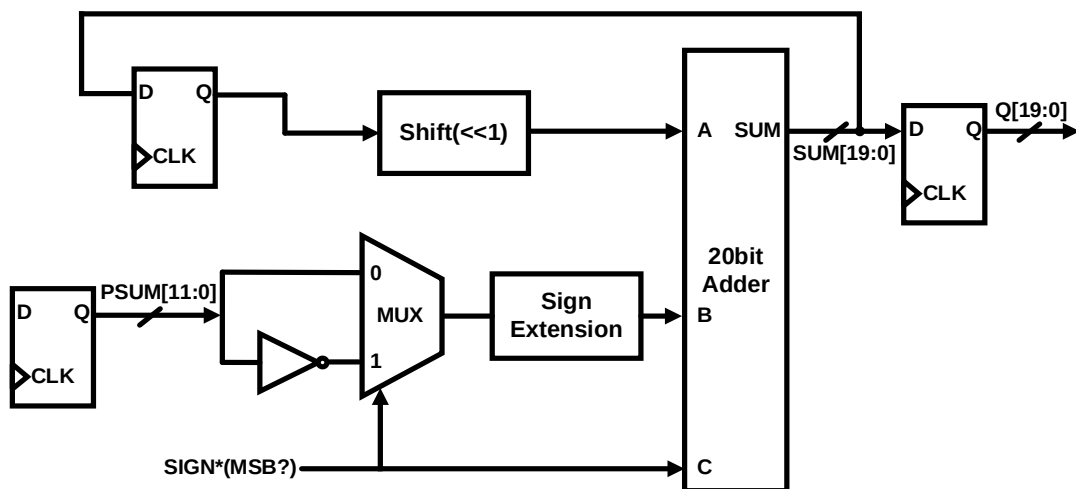


图 3-7 累加器电路

累加器电路对 PSUM 做位扩展操作时，需要先根据符号标志位 SIGN 与输入计算

数据的最高位标志信号 **MSB** 的值来判断进行何种操作。如果符号标志位 **SIGN** 为低电平，则证明输入计算数据为无符号数，不论当前处理的是输入计算数据的哪一位，都只需要对 **PSUM** 做零扩展操作。如果符号标志位 **SIGN** 为高电平，则说明输入计算数据为有符号数，若此时 **MSB** 也为高电平，则说明当前处理的是输入数据的最高位与权重的乘累加和 **PSUM**，又因为输入数据为有符号数，即最高位为符号位，又因为 0 符号位与权重相乘不对乘累加和 **PSUM** 产生贡献，则此时的 **PSUM** 可以理解为部分权重值的累加和乘上 -1，因此需要对 **PSUM** 取补码，才能进行求和操作。所以当 **SIGN*MSB** 为高电平时，对 **PSUM** 值取反，并且 20bits 加法器求和时多加了一个逻辑高电平，从而实现取补码的操作。

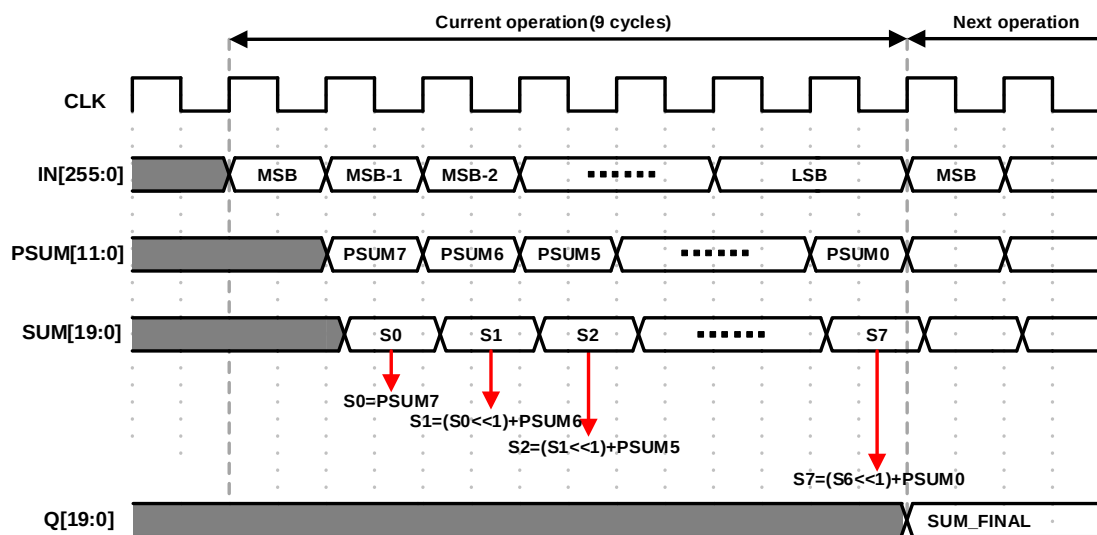


图 3-8 卷积计算时序图

整个架构内卷积运算的时序如图 3-8 所示，256 组 8 比特输入计算数据分别从高位（MSB）到低位（LSB）每次输入 1 比特逐位输入到架构内。数据在时钟上升沿处被采样，经过存内计算阵列与加法器树电路处理后，累加器在下一个时钟上升沿采样部分和 **PSUM**，并将上一周期产生的累加和 **SUM** 向左移一位，再将两者求和。经过 9 个周期的反复迭代，输出最终的卷积计算结果 **Q**。可以看出，对于 **N** 比特的输入数据，整个卷积运算操作需要 **N+1** 个周期才能完成，因为要多预留一个周期给累加器做最后一步移位累加操作。

3.2.4 读入读出驱动电路设计

读入读出驱动电路具备两种功能：（1）对阵列进行写权重操作时，驱动电路根据

控制信号将输入权重值赋值到位线上；(2) 对阵列进行读权重操作时，将位线上的电平信号进行差分放大，确保将权重稳定读出，并且在不进行读操作时，不论位线上电平如何变化，输出端的信号要保持稳定，避免信号出现不定态。

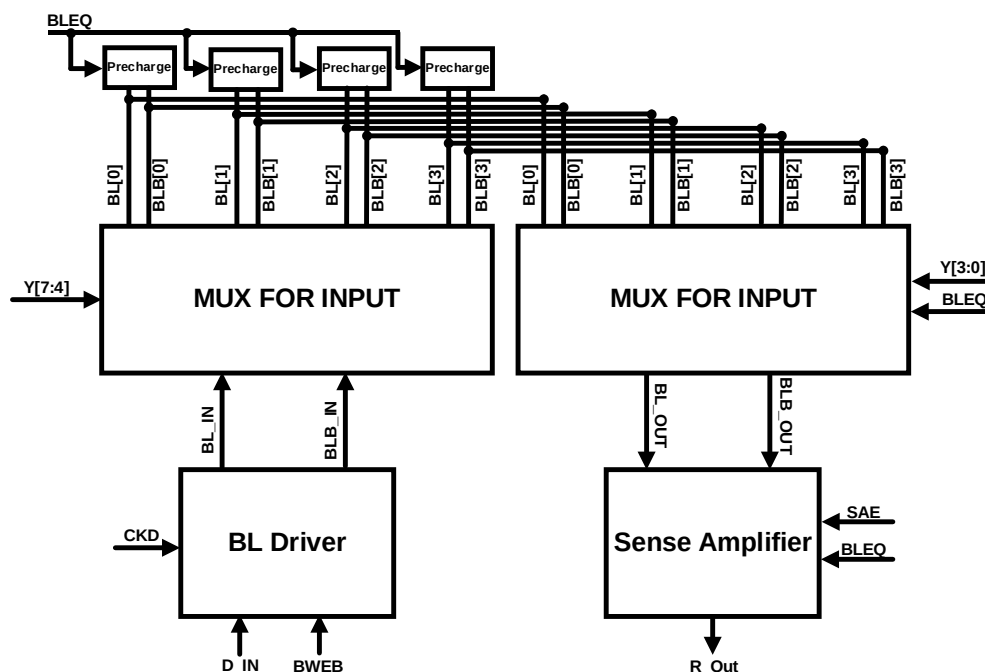


图 3-9 读入读出驱动电路结构示意图

如图 3-9 所示，为读入读出驱动电路模块的架构图，其内部又包含 5 个子电路单元，分别是位线驱动电路、输入多路选择器电路、输出多路选择器电路、灵敏放大器电路以及位线预充电电路。阵列进行写入操作时，CKD 信号为高电平，当列选信号 BWEB 选中时，位线驱动电路使能权重值 D 输入，并将其差分输出到输入多路选择器中，多路选择器根据独热信号 Y[7:4] 的值，将 BL_IN 与 BLB_IN 的值赋值到四组位线的其中一组，从而写入存内计算单元中。阵列进行读操作时，存内计算单元先将存储的 4 比特权重值读取到四组位线上并输入到输出多路选择器中，同样多路选择器根据独热信号 Y[3:0] 选择一组位线输出到灵敏放大器中进行信号放大，最终读出 1 比特的权重值。当阵列处于保持模式时，读入读出驱动模块内只使能位线预充电电路，将所有位线预充电到高电平，从而保证存储单元内数据的稳定。

位线驱动电路如图 3-10 所示，写驱动信号 CKD 为低电平时，将输入权重值 DL_IN 与列选信号 BWEB 预采样到电路里，产生 DXL 与 BWEBXL 信号，当 CKD 信号为高电平时，DXL 与 BWEBXL 通过反馈回路维持信号稳定，同时 DXL 与其反向信号分别与 CKD、BWEBXLB 通过两个与非门逻辑单元，从而将 DXL 的值差分输出产

生 BL_IN 与 BLB_IN。当列选信号 BWEB 为高电平时，即该列未被选中，则不论 CKD 与 D_IN 如何翻转，输出 BL_IN 与 BLB_IN 始终为高电平。位线驱动电路中预采样电路的工作时间决定了 D_IN 与 BWEB 需要在 CKD 上升沿多久之前到达，即为输入信号 D_IN 与 BWEB 相对于 CKD 的建立时间。

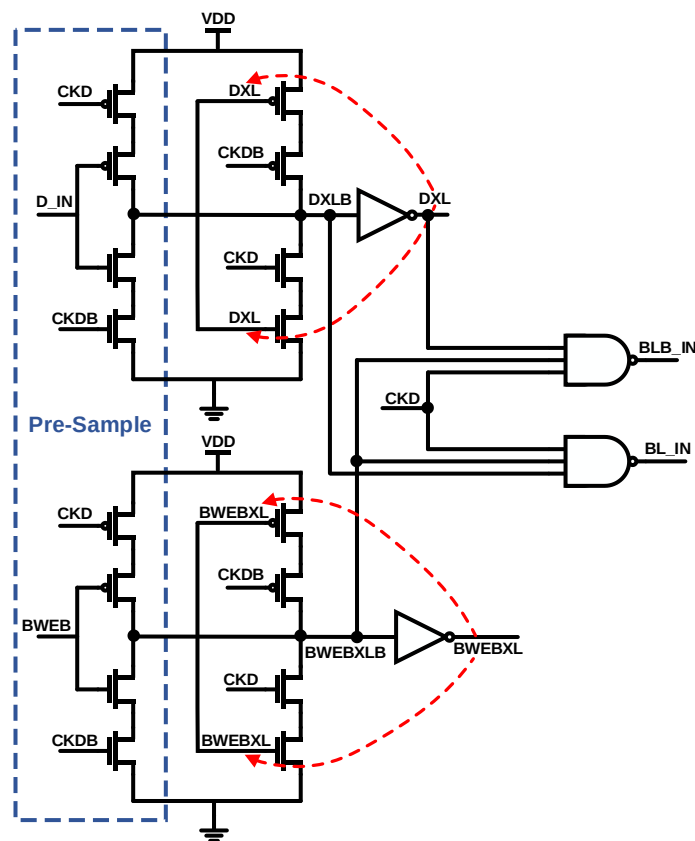


图 3-10 位线驱动电路

多路选择器电路可以减少输入与输出的端口数量，对于大规模存内计算阵列其大大节省了外围电路消耗的面积，然而由于分时选择位线写入的机制，导致存内计算阵列需要更多的周期完成权重值的刷新。如图 3-11 (a) 与 3-11 (b) 所示，分别为输入多路选择器与输出多路选择器的电路示意图。输入多路选择器由独热信号 Y[7:4]选择开启，当选中某一对 NMOS 管开启时，将总线 BL_IN 与 BLB_IN 的值赋值给四组位线的其中一组，每一组位线 BL[n]与 BLB[n]通过交叉耦合的连接方式来弥补 NMOS 管传输时造成的电压损失，确保写入权重值的正确性。输出多路选择器根据独热信号 Y[3:0]选择开启其中一对 PMOS 管，从而将四组位线上的值选择输出到总线 BL_OUT 与 BLB_OUT 上，接着 BL_OUT 与 BLB_OUT 差分输入到灵敏放大器进行电压放大。

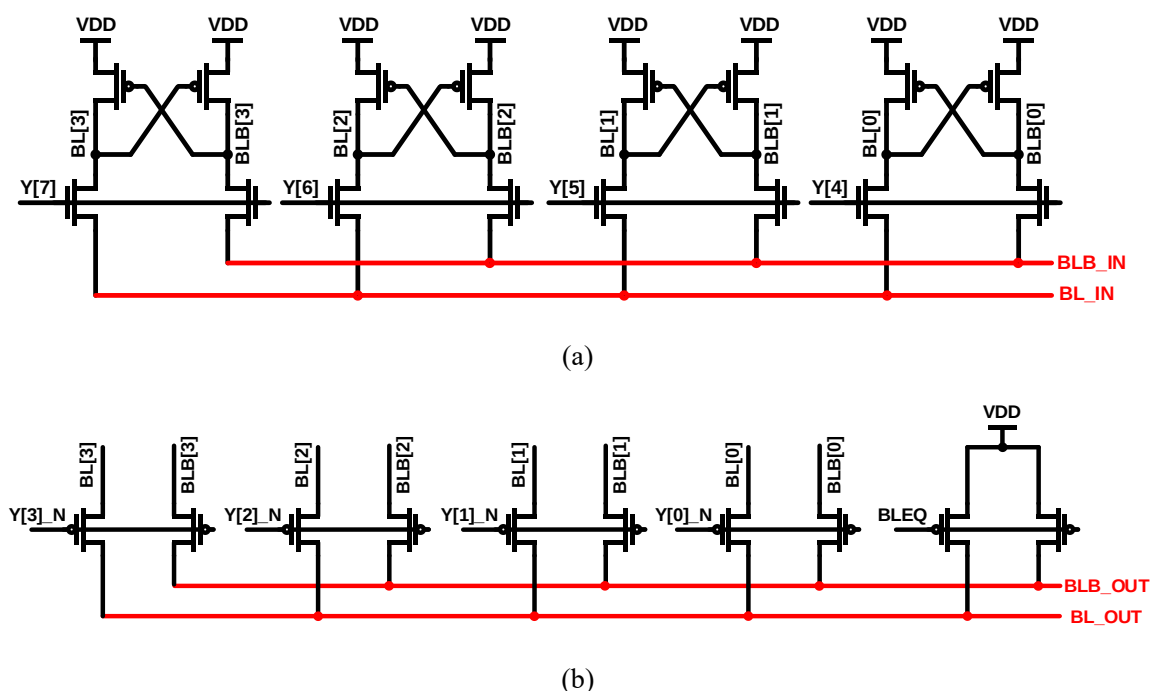


图 3-11 (a)输入多路选择器电路 (b)输出多路选择器电路

灵敏放大器通过检测 BL_OUT 与 BLB_OUT 两条位线间的电压差值来得到所读取的数据，这样即使不让位线全摆幅放电，也可以在输出端得到正确的读出值。如图 3-12 所示为电压锁存型灵敏放大器结构图，电路主要由左侧的差分放大器以及右侧的锁存器两部分组成，BLEQ 为预充电信号，SAE 为读使能信号。整个电路的工作过程如下：阵列在预充电阶段时，BLEQ=0，SAE=0，差分放大器与锁存器均不工作，M1 与 M2、M3 与 M6 管导通，对位线 BL、BLB 进行预充电。阵列进行读权重操作时，BLEQ=1，阵列某一行的字线开启，根据存储的权重值对 BL 或 BLB 的其中一根线进行放电，当位线上的电压下降到一个阈值后，SAE 变为高电平，M1 与 M2 管被关断，将放大器内部的节点与阵列的位线进行隔断，同时 M9 管导通，放大器内部的节点通过一对交叉耦合的反相器进行电平放大，然后将放大后的信号 R_IN 输入给右侧的锁存器电路，锁存器电路采样 R_IN 并输出一个稳定的逻辑电平 R_OUT，完成一次读权重值的操作。读操作完成后，SAE 变为低电平，锁存器此时采样关闭，并经由反馈回路维持 R_OUT 信号的稳定，随之 BLEQ 也变为低电平，对位线继续进行预充电，为下一次的读操作做好准备。整个电路具有高灵敏度、输出稳定、精确控制以及低功耗的特点，可实现对阵列内存储权重值的快速读取。

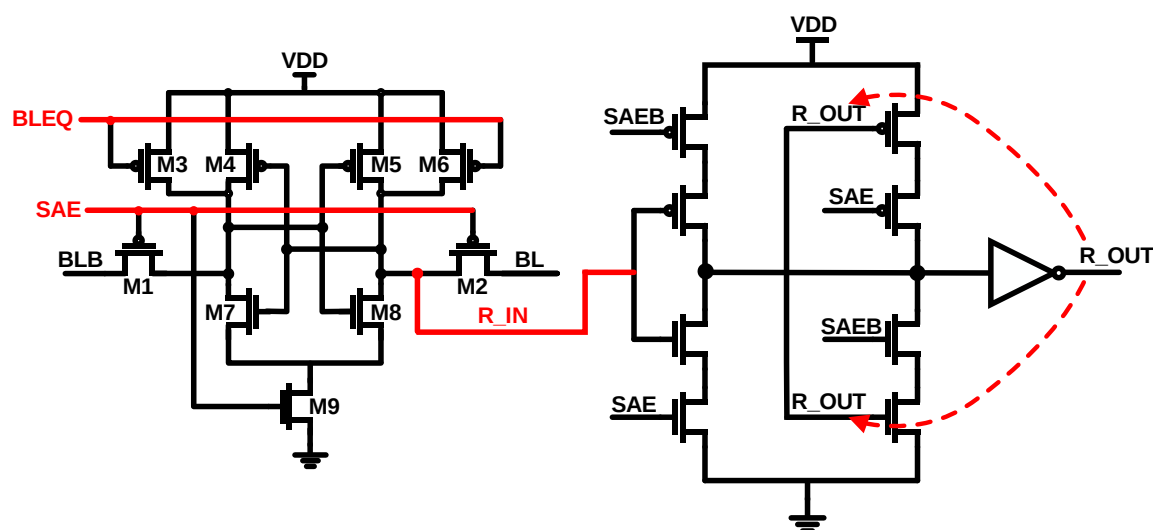


图 3-12 电压锁存型灵敏放大器

3.2.5 时钟控制电路设计

如图 3-13 所示，时钟控制电路的输入信号为 10 比特的地址信号 A、片选信号 CEB、读写控制信号 WEB 以及时钟信号 CLK。电路负责生成由时钟控制的写使能信号 CKD、读使能信号 SAE、预充电信号 BLEQ，并采样输入的地址信号，输出三组地址预解码信号（DEC2~DEC0）以及多路选择器控制信号 Y。

电路的时序是通过冗余阵列的放电来灵活控制的，冗余阵列是一个规模为 256 行 $\times$ 1 列的标准 SRAM 阵列，其内部的存储数据固定为“0”，TRKWL 为低电平时，将冗余 SRAM 阵列的位线 TRKB L 预充电到高电平，并关闭 SRAM 单元，当 TRKWL 为高电平时，停止预充电并开启若干个 SRAM 单元对位线上的电平做放电，直至 TRKB L 放电到低电平。时钟控制电路的工作过程如下：初始状态下，TRKW L 为低电平，TRKB L 为高电平。当 CEB=0，WEB=0，系统进入写周期，电路在 CLK 上升沿处采样地址预解码信号与读写控制信号，输出有效地址信号、多路选择器控制信号 Y 与写使能信号 CKD。由 3.2.4 节可知，CKD 信号会驱动位线驱动电路，Y 信号开启多路选择器，地址信号译码出字线 WL，从而驱使存内计算阵列进行写权重操作，同时 TRKW L 翻转为高电平，冗余阵列开始对 TRKB L 进行放电，当 TRKB L 放电到逻辑低电平时，RSTN 模块的输出变为低电平，导致 D 触发器的置位端有效，将采样的有效信号全部置位低电平，此时存内计算阵列的写操作停止，一段时间后预充电信号 BLEQ 变为低电平，对存内计算阵列位线进行预充电，完成一次写操作。TRKB L 变为低电平的同时，TRKW L 也会翻转为低电平，又慢慢将 TRKB L 预充电回高电平，为下

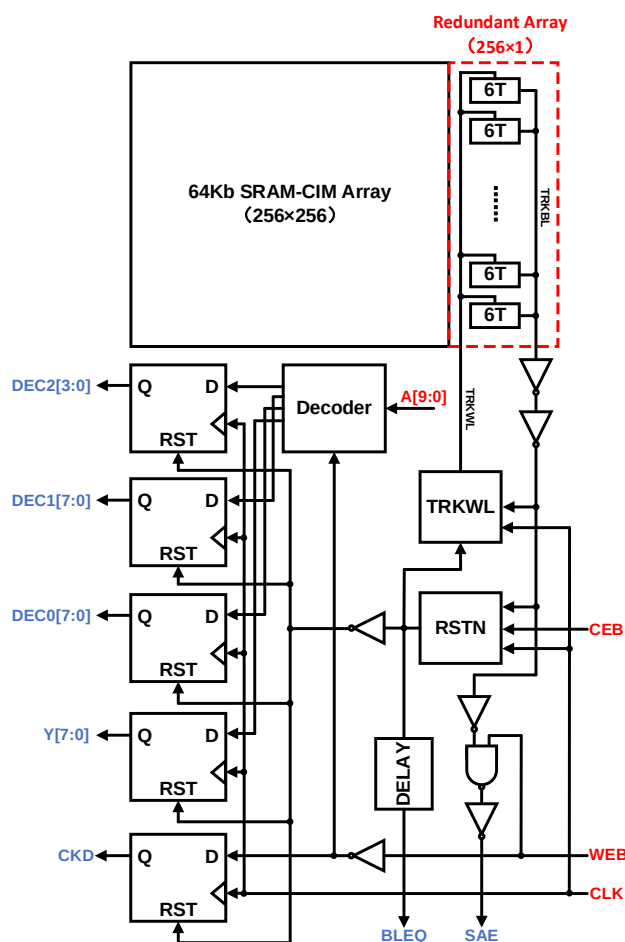


图 3-13 时钟控制电路结构示意图

图 3-14 展示了 RSTN 模块内部的电路图，当 TRKBL 为高电平时，电路采样使能，

时钟信号 CLK 为低电平时，预采样片选信号 CEB，时钟信号 CLK 为高电平时，通过反馈回路稳定输出 EN。当 CEB 为低电平时，输出高电平信号 EN，待 TRKBL 放电到低电平时，电路采样停止，TRKBL 开启 PMOS 管对内部节点电平进行上拉，输出 EN 翻转到低电平，直到 TRKBL 再次预充电回高电平，电路采样功能恢复。当 CEB 为高电平时，输出 EN 始终为低电平。

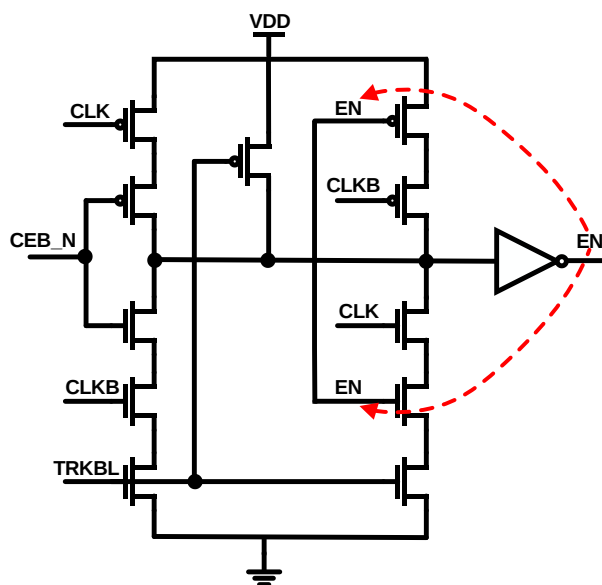


图 3-14 RSTN 模块内部电路示意图

TRKWL 模块内部电路如图 3-15 所示，初始状态下，TRKWL 为低电平，TRKBL 为高电平。由 RSTN 模块的电路原理可知，当片选使能 CEB 为低电平时，EN 为高电平，时钟上升沿到来时，三输入与非门的输出变为低电平，TRKWL 翻转到高电平，冗余阵列开启对 TRKBL 放电，当 TRKBL 放电到低电平时，TRKWL 又翻转到低电平，从而导致冗余阵列停止放电并将 TRKBL 预充电回高电平，此时 TRKWL 模块内部的电平达到稳态值，直到下一个时钟上升沿到来电路跳脱出该稳态继续进行相同的电平翻转。当片选使能 CEB 为高电平时，EN 始终为低电平，这会导致 TRKWL 始终保持初始值“0”无法进行翻转，电路始终处于稳态。TRKWL 模块可以产生受时钟控制的周期性变化的信号 TRKWL，从而用来控制冗余阵列的充放电，以达到时序控制的目的。

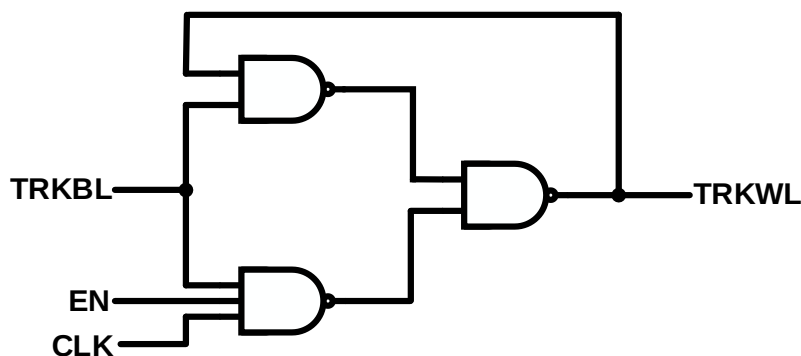
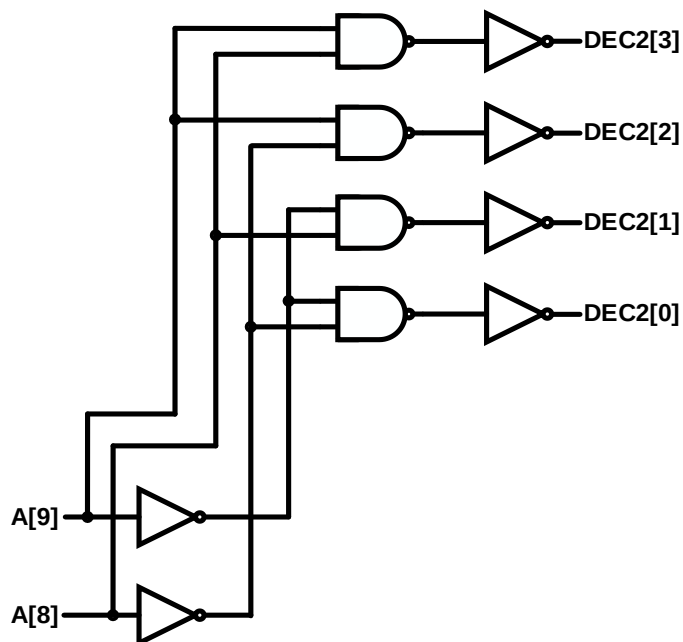
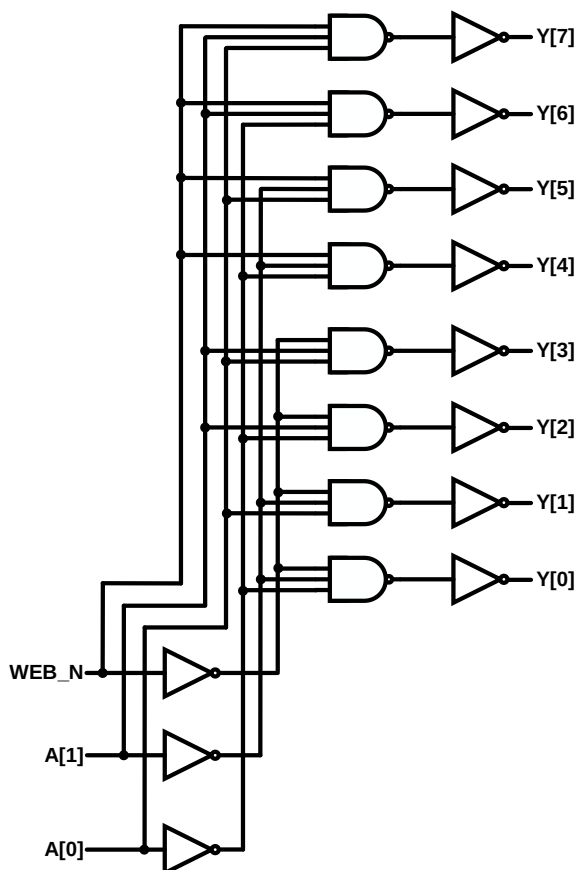


图 3-15 TRKWL 模块内部电路示意图

时钟控制电路内有一个预解码模块，其负责对 10 比特的地址信号 A 做预解码，然后将译码出来的信号通过时钟采样后分别发送给字线译码器电路与读入读出驱动电路。预解码模块内主要由两种译码器构成，如图 3-16(a)与 3-16(b)所示，它们分别是：2-4 译码器与 3-8 译码器。地址信号 $A[9:0]$ 大体上可以分为两部分：（1） $A[9:2]$ 为行地址信息，负责解码出 256 比特的字线信号 WL ，其中 $A[9:8]$ 送到 2-4 译码器中解码出 $DEC2[3:0]$ ， $A[7:5]$ 与 $A[4:2]$ 分别送到两个 3-8 译码器中解码出 $DEC1[7:0]$ 与 $DEC0[7:0]$ ，三组地址预解码信号接着会发送到字线译码器电路做进一步的译码操作；（2） $A[1:0]$ 为 MUX 控制信息，其与读写使能信号 WEB 一起送到 3-8 译码器中，解码出多路选择器控制信号 $Y[7:0]$ ，当 $WEB=0$ 时， $Y[7:4]$ 有效， $Y[3:0]$ 始终为“0”，反之同理。



(a)



(b)

图 3-16 两种译码器电路：(a)2-4 译码器 (b)3-8 译码器（编译 Y[7:0]为例）

3.2.6 字线译码器电路设计

字线译码器电路负责编译时钟控制电路输出的三组地址预编码信号 DEC2[3:0]、DEC1[7:0]、DEC0[7:0]从而产生 256 比特的字线信号 WL。电路结构如图 3-17 所示，三组地址信号均为独热信号，通过用三输入与非门组合独热信号中的“1”，可以得到 $4 \times 8 \times 8$ 中输出结果，即为 256 根字线信号，电路每一次编译只能产生一根高电平信号 WL，从而开启存内计算阵列的某一行进行读写操作。由 3.2.5 节可知，三组地址信号在读写过程开启一段时间后会清零，因此字线译码器的输出 WL 也会被全部置位零，导致存内计算阵列关闭内部单元，停止读写操作。当片选使能信号 CEB 无效时，输入三组地址信号始终为低电平，导致编译出来的字线 WL 也始终为零，不开启存内计算阵列进行操作，从而确保阵列内存储的权重值的稳定。

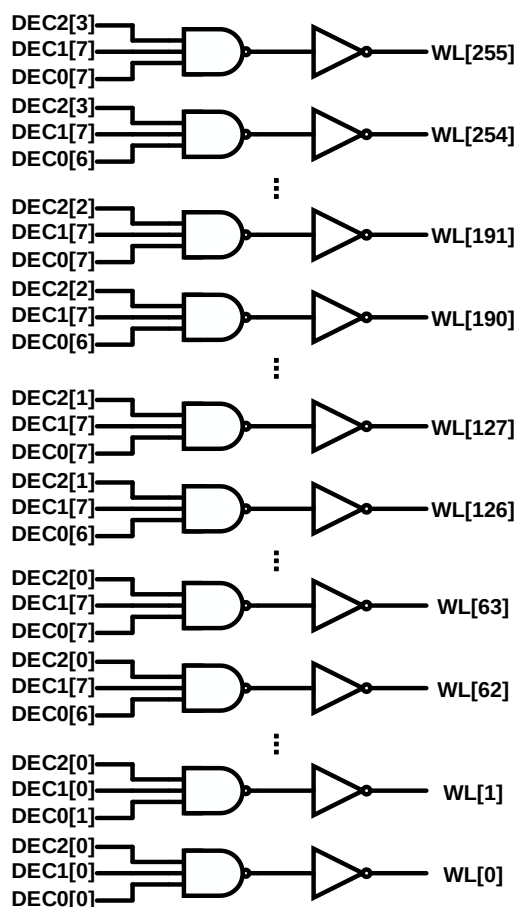


图 3-17 字线译码器电路

3.3 电路性能仿真与分析

根据 3.2 节的电路原理分析，本文基于台积电 28nm（tsmc28hpcplus）的工艺制程搭建了整体架构的电路并对整个系统的功能与性能进行了完备的后仿真。本节首先列举了关键存内计算单元的稳定性与性能仿真，然后列举了系统工作过程的瞬态仿真及系统的性能汇总，最后总结了整个存内计算架构的接口功能及时序，并列举了整体芯片的版图。

3.3.1 存内计算单元性能仿真

存内计算单元的版图如图 3-18 所示，红框内为 6 管 SRAM 单元，蓝框内为或非门单元。版图设计时将 SRAM 单元紧挨或非门单元，减少数据传输的路径，提高计算速度，同时将整个存内计算单元设计的非常紧凑，减少存内计算阵列的面积开销。存内计算单元的版图长 1.05um，宽 1.01um。

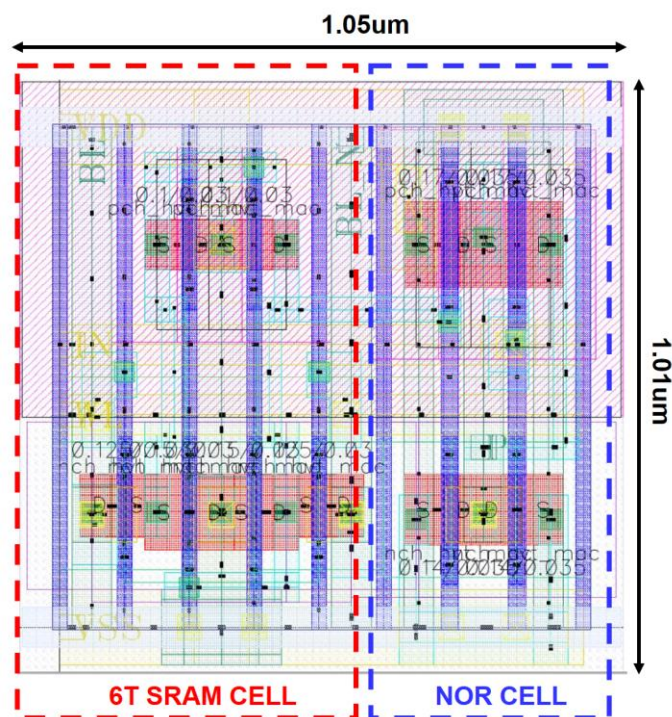
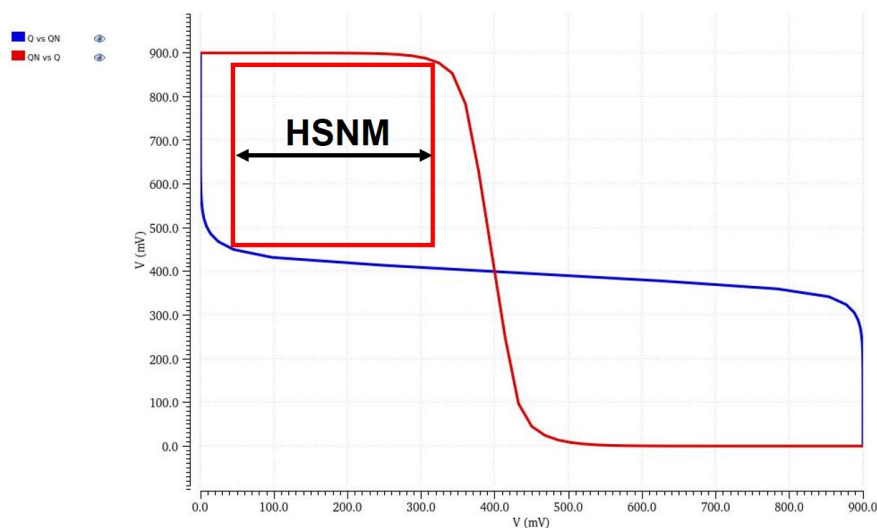


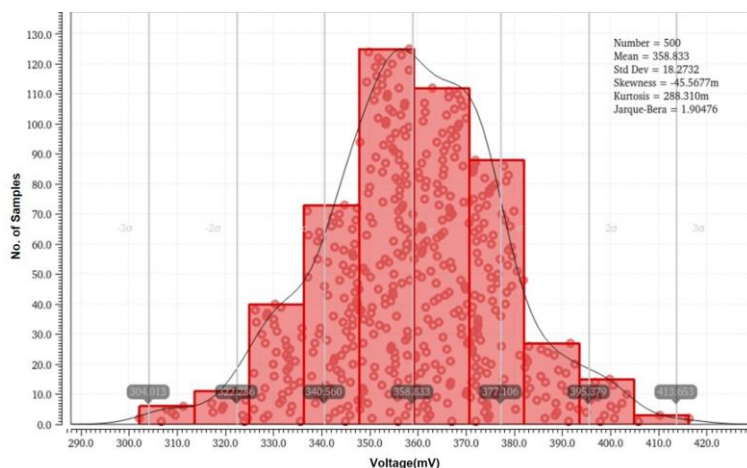
图 3-18 存内计算单元版图

存内计算单元在进行存储操作时，内部数据的稳定性是决定其是否能被正常使用的最重要因素，我们一般用静态噪声容限（Static Noise Margin, SNM）来衡量其内部节点数据的稳定性，SNM 的值越大，表示存内计算单元的抗噪声能力强，单元的稳定性更好。SNM 根据操作类型的不同，又分为 HSNM、RSNM 与 WSNM，接下来分别对存内计算单元的三种噪声容限进行了仿真与分析。

保持噪声容限（Hold Static Noise Margin, HSNM）指的是存内计算单元进行保持（Standby）操作时，在保证存储数据不翻转的情况下，内部的存储节点上所能施加的最大噪声电压^[41]。仿真时我们设置字线 WL 的电平始终为“0”，两根位线 BL 与 BLB 的电平始终为“1”，将存储节点 Q 的电压从 0V 扫描到电源电压 0.9V，得出反向存储节点 QN 随 Q 的变化曲线，接着更改曲线的横纵坐标后再将其与原曲线合并，得到存内计算单元进行 HSNM 分析的蝴蝶曲线图，如图 3-19（a）所示。HSNM 的值指的是两条曲线中间包含的最大正方形的边长，tt 工艺角/0.9V 电压下 HSNM 的值为 358mV。为了确保结果的准确性，在 mc 工艺角/0.9V 电压下对存内计算单元的 HSNM 进行 500 次蒙特卡洛仿真，仿真结果如图 3-19（b）所示，可以得到 0.9V 电源电压下存内计算单元的 HSNM 为 358.833mV。



(a)

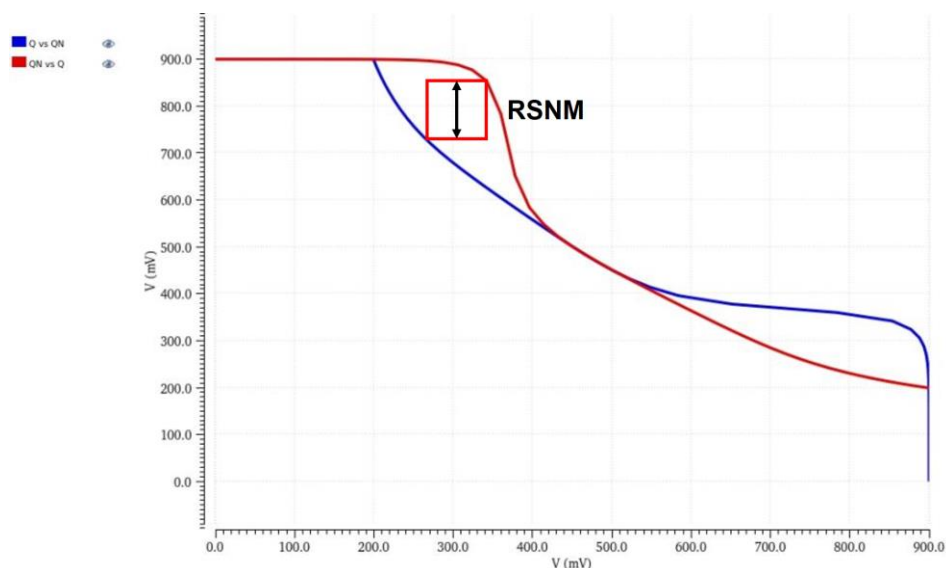


(b)

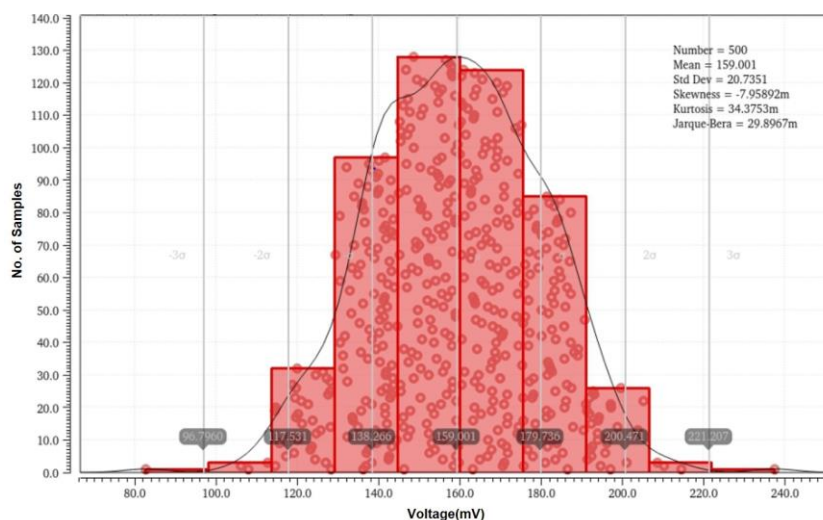
图 3-19 HSNM 仿真分析: (a)蝴蝶曲线图 (b)0.9V 电压下蒙特卡洛仿真结果

读噪声容限 (Read Static Noise Margin, RSNM) 指的是存内计算单元进行读 (Read) 操作时, 在保证内部存储的数据不变的情况下, 存储节点上所能施加的最大噪声电压^[41]。与 HSNM 仿真方法类似, 仿真时我们设置字线 WL 的电平始终为 “1”, 位线 BL 与 BLB 的电平始终为 “1”, 同样对 Q 点进行扫描, 得到 RSNM 的蝴蝶曲线如图 3-20 (a) 所示, RSNM 的值仍为两条曲线中间包含的最大正方形边长, tt 工艺角/0.9V 电压下 RSNM 的值为 158.3mV。同样为了结果的准确性, 在 mc 工艺角/0.9V 电压下对 RSNM 进行 500 次蒙特卡洛仿真, 如图 3-20 (b) 所示, 得到 0.9V 电压下存内计算单元的 RSNM 为 159mV。可以看出, 相较于保持状态, 单元进

行读操作时噪声容限更小，这是因为存内计算单元进行读操作时字线 WL 打开，存储节点与位线连通，其内部的值更容易发生翻转。



(a)



(b)

图 3-20 RSNM 仿真分析: (a)蝴蝶曲线图 (b)0.9V 电压下蒙特卡洛仿真结果

此外，为了观察不同电源电压对读噪声容限的影响，将电源电压按步长为 5 次从 0.8V 扫描到 1.0V 观察蝴蝶曲线的变化状态，如图 3-21 所示，可以看出电源电压越小，曲线越靠近纵坐标轴，即蝴蝶曲线包含的面积越来越小，所以 RSNM 的值越来越小，存内计算单元越不稳定。

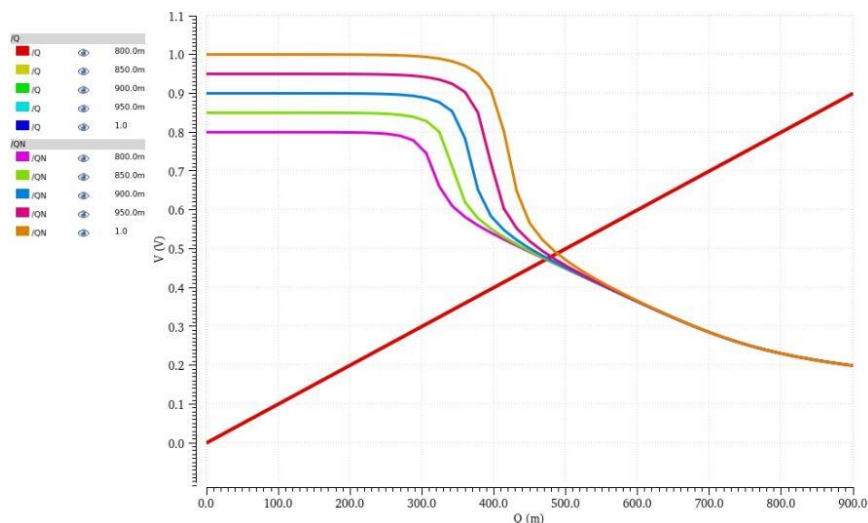
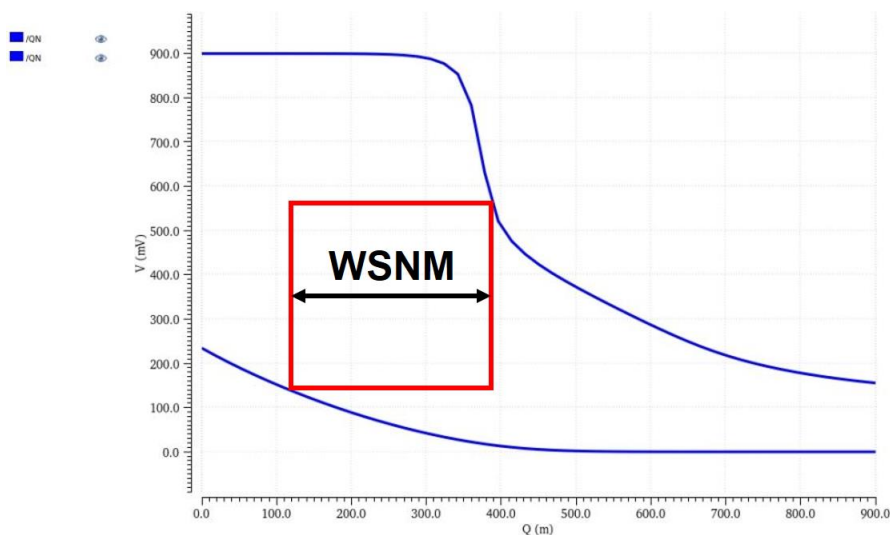
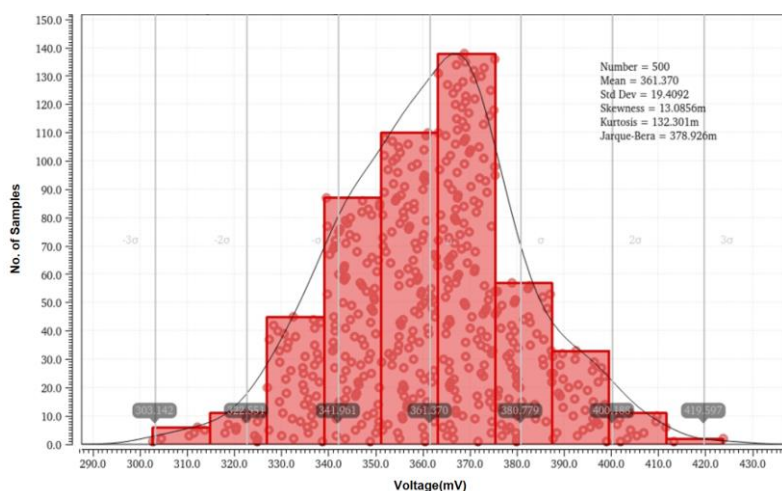


图 3-21 RSNM 随电压变化情况

写噪声容限 (Write Static Noise Margin, WSNM) 指的是存内计算单元进行写 (Write) 操作时, 在确保写入的值不被刷新的情况下, 内部存储节点上所能施加的最大的噪声电压。仿真时我们设置字线 **WL** 的电平始终为 “1”, 分别设置单元内存储的值为 “1” 与 “0”, 然后每次写入相反的值, 并将存储节点 **Q** 的电压从 0V 扫描到电源电压 0.9V, 得出两次写入过程时反向存储节点 **QN** 随 **Q** 的变化曲线, 如图 3-22 (a) 所示, 这两条曲线包含的最大正方形边长即为 **WSNM**。在 **tt** 工艺角/0.9V 电压下, **WSNM** 的值为 360.3mV。同样对 **WSNM** 进行 500 次蒙特卡洛仿真, 如图 3-22(b) 所示, 得到 0.9V 电压下 **WSNM** 的值为 361.37mV。



(a)



(b)

图 3-22 WSNM 仿真分析: (a)QN 随 Q 变化曲线图 (b)0.9V 电压下蒙特卡洛仿真结果

此外,对存内计算单元进行计算功能仿真,并评估不同工艺角/电源电压下存内计算单元的计算性能差异。我们对存内计算单元的输入四组可能的输入情况 ($Q=0$, $IN=0$; $Q=0$, $IN=1$; $Q=1$, $IN=0$; $Q=1$, $IN=1$),得出四种情况下单元进行计算时所花费的最大延时与所消耗的最大功耗,并在不同工艺角/电源电压下对计算性能进行对比。如图 3-23 (a) 与 (b) 所示,可以得到在 $ffg/0.99V$ 情况下,计算所需延时最小,为 $14.962ps$,在 $ssg/0.81V$ 情况下,计算所消耗的功耗最少,为 $90.92nW$ 。

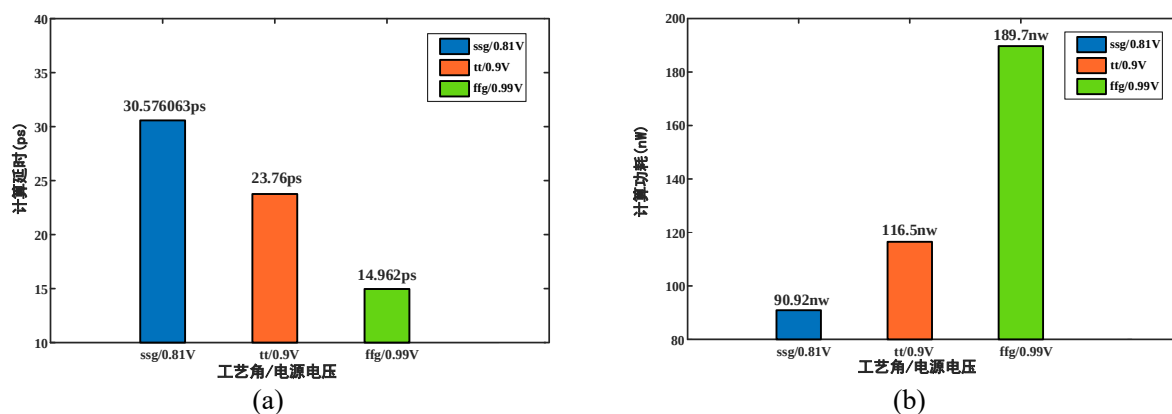


图 3-23 存内计算单元计算性能仿真分析: (a)计算延时对比 (b)计算功耗对比

3.3.2 系统性能仿真与分析

系统中一个存内计算处理器 (Computing in memory Macro, CIM Macro) 的版图如图 3-24 所示。红框内为 $1Kb$ (256 行 $\times$ 4 列) 大小的存内计算阵列,蓝框内为阵列的读入读出驱动电路,绿框内为 8 级混合并行加法器树电路。版图的长为 $259.705\mu m$,宽为 $32.235\mu m$,其中加法器树的面积占比为 66.884% 。可以看出即使采用 $11T$ 与 $28T$ 交替运行的混合加法器树结构,加法器树的面积开销占比仍然占了整个系统相当大的

一部分。

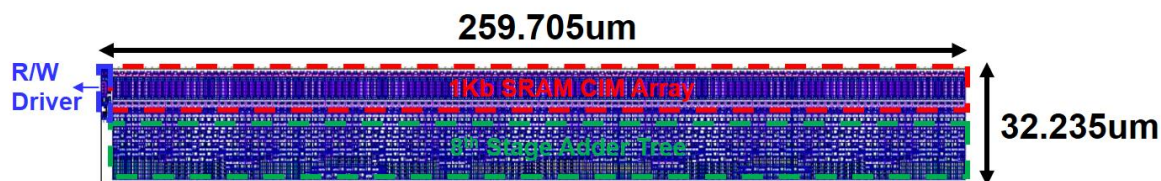


图 3-24 存内计算处理器（CIM Macro）版图

如图 3-25 所示，将 32 个 CIM Macro、字线译码器与时钟控制电路的版图在顶层连接起来。红框内为 1 个 CIM Macro，蓝框内为字线译码器，绿框内为时钟控制电路，一个时钟控制电路驱动控制 32 个 CIM Macro，信号线每经过 8 个 CIM Macro 会添加 1 级 Buffer 来增强驱动能力。字线译码器放在整个版图的正中间，向两边发散 256 根字线，从而确保版图的对称性。顶层版图的长为 1081.565um，宽为 289.25um。

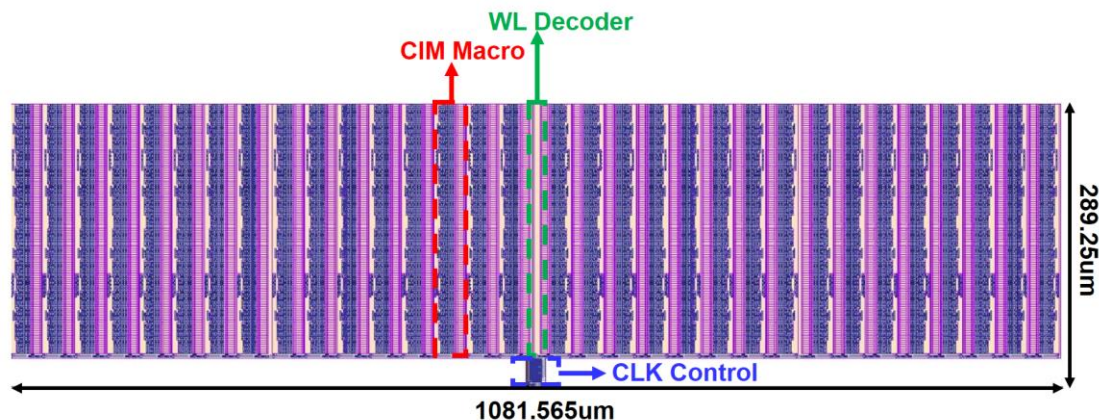
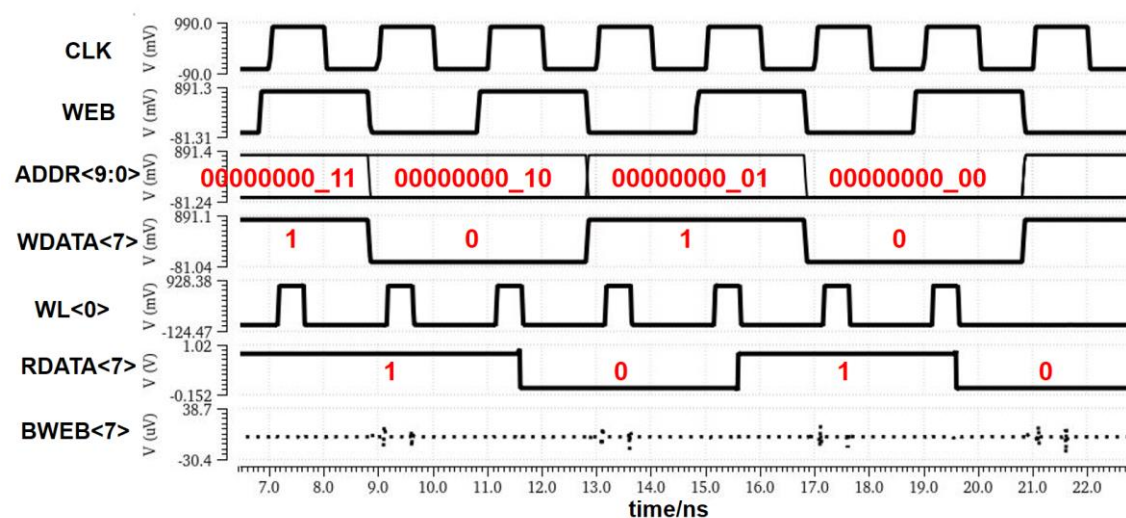
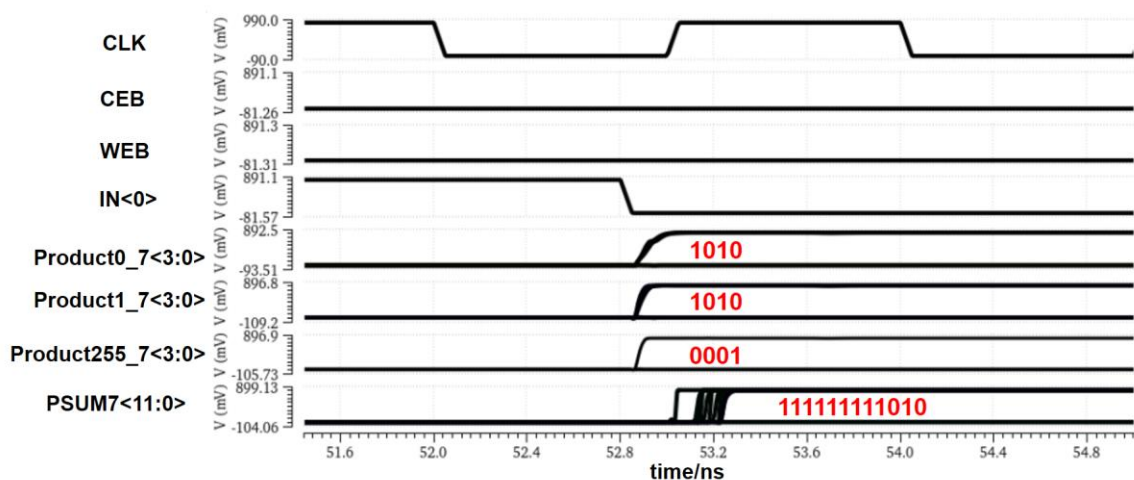


图 3-25 串行单比特存内计算架构顶层版图

对系统的写、读与计算操作过程分别进行瞬态仿真，验证功能的正确性。通过随机数发生器对存内计算阵列输入权重值并将存内计算阵列的数据全部存满，存满后再开始进行计算。如图 3-26(a)所示为存内计算设计的读写瞬态仿真波形图，对存内计算阵列的第 1 行（WL=0）写入“1010”的权重值，每写一次权重在下一个周期将这个值读取出来验证，从而在连续两个操作周期内分别验证写操作与读操作。将整个阵列的权重值全部刷新完成后，开始验证计算功能。如图 3-26(b)所示为存内计算设计的计算瞬态仿真波形图，计算时 CEB 置为“0”，确保存内计算阵列内部数据的稳定，然后输入有效数据 IN，观察特定行产生的中间积从而确保乘法过程无误，最后观察并行加法器树的输出检验累加结果的正确性。



(a)



(b)

图 3-26 系统的瞬态仿真: (a)读写操作过程 (b)计算过程

此外在不同的工艺角/电压条件下, 分别评估系统进行读操作与计算操作所需的延时。选取版图上距离输入端口最远一列的存内计算阵列, 即为读取或计算的关键路径, 测量从时钟上升沿到该列输出读取数据或计算结果所需的延时, 即为系统进行读取或计算所需的最大延时。如图 3-27 所示, 可以总结出在不同工艺角/电源电压下读取延时始终大于计算延时, 且在 $ssg/0.81V$ 的条件下系统的读取延时为 $2.013ns$, 计算延时为 $1.912ns$, 满足系统 $500MHz$ 的工作频率要求。

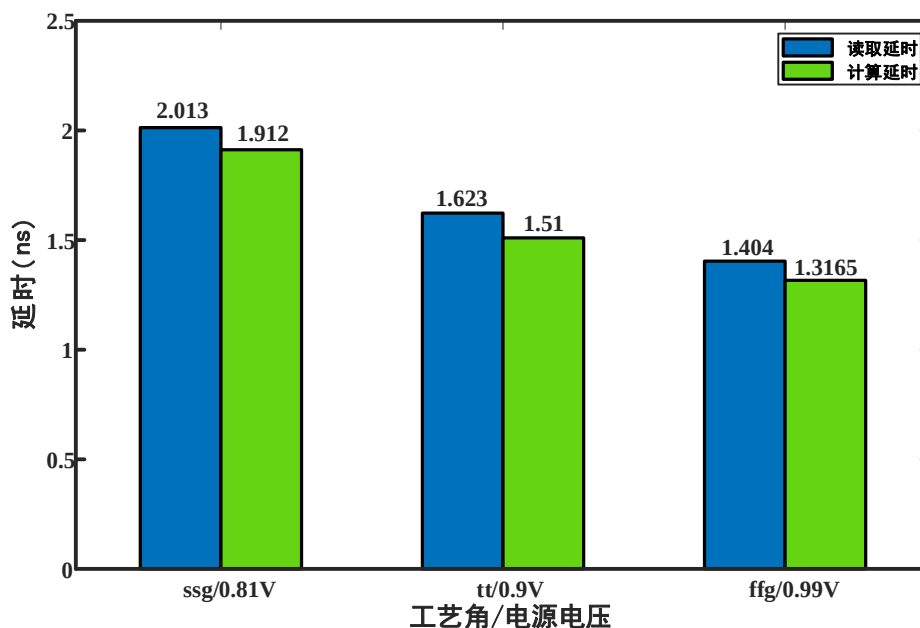


图 3-27 系统的读取延时与计算延时对比

同时通过仿真得出在不同工艺角/电源电压条件下，系统进行写操作、读操作与计算操作分别所消耗的功耗。如图 3-28 所示，可以看出计算功耗始终高于读功耗与写功耗，ffg/0.99V 条件下系统消耗的计算功耗最大，为 64.376mW，ssg/0.81V 条件下最低，为 52.672mW。不同条件下系统的读功耗与写功耗总是相近的，大约相差 0.4mW 左右，这是因为读写操作过程类似，系统内进行动态翻转的电路几乎相同。

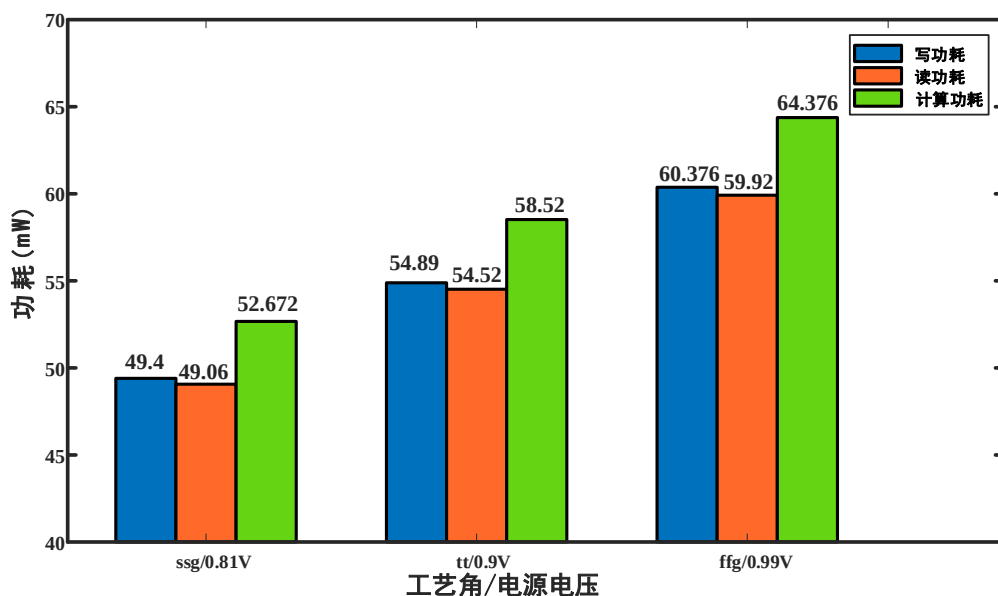


图 3-28 系统的三种操作功耗对比

如图 3-29 所示，为系统内不同模块的计算功耗占比。可以看出系统进行计算操作时，加法器树与累加器的功耗开销占比最大，达到 51%。这是因为系统进行卷积运算

时加法器树与累加器需要在连续的操作周期内不断进行翻转并且加法器树的面积开销比较大，同一周期内并行运算的组数较多，从而产生了较大的动态功耗。

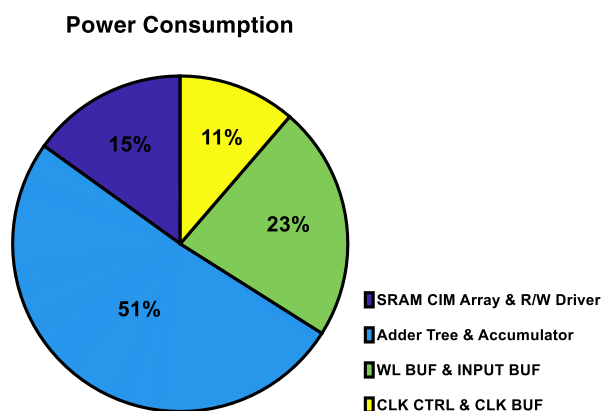


图 3-29 系统内各模块的计算功耗占比

综合考量系统的计算速度与计算能耗，我们得出不同工艺角/电源电压下的能耗效率变化趋势如图 3-30 所示。虽然高速工艺角/高电源电压提升了系统的计算速度，但是系统的功耗也显著提高，并且功耗的增量大于速度的提升量，因此能耗效率是逐渐递减的趋势。

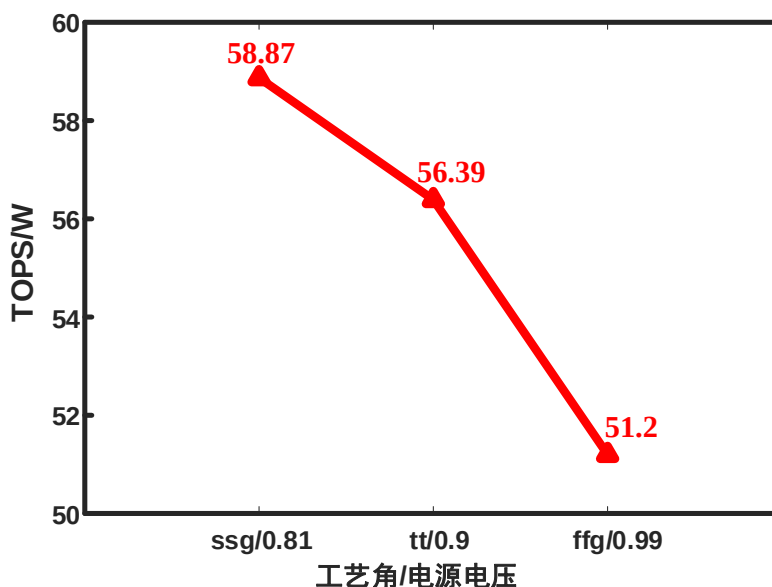


图 3-30 系统的能耗效率随工作条件的变化趋势

综合以上仿真分析，总结串行单比特存内计算架构的各项指标如表 3-3 所示。仿真结果说明，系统的工作频率可以满足 500MHZ，因此在进行 4-bit 输入与 4-bit 权重 (4b/4b) 的卷积运算时系统的算力可以达到 3300GOPS，进行 8b/8b 运算时算力达到

917GOPS。系统在 t_t 工艺角/0.9V 电压下，计算消耗的功耗为 58.52mW，同时可以获得 15.67TOPS/W 到 56.39TOPS/W 的能耗效率。

表 3-3 串行单比特存内计算架构性能汇总

架构性能总结	
工艺制程	28nm CMOS
存内计算阵列大小	64Kb
存内计算单元类型	6T+1NOR
供电电压(V)	0.9
输入精度(bit)	1~8
输入符号	Signed/Unsigned
权重精度(bit)	4/8/12/16
权重符号	Signed/Unsigned
输出精度(bit)	16(4b/4b)/24(8b/8b)
工作频率(MHz)	500
计算功耗(mW)	58.52(t_t /0.9V)
吞吐率(GOPS)	3300(4b/4b)/917(8b/8b)
能耗效率(TOPS/W)	56.39(4b/4b)/15.67(8b/8b)

3.3.3 串行单比特输入存内计算架构接口时序

整个芯片除了核心的存内计算架构设计外，还包含了由 memory compiler 生成的用来检验计算结果的标准 SRAM 阵列、用来减少芯片 IO 数量的串行外围设备接口（Serial Peripheral interface, SPI）以及控制外围电路时序的数字电路等。外围的数字电路与存内计算设计在数字后端进行连接时，为了整体芯片时序约束的正确性，需要将存内计算设计看作为 IP，提供这个 IP 设计各个引脚的功能描述及时序约束文件（.lib），因此需要对存内计算架构设计的输入输出端口进行功能汇总，并且通过仿真测量出每个输入端口的建立时间（set up time）以及保持时间（hold time）及输出数据的传输延时。

如表 3-4 所示，描述了存内计算架构所有接口的数据位宽、输入输出方向及功能介绍。其中 CEB 为片选使能，WEB 为读写使能，IN_VLD 为计算使能，这三组信号为主要的功能控制信号，通过组合不同的信号值可以让系统在保持、读、写、计算四种操作模式之间来回切换，具体如下：

- (1) CEB=1、IN_VLD=0 时系统不进行任何操作，进入保持 (standby) 模式。
- (2) CEB=0、IN_VLD=0、WEB=0 时系统进入写入 (write) 模式。
- (3) CEB=0、IN_VLD=0、WEB=1 时系统进入读出 (read) 模式。
- (4) CEB=1，IN_VLD=1 (IN_VLD=1 时输入计算数据有效) 时系统进入计算 (compute) 模式。

表 3-4 串行单比特存内计算架构接口功能汇总

接口名称	数据位宽	方向	功能说明
CLK	1	INPUT	工作时钟，最高频率 2GHz
RSTN	1	INPUT	复位信号，低电平有效
SIGN_W	64	INPUT	权重符号，1 表示有符号数
SIGN_IN	1	INPUT	输入符号，1 表示有符号数
SHIFT	1	INPUT	累加时是否移位，1 表示移位
CLEAR	1	INPUT	累加值清除，1 表示清除
WEB	1	INPUT	写使能信号，0 表示写入，1 表示读出
CEB	1	INPUT	片选使能，0 表示选中存内计算阵列进行操作，1 表示未选中存内计算阵列，其无法进行任何操作
A	10	INPUT	地址信号，前 8 位负责编码 256 位的字线信号，后 2 位负责编码多路选择器 MUX 的控制信号
D	64	INPUT	64 个存内计算处理器的输入权重值
BWEB	64	INPUT	阵列列选信号，低电平有效，可以选择开启任意列进行读写操作
IN_VLD	1	INPUT	计算有效信号，1 表示输入数据有效，系统开始计算，0 表示输入数据无效
IN	256	INPUT	256 组并行单比特输入数据
R_OUT	64	OUTPUT	读取存内计算阵列内存储的值，64 列并行行读取
PSUM	12*64	OUTPUT	加法树的部分和输出，64 列并行输出
Q	20*64	OUTPUT	卷积计算结果，64 列并行输出

输出信号为读出权重值 R_OUT 、加法器树输出部分和 $PSUM$ 及累加器输出卷积计算结果 Q ，其中 $PSUM$ 的值作为测试接口，方便判断流水线的第一级操作功能是否异常。三组输出值会送入到外围 SRAM 阵列中待验证。

如表 3-5 所示，列举了串行单比特存内计算架构在不同工艺角/电源电压条件下的接口时序，芯片在顶层进行时序约束时重点关注 $ssg/0.81V$ 下输入端口的最长建立时间（set up time）以及 $ffg/0.99V$ 下输入端口的最长保持时间（hold time）。此外，还列举了读出权重输出 R_OUT 与部分和输出 $PSUM$ 到时钟上升沿的延时，读权重值所需要的最长延时为 $2.013ns$ ，计算部分和的最长延时为 $1.912ns$ 。

表 3-5 串行单比特存内计算架构接口时序汇总

参数	定义	$ssg/0.81V$	$tt/0.9V$	$ffg/0.99V$
t_{cyc}	时钟周期	$2ns$		
t_{ckh}	时钟高电平时间	$1ns$		
t_{ckl}	时钟低电平时间	$1ns$		
t_{cs}	CEB 建立时间	$0.1284ns$	$0.1168ns$	$0.108ns$
t_{ws}	WEB 建立时间	$0.125ns$	$0.1123ns$	$0.1042ns$
t_{as}	ADDRESS 建立时间	$0.1262ns$	$0.114ns$	$0.1065ns$
t_{ds}	D 建立时间	$0.124ns$	$0.11ns$	$0.1028ns$
t_{is}	IN 建立时间	$0.118ns$	$0.106ns$	$0.009ns$
t_{ch}	CEB 保持时间	$0.0482ns$	$0.031ns$	$0.0242ns$
t_{wh}	WEB 保持时间	$0.049ns$	$0.0332ns$	$0.025ns$
t_{ah}	ADDRESS 保持时间	$0.052ns$	$0.0346ns$	$0.0284ns$
t_{dh}	D 保持时间	$0.082ns$	$0.0754ns$	$0.068ns$
t_{ih}	IN 保持时间	$0.032ns$	$0.0258ns$	$0.018ns$
t_{clk-R}	时钟上升沿到有效输出 R_OUT 的时间	$2.013ns$	$1.623ns$	$1.404ns$
t_{clk-PS}	时钟上升沿到有效输出 $PSUM$ 的时间	$1.912ns$	$1.51ns$	$1.3165ns$

得到串行单比特存内计算架构接口的功能及时序后，将存内计算设计的版图与外围数字电路版图在数字后端进行连接，并在顶层对整体版图的时序进行约束，得到整个芯片的版图如图 3-31 所示，版图长为 $1700\mu m$ ，宽为 $1300\mu m$ 。

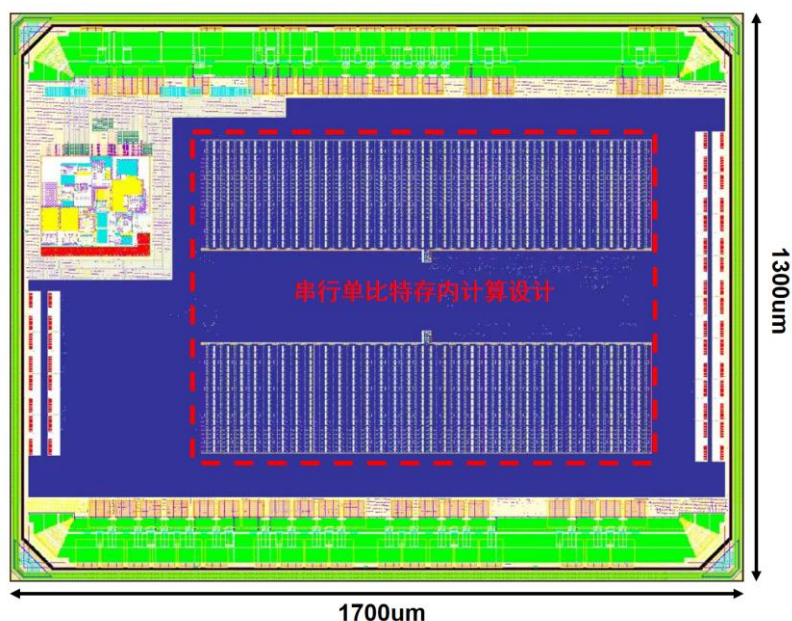


图 3-31 存内计算芯片版图

3.4 本章小结

本章提出了一种基于 6T SRAM 的串行单比特输入存内计算设计，将多比特输入与多比特权重的卷积运算在多周期内以流水线的方式实现。输入数据从高位（MSB）到低位（LSB）每个操作周期输入 1 比特，分多周期将数据依次输入到存内计算架构中。每个操作周期内存内计算架构内部并行运算多组 1 比特输入与多比特权重值的乘积，并将这些中间积通过加法器树累加起来得到部分和，通过在多个操作周期内对多组部分和移位并累加，最终得到卷积运算的结果。该存内计算设计基于台积电 28nm 的工艺流片，存内计算单元的类型为 6T+1NOR，系统的工作频率为 500MHz，其支持输入数据位宽从 1-bit 到 8-bit，支持权重数据位宽为 4-bit/8-bit/12-bit/16-bit。在 $t_t/0.9V$ 的工作条件下，系统在处理 4-bit 输入与 4-bit 权重的卷积计算时获得 3300GOPS 的吞吐率与 56.39TOPS/W 的能耗效率，在处理 8-bit 输入与 8-bit 权重的卷积计算时实现 917GOPS 的吞吐率与 15.67TOPS/W 的能耗效率。

第4章 基于复用 Booth 计算单元和 6T SRAM 的可重构存内计算设计

第 3 章提出的串行单比特输入存内计算架构通过高并行运算的方式,极大程度地提高了系统的吞吐率,从而提高了系统的能耗效率。但是 256 组中间积的并行运算导致系统内加法器树的面积与功耗开销较大,在中间积位数提高的情况下,加法器树的巨大功耗对系统能效比的限制愈加严重。此外架构内每一个 SRAM 单元内都嵌入了一个或非门单元,提高了系统的静态功耗,并且在面对规模较小的卷积神经网络时,其输入数据小于 256 组,这就会导致存内计算阵列的利用率达不到 100%,在系统进行卷积计算的过程中消耗了多余的静态功耗。因此,为了使系统能保持较高的能耗效率,同时降低系统的静态功耗,并且能灵活应对不同规模的卷积神经网络,提高存内计算阵列的利用率,本章提出了一种基于复用 Booth 计算单元和 6T SRAM 的可重构数字存内计算设计。它主要采用了(1)分时复用计算单元的计算模式,降低了一个周期内并行运算的组数,从而降低了加法器树的规模,并且支持卷积计算组数的可重构以灵活适应不同规模的卷积神经网络;(2)串行双轨输入的数据输入模式保证了系统的吞吐率;(3)基于 Booth 编码原理的输入编码模块;(4)可实现 Booth 运算的 Booth 乘法计算单元;(5)可配置的移位累加器实现卷积计算规模的重构。设计基于台积电 28nm, SRAM 阵列大小为 64Kb,系统的计算频率为 500MHZ,计算精度为 100%,计算输入数据组数可配置,可达 64N (N 最大为 4),同时系统具有 482GOPS 到 1820GOPS 的吞吐率与 21.64TOPS/W 到 61.745TOPS/W 的能耗效率。

4.1 复用 Booth 计算单元和 6T SRAM 存内计算设计架构

图 4-1 展示了基于复用 Booth 乘法计算单元 (Booth multiplication computing cell, BMCC) 和传统 6T SRAM 单元的存内计算设计架构,架构主要组成模块有:64 个 Booth 乘法存内计算处理器 (Booth multiplication computing in memory Macro, BM-CIM Macro),输入编码模块 (Input Encoder),地址译码器 (Address Decoder) 和时序控制模块 (Timing Control)。

每个 Booth 乘法存内计算处理器包含 5 个模块,分别是:1Kb 大小的存内计算阵列、阵列读写驱动电路、位扩展单元、加法器树与累加器电路。存内计算阵列由 64 组 Booth 乘法存内计算数据块 (Booth multiplication CIM data bank, BMCDB) 组成,每一组数据块内包含一个规模为 4 行×4 列大小的 SRAM 子阵列和一个 Booth 乘法计算单

元 (Booth multiplication computing cell, BMCC)。计算时, 四行 SRAM 阵列共用一个计算单元, 分时将权重值读取到 BMCC 中进行计算。阵列读写驱动电路与第三章提出的结构相同, 负责使能权重的写入与放大读出的权重值。位扩展单元根据输入编码模块输出的控制信号对 BMCC 输出的 64 组数据进行相应的扩展操作, 得到 64 组 6 比特的中间积。加法器树将这些中间积累加求和, 得到部分和后送入累加器电路进行移位累加, 加法器树的结构仍采用第三章提出的 11T 与 28T 交替运行结构, 减少加法器树的功耗与面积, 累加器具有可配置移位周期的功能, 从而实现卷积运算规模的重构。输入编码模块由 64 个 Booth 乘法编码器 (Booth multiplication Encoder, BM Encoder) 组成, 其依据 Booth 编码原理, 对多比特输入进行编码得到 Booth 运算的控制信号, 驱动 BMCC 进行相应的操作。时序控制模块负责整个系统的时序控制与工作模式切换, 地址译码器负责编译行地址与列地址, 对选中的存储单元进行读权重、写权重或计算操作。

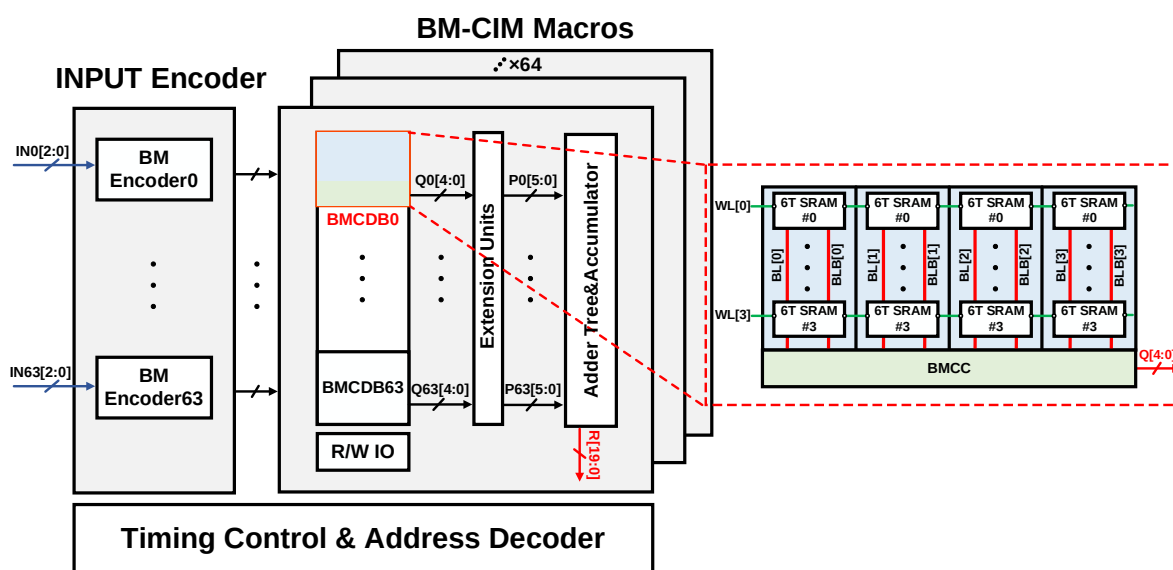


图 4-1 基于复用 Booth 计算单元和 6T SRAM 的存内计算设计架构

4.2 架构分时复用策略与可重构卷积运算时序

如图 4-2 所示, 我们将 1Kb 阵列重新排布, 排列成 4 行 $\times$ 64 列 $\times$ 4 比特的规格, 每一列是一组 Booth 乘法存内计算数据块 (BMCCDB), 每一组 BMCCDB 中四个 SRAM Bank 分时复用同一个 Booth 乘法计算单元 (BMCC)。系统处于存储模式时, 通过同时选中行选信号 WL 与列选信号 CSL 来对选中的 SRAM Bank (规模为 1 行 $\times$ 4 列) 进行

读操作或者写操作。当系统处于计算模式时，列选信号 CSL 全部为低电平，字线 WL 开启选中其中一行将 64 个 4 比特的权重值输入到 BMCC 单元中做 Booth 运算操作。

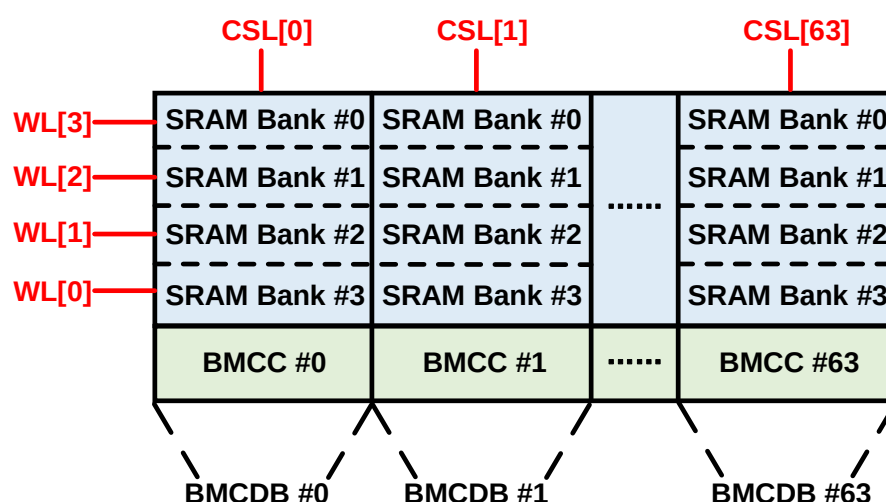


图 4-2 存内计算阵列布局

由于每四个 SRAM Bank 分时复用一個 BMCC 单元，这意味着如果要将阵列内存储的所有权重全部计算完，相比于存储单元与计算单元 1:1 配置的方式，系统需要额外消耗多个运算周期，若仍采用第三章提出的穿行单比特的数据输入模式，会导致系统的吞吐率严重下降，因此本章采取串行双比特输入的卷积计算架构，每个周期处理 2 比特数据，减少运算所需要的周期数。如图 4-3 所示，2 比特输入数据与 4 比特固定位宽的数据相乘得到部分积，多组部分积累加起来得到部分和后，移位累加器将输出端累积的值向左移两位并与部分和进行求和，逐步迭代，通过流水线的方式可以输出卷积计算结果。

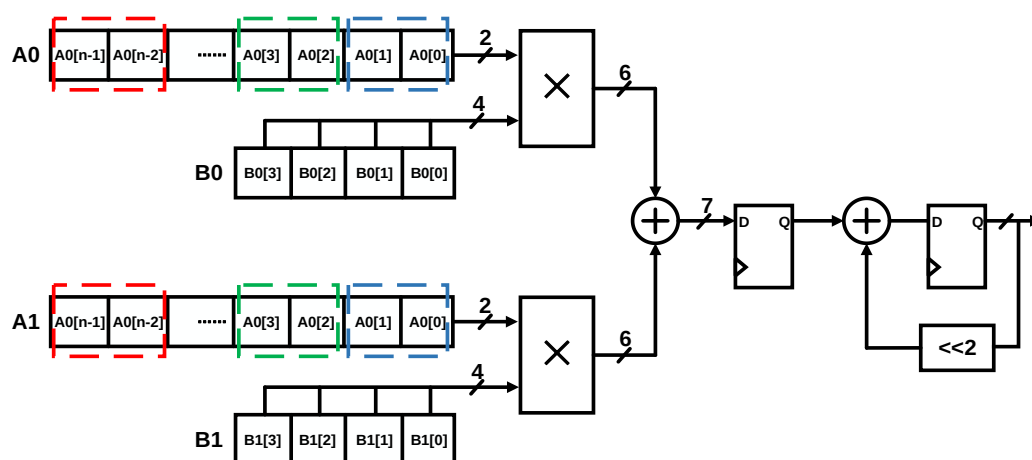


图 4-3 串行双比特输入卷积计算架构

根据存内计算阵列的布局与系统采取的卷积计算架构，可以得到 Booth 乘法存内

计算处理器（BM-CIM Macro）内部结构示意图如图 4-4 所示。BM-CIM Macro 的工作过程如下：系统内权重被划分为四组，存内计算阵列中同一行存储的权重值为同一权重集，系统进行卷积计算操作时，开启阵列的其中一行将一组权重集送到 BMCC 单元中进行 Booth 运算，BMCC 将运算后的结果输出给位扩展单元进行位扩展操作，输出 64 个 6 比特的中间积到加法器树模块。由于分时复用导致系统内并行运算组数减小，因此加法器树模块的规模减小，其仅需要经过 6 级逐级累加，就可以输出 12 比特的部分和 PSUM 到累加器模块。累加器模块捕获 PSUM 并对其做位扩展，同时根据系统配置的运算组数决定是否对累积得到的 SUM 值向左移两位，再将处理完的两个加数进行相加，多周期迭代后，通过时钟采样得到 20 比特的卷积计算结果 R。

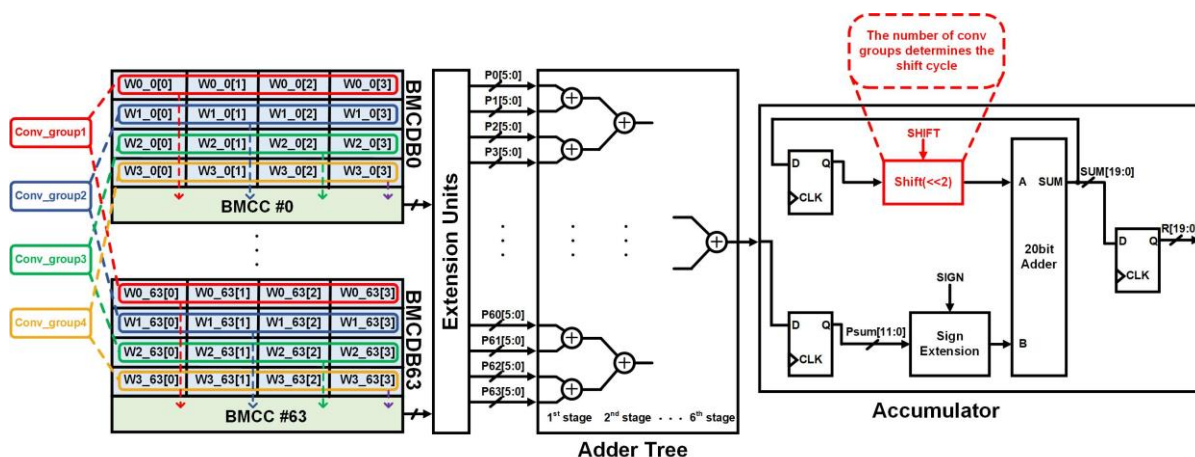


图 4-4 Booth 乘法存内计算处理器内部结构示意图

该架构采用分时复用技术并通过动态配置累加器的移位周期，实现了对不同规模卷积神经网络的适应性。在进行卷积运算时，可以根据卷积神经网络的大小调整运算组数。如图 4-5 所示，为可重构卷积运算的操作时序图。对于大规模网络，系统可能需要同时处理四组不同的权重集。在这种情况下，系统会在连续的四个时钟周期内输入具有相同位权的计算数据，这四组计算数据依次与四组权重集进行计算，生成四组 12 位的部分和输出，这些部分和的位权也是相同的，由于位权相同，累加器在这四个周期内无需移位，它简单地将这些部分和进行累积，当输入数据的位权在第五个时钟周期改变时，累加器将激活移位信号，把之前累积的结果向左移动两位，并与新的部分和相加。因此，在大规模网络的情况下，累加器每四个时钟周期移位一次。相对地，对于小规模卷积神经网络，系统可能只需要处理一组权重集。这时，每个时钟周期后输入数据的位权就会变化，因此累加器在每个时钟周期后都会进行移位操作，以适应快速变化的数据位权。综上所述，架构能够灵活调整卷积计算的组数，以便高效地

处理从小到大不同规模的卷积神经网络，从而优化计算资源并提高处理效率。

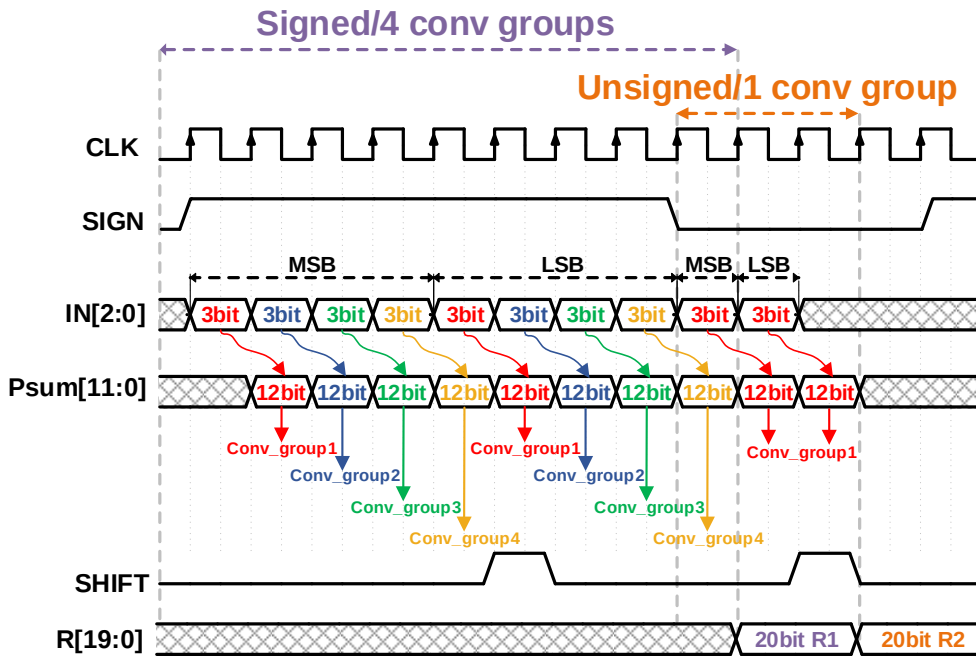


图 4-5 可重构卷积计算操作时序图

4.3 复用 Booth 计算单元和 6T SRAM 存内计算电路设计

由 4.2 节可知，为了缓解分时复用策略带来的系统吞吐率降低的问题，架构采取串行双比特的数据输入模式，每个周期内处理 2 比特的输入数据，从而减少卷积运算的周期数。如果采用传统的阵列乘法器结构直接将 2 比特数据与权重进行相乘，计算单元的面积与功耗开销会很大，其不适合集成在大规模的阵列中。因此架构基于 Booth 乘法原理，首先对输入数据做预编码，生成若干个 Booth 运算的控制信号，这些控制信号会驱动计算单元进行相应的 Booth 操作，例如移位、反向、清零等，大大降低了计算单元的硬件实现复杂度，减小了存内计算阵列的面积与功耗开销，提高了系统的能耗效率。

4.3.1 Booth 乘法编码电路设计

将 m 比特有符号数 x 与 n 比特有符号数 y 进行相乘， n 比特有符号数 y 的二进制数补码可以表示成：

$$y = -2^{n-1}y_{n-1} + 2^{n-2}y_{n-2} + 2^{n-3}y_{n-3} + \cdots + 2^0y_0 \quad (4-1)$$

又由关系式 $2^n = 2^{n+1} - 2^n$ ，可以将上述公式重新复写为：

$$\begin{aligned}
 y &= -2^{n-1}y_{n-1} + (2^{n-1}y_{n-2} - 2^{n-2}y_{n-2}) + (2^{n-2}y_{n-3} - 2^{n-3}y_{n-3}) + \dots \\
 &= 2^{n-1}(y_{n-2} - y_{n-1}) + 2^{n-2}(y_{n-3} - y_{n-2}) + 2^{n-3}(y_{n-4} - y_{n-3}) + \dots \\
 &= 2^{n-2}(y_{n-3} + y_{n-2} - 2y_{n-1}) + 2^{n-4}(y_{n-5} + y_{n-4} - 2y_{n-3}) \dots + 2^0(y_{-1} + y_0 - 2y_1) \\
 &= \sum_{i=0}^{n-2} 2^i (y_{i-1} + y_i - 2y_{i+1}) \quad (4-2)
 \end{aligned}$$

根据表达式 (4-2) 我们可以看出, 通过编码 y 的连续三位, 我们可以得到部分积有以下 5 种情况: 0 、 x 、 $-x$ 、 $2x$ 与 $-2x$, 再根据当前处理位的位权, 将得到的部分积向左移动 i 位 (i 为偶数), 最后将所有的部分积求和, 得到 x 与 y 的乘积结果。同理, 本章提出的架构进行运算时, 输入数据从高位到低位每次按 3 比特位宽, 每次向右移动两位, 逐步将数据送入到 Booth 乘法存内计算处理器 (BM-CIM Macro) 中与权重值进行运算, 高位权的数据产生的部分和每次会向左移动两位, 通过多周期的反复迭代, 从而实现流水线型 Booth 乘法运算。

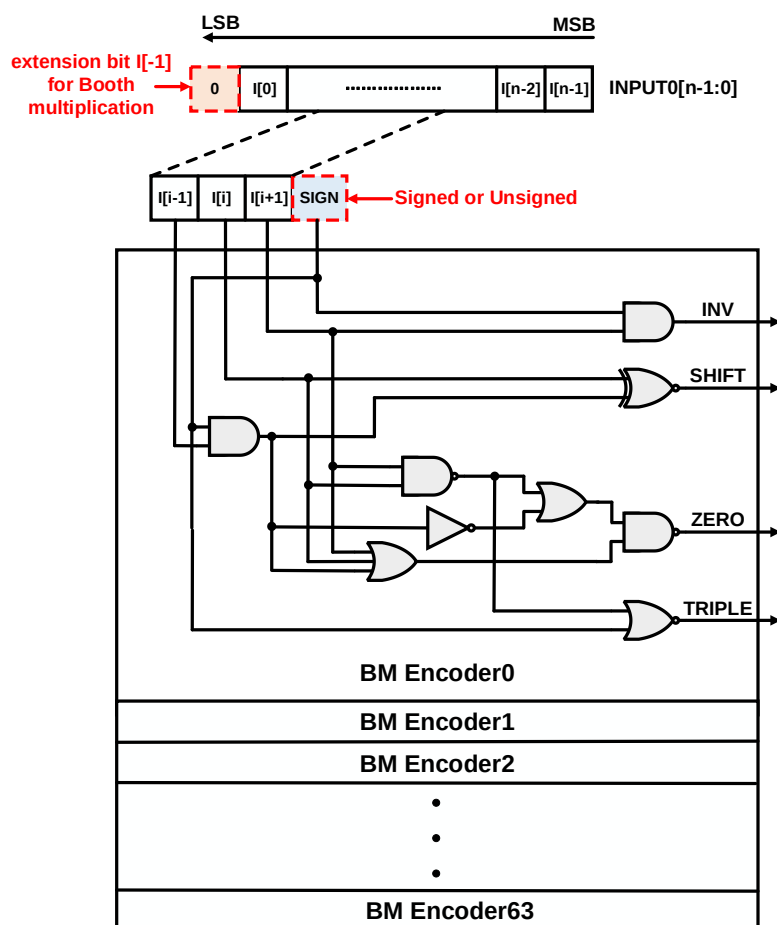


图 4-6 Booth 乘法编码电路

如图 4-6 所示, 为 Booth 乘法编码电路结构图, 我们在 n 比特输入 INPUT 的末尾补一位 “0”, 然后将 INPUT 从 MSB 到 LSB 每次按 3 比特位宽逐步输入到编码电路里。

编码电路依据输入数据的与符号标志位 **SIGN** 的值，分别编码出以下控制信号：反向使能信号 **INV**、移位使能信号 **SHIFT**、清零使能信号 **ZERO**、三倍乘使能信号 **TRIPLE**。这些信号通过进行组合来驱动 **Booth** 乘法计算单元与位扩展单元进行相应的操作，从而输出正确的部分积。

Booth 乘法编码电路的操作真值表如图 4-7(a)与 4-7(b)所示。当 **SIGN**=1 时，系统进行 **Booth** 乘法运算，编码电路依据 **Booth** 编码原理对输入数据进行 **Booth** 编码，可以看出如果 **ZERO**=1，不论其它 3 个控制信号的值如何，对乘数（权重值）清零。如果 **ZERO**=0，若仅有 **SHIFT**=1，将乘数扩大一倍；若仅有 **INV**=1，对乘数进行取补；若 **SHIFT** 和 **INV** 均为 1，则先将乘数扩大一倍，再取补码；若 **SHIFT** 和 **INV** 均为 0，不对乘数做任何操作。当 **SIGN**=0 时，系统进行算术乘法运算，编码模块只根据 3 比特输入的高两位进行算术编码。高两位数据为“00”时，对乘数清零；数据为“01”时，不对乘数操作；数据为“10”时，将乘数扩大一倍；数据为“11”时，将乘数扩大两倍。

SIGN=1(Booth Multiplication)					
INPUT[2:0]	ZERO	SHIFT	INV	TRIPLE	Operation(Weight is x)
000	1	-	-	-	0
001	0	0	0	0	x
010	0	0	0	0	x
011	0	1	0	0	2x
100	0	1	1	0	-2x
101	0	0	1	0	-x
110	0	0	1	0	-x
111	1	-	-	-	0

(a)

SIGN=0(Arithmetic Multiplication)					
INPUT[2:0]	ZERO	SHIFT	INV	TRIPLE	Operation(Weight is x)
000	1	-	-	-	0
001	1	-	-	-	0
010	0	0	0	0	x
011	0	0	0	0	x
100	0	1	0	0	2x
101	0	1	0	0	2x
110	0	0	0	1	3x
111	0	0	0	1	3x

(b)

图 4-7 Booth 乘法编码电路的操作真值表

4.3.2 Booth 乘法计算单元电路设计

如图 4-8 所示，展示了本设计中 **Booth** 乘法存内计算数据块（**BMCD**）的电路示意图。每个 **BMCD** 由 4×4 的 6T SRAM 子阵列和一个 **Booth** 乘法计算单元（**BMCC**）组成。**BMCD** 可以工作在两种模式下：存储模式与计算模式。

在存储模式下，**BMCC** 内的双路到单路转换器（Dual to single converter, **DTSC**）不使能，无法采样位线上的权重值，并且由于系统不使能输入计算数据的采样，因此驱动 **BMCC** 的控制信号 **SHIFT** 与 **INV** 均为“0”，所以 **BMCC** 的输出 **Q[4:0]** 始终为低电平。 4×4 大小的 SRAM 子阵列可以通过开启字线正常进行读写操作。

在计算模式下，DTSC 使能，子阵列开启字线 WL[3]-WL[0]的其中一根，将子阵列中其中一行存储的 4 比特权重值读出到 BMCC 中，经由 DTSC 采样，权重值通过总线 GBL[3:0]输入到 5 比特移位器中。移位器根据移位使能信号的值，判断是否将 GBL[3:0]左移一位。若 SHIFT=1，将权重值向左移一位，最低位补“0”。若 SHIFT=0，根据符号标志位 SIGN 对权重值进行位扩展，SIGN=1 时，则进行符号位扩展，SIGN=0 时，则进行零扩展。接着移位器将处理完的 5 比特数据送到异或门单元的其中一端，异或门单元的另一端输入为反向使能信号 INV。由异或逻辑表达式可得 $A \oplus INV = A' \cdot INV + A \cdot INV'$ 。INV=1 时， $A \oplus INV = A'$ ；INV=0 时， $A \oplus INV = A$ 。因此异或门单元根据反向使能信号 INV 的值对 5 比特数据进行取反，最终输出 Q[4:0]。

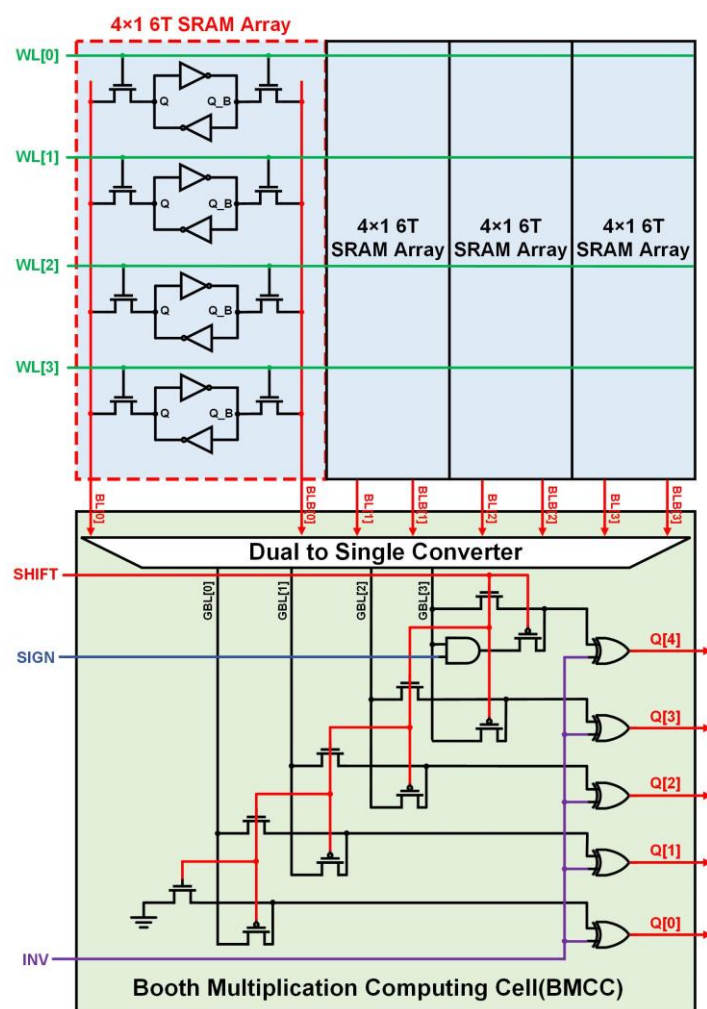


图 4-8 Booth 乘法存内计算数据块电路示意图

4.3.3 位扩展单元设计

权重数据在 Booth 乘法计算单元 (BMCC) 经过初步处理，得到 5 比特的扩展信号 Q，Q 接着要输入到位扩展单元中做进一步处理，从而得到输入数据与权重值相乘

的部分积。位扩展单元的控制信号也由输入编码模块产生，分别是清零使能信号 ZERO、三倍乘使能信号 TRIPLE 以及反向使能信号 INV。位扩展单元具有固定优先级仲裁功能，对于三种控制信号请求，ZERO 的仲裁级别最高，INV 的仲裁级别最低。如图 4-9 所示为位扩展单元的工作流程，输入数据 Q 送到位扩展单元内处理，位扩展单元根据固定优先级响应控制信号并对 Q 做相应的操作。首先判断 ZERO 是否为“1”，若满足该约束，则将输出信号 $P[5:0]$ 清零。若不满足，则继续判断 TRIPLE 是否为“1”，若满足该约束，则将输入信号 $Q[4:0]$ 扩大两倍并输出。若仍不满足，则最后判断 INV 是否为“1”，若满足该约束，则意味着此时处理的输入 Q 为有符号数，且在 BMCC 单元中经过反向处理，因此对 $Q[4:0]$ 加一完成取补码的操作并进行符号位扩展。若以上约束均不满足，则直接对 $Q[4:0]$ 做零扩展后输出。经过位扩展单元仲裁处理后，输出 $P[5:0]$ 即为输入数据与权重值相乘的部分积。

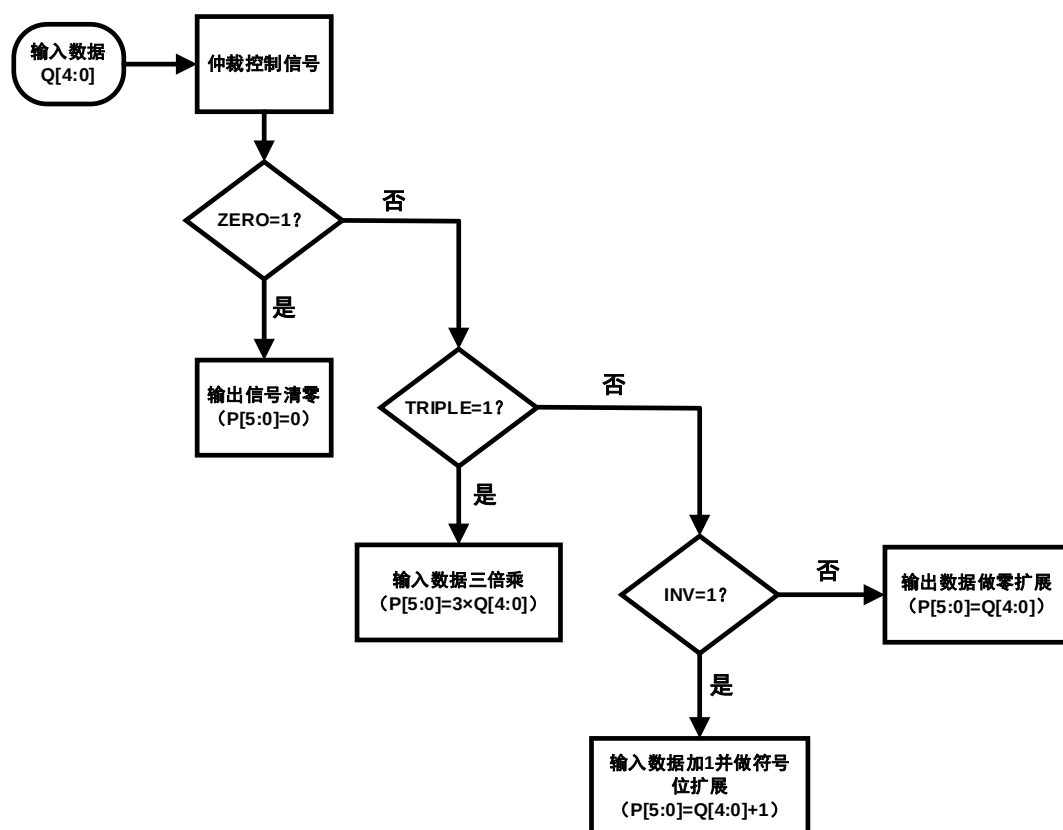


图 4-9 位扩展单元工作流程

4.3.4 时钟控制电路与地址译码器电路设计

本设计中的时钟控制电路如图 4-10 所示，其电路结构与时序控制原理与第三章提出的时钟控制电路大致相同，添加了卷积计算的标志信号 ACTIVE。当 ACTIVE 为低电平时，系统处于存储模式，电路根据片选使能 CEB 与读写使能 WEB 生成控制信号

驱动存内计算阵列进行读写操作。当 ACTIVE 为高电平时，系统处于计算模式，将 DEC1、DEC0、Y、CKD 与 SAE 信号全部清零，确保存内计算阵列不开启列选信号并且关闭读写驱动电路。此时仅有字线 WL 信号采样使能，根据 WL 信号的值开启存内计算阵列的某一行将一组权重集读出到 Booth 乘法计算单元 (BMCC) 中，同时根据冗余阵列位线放电的速度，当 TRKBL 放电到“0”时，BMCC 的使能信号 BMCC_EN 变为电平，将存内计算阵列位线上的权重值采样进来，对其进行 Booth 运算操作。

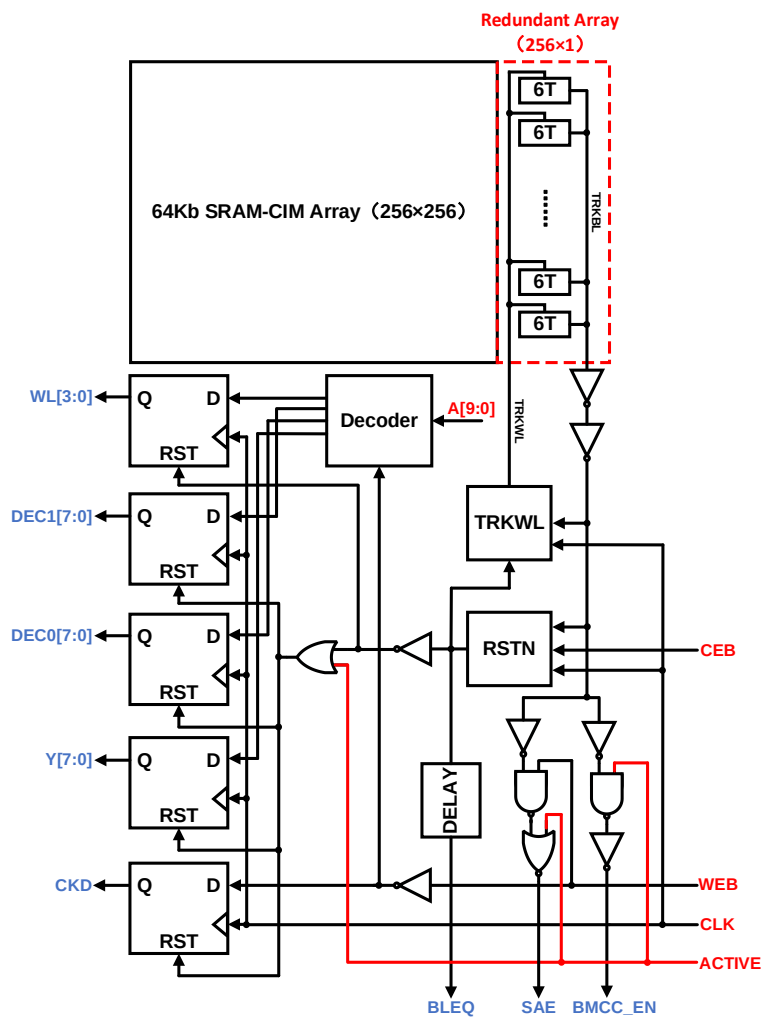


图 4-10 添加计算标志位的时钟控制电路

10 比特地址信号 A 的组成相对第三章里提出的有所改变，其主要分为三个部分：A[9:8] 为行地址信息，负责译码出字线信号 WL[3:0]；A[7:2] 为列地址信息，其中 A[7:5] 负责译码出 DEC1[7:0]，A[4:2] 负责译码出 DEC2[7:0]，两组列地址预解码信号送入到列地址译码器做进一步预解码，如图 4-11 所示，编译出列选信号 CSL[63:0]；A[1:0] 为 MUX 控制信息，与读写使能 WEB 一起编译出多路选择器控制信号 Y[7:0]。

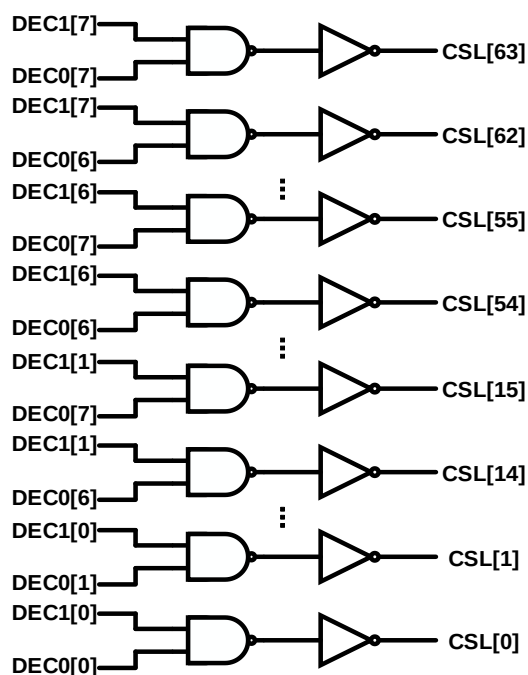


图 4-11 列地址译码器

4.4 电路性能仿真与分析

依据以上的系统时序分析及电路原理分析，仍基于台积电 28nm（tsmc28nm）的工艺制程搭建本章架构并对系统的功能与性能进行了完整的仿真分析。首先验证了 Booth 编码模块与 Booth 计算单元联动的计算功能正确性，然后对 Booth 计算单元的延时与功耗在不同电源电压下进行了完备的分析。最后，归纳了整个系统的计算延时与计算功耗，并分析了不同电源电压及不同卷积计算组数的情况下系统能耗效率的变化趋势。

4.4.1 Booth 乘法存内计算数据块性能仿真

为了检验一次完整 Booth 运算功能的准确性，我们将 Booth 乘法存内计算数据块（SRAM 子阵列+复用 Booth 计算单元）、Booth 编码模块及位扩展单元联合仿真，并分析 Booth 乘法存内计算数据块（BMCDB）的计算延时及计算功耗。如图 4-12 所示，为 BMCDB 进行计算操作时的瞬态仿真波形图。进行计算操作之前，我们在 SRAM 子阵列（ 4×4 ）内预先存入 4 组权重值，分别是 0111、0111、1000 以及 0001。然后在连续四个操作周期内依次将 4 组权重值送入 BMCDB 中进行运算，4 个周期内的输入数据分别是 110、110、111 与 010。第一个周期与第二个周期内 110 的 Booth 编码结果为“-x”，因此对 0111 取补码并进行符号位扩展，得到 6 位中间积 111001。第三个周

期内 111 的 Booth 编码结果为“0”，从而中间积输出清零，为 000000。第四个周期内 010 的 Booth 编码结果为“x”，因此中间积为 0001 的符号位扩展，即 000001。

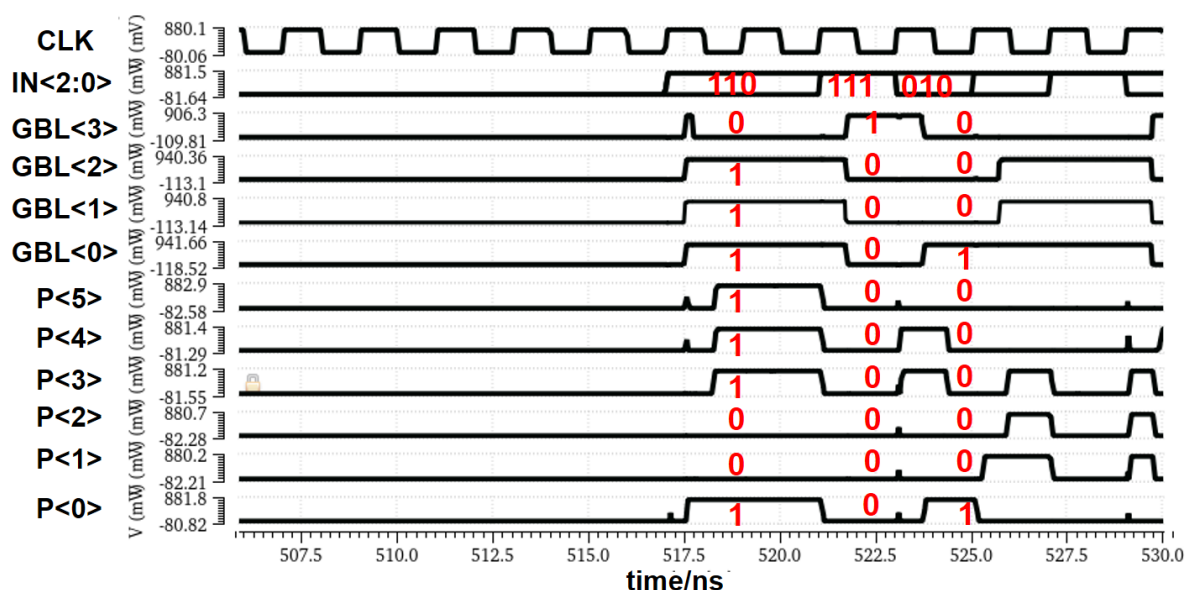


图 4-12 BMCDB 电路的瞬态仿真

此外，分别评估在 0.7V、0.8V 与 0.9V 电源电压下，BMCDB 进行卷积运算时所消耗的功耗及所需要的延时。如图 4-13(a)与 4-13(b)所示，可以看出 0.7V 电压下 BMCDB 的计算功耗很低，仅有 2.808uW，但是计算速度过慢，需要 2.24ns 才能完成中间积的乘法运算，导致系统计算频率达不到 500MHz。在 0.9V 电压下，计算仅需 0.858ns，但单个 BMCDB 消耗的功耗过大，达到 10.82uW，不利于提升整个系统的能耗效率。初步看出 0.8V 的电压下，BMCDB 的功耗指标与速度性能之间平衡的较好，还需要根据系统的完整仿真才能确定架构最佳的供电电压。

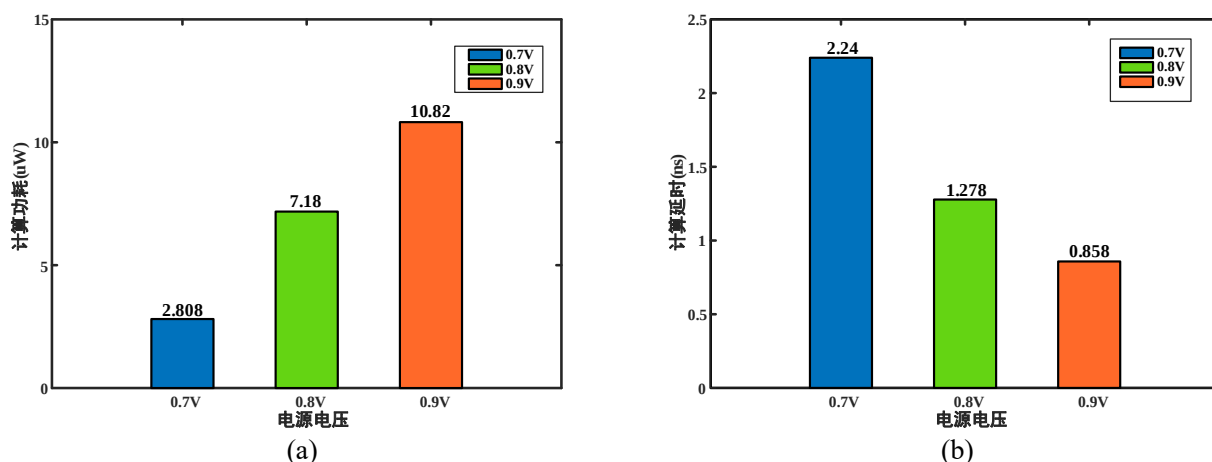


图 4-13 BMCDB 性能仿真分析：(a)计算功耗对比 (b)计算速度对比

4.4.2 系统性能仿真与分析

首先分别验证整个架构的写操作与卷积计算操作的功能正确性。如图 4-14 所示，为系统进行写权重操作时的瞬态仿真波形图。时钟频率为 2GHz，在 512ns 内将 64Kb 大小的存内计算阵列全部存满，验证第一列的第 4 行与第 3 行存储的数据是否正确。系统进行写操作时，WEB 置为“0”，BWEB 为“0”时选中某一列进行写操作。可以看出第 4 行与第 3 行存入的值均为 0111。

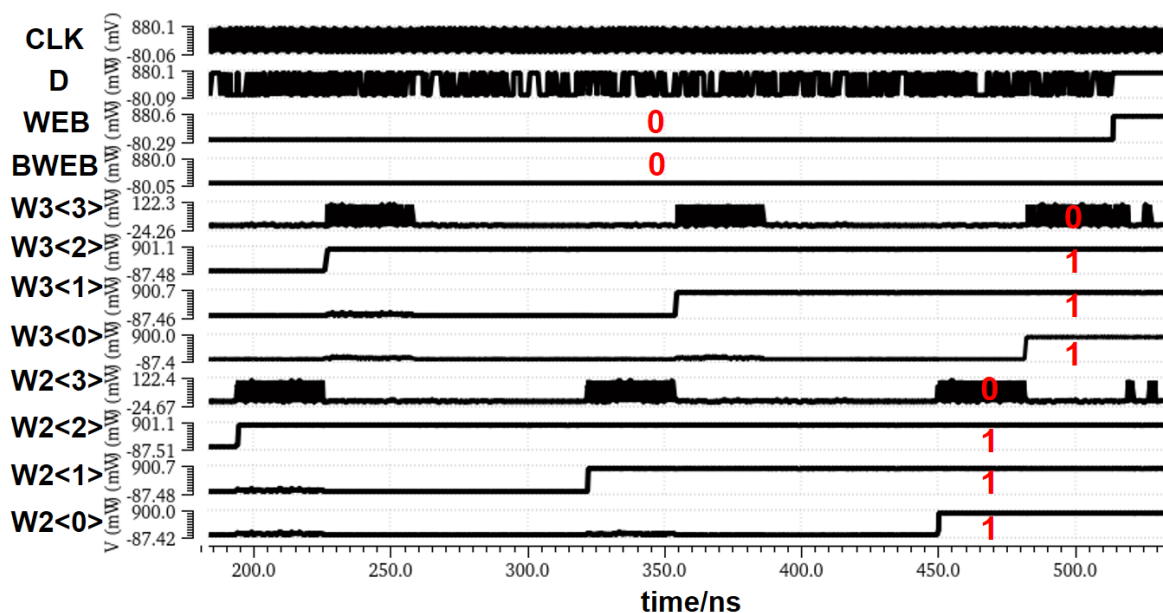


图 4-14 系统写操作过程的瞬态仿真

如图 4-15 所示，为系统进行卷积运算操作时的瞬态仿真波形图。测试时所有输入数据 IN 为 4 比特，卷积计算组数为 64×64 组，输入数据从 MSB 开始每个操作周期输入 2 比特到架构中进行运算。位权为 IN<3:2>的输入在连续的四个操作周期内依次输入到架构中通过乘累加运算求出相应的部分和，待第五个周期输入位权变化成 IN<1:0>时，将 IN<3:2>产生的部分和向左移动两位，并与 IN<1:0>产生的部分和累加，最终经过九个操作周期输出最终的 20 位卷积计算结果 R。从波形图可以看出，517ns 时将计算有效信号 ACTIVE 置为“1”，同时 SIGN 符号标志位有效，此时系统开始进行基于 Booth 编码的乘累加运算。在计算过程中多组中间积 P 与部分和 PSUM 不断翻转，经历 8 个操作周期后 4 比特 IN 全部输入完成，移位累加器将计算完成信号 DONE 置为“1”，使能累加器内的触发器采样最终计算结果，在 535ns 时输出 20 位的卷积计算结果 R。

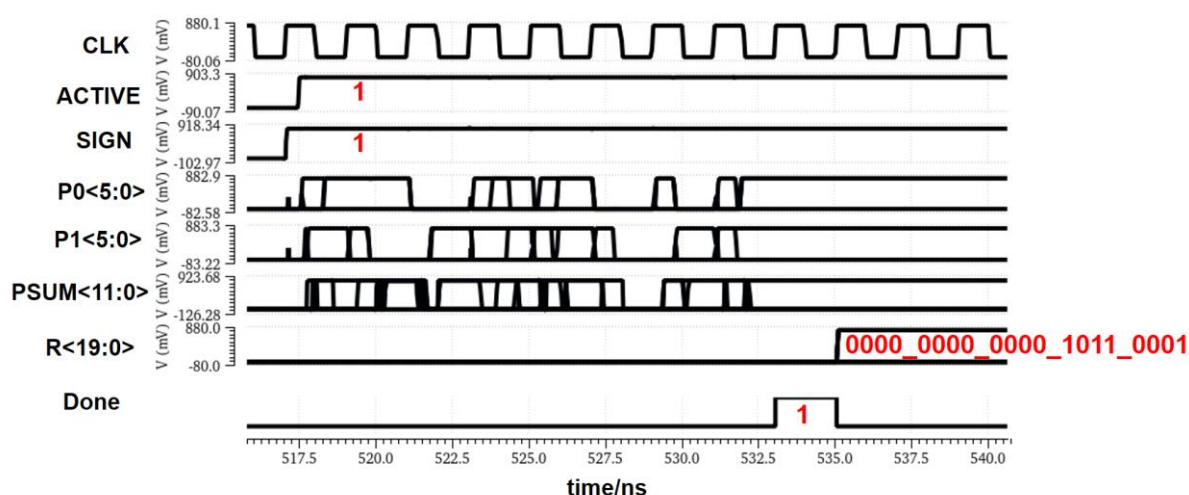
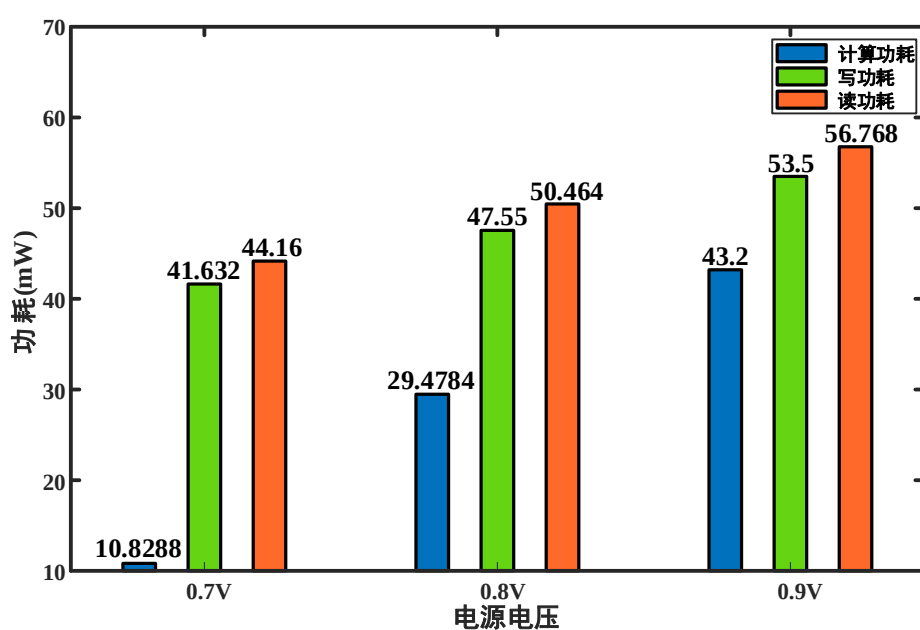
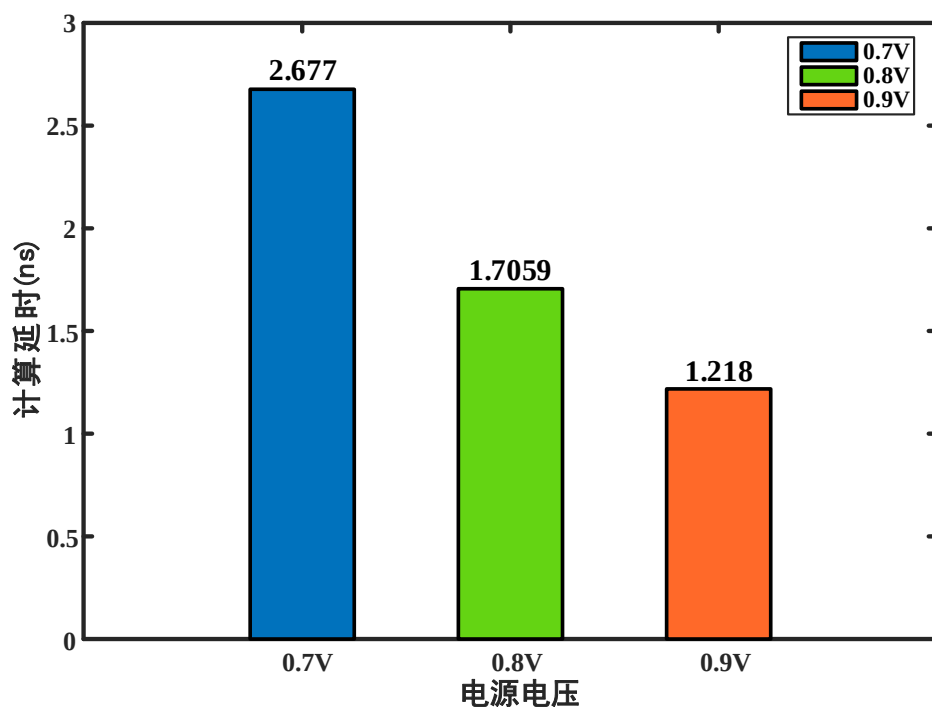


图 4-15 系统计算操作过程的瞬态仿真

此外，分别评估在 0.7V、0.8V 与 0.9V 三种电源电压下，系统处理 4-bit 输入与 4-bit 权重的卷积运算所消耗的功耗及所需要的延时。如图 4-16(a)所示，为不同电压下系统三种操作功耗的对比。可以看出不同电压下系统的读功耗与写功耗相差 3mW 左右，而计算功耗始终小于读写功耗，并且在 0.7V 电压下计算功耗仅有 10.828mW，在 0.9V 电压下计算功耗达到 43.2mW。如图 4-16(b)所示，对比了不同电压下系统的计算速度，可以看出虽然 0.7V 电压下系统消耗的能耗很低，但是其计算速度非常慢，从数据输入到加法器树输出部分和需要耗费 2.677ns，这导致系统的计算频率达不到 500MHz，严重降低了系统的算力性能。相反 0.9V 电压下计算所需延时短，仅需 1.218ns，系统的吞吐率很高，但是功耗开销较大。



(a)



(b)

图 4-16 (a) 不同电压下系统的功耗对比 (b) 不同电压下系统的计算速度对比

如图 4-17 所示，为系统内不同模块的计算功耗占比，对比第三章里架构的计算功耗占比图可以看出，由于本章架构基于分时复用 Booth 计算单元的策略，因此一个操作周期内并行计算的组数减小，导致加法器树的规模减小，降低了加法器树的面积与功耗开销，其计算功耗在整个系统内占比 21%。同时由于 Booth 计算单元相比于或非门单元复杂很多，导致存内计算阵列的功耗成为系统的主要功耗来源，占比 46%。整个架构由于采取分时复用策略，计算过程中其消耗的能耗相较于第三章提出的架构降低很多。

Power Consumption

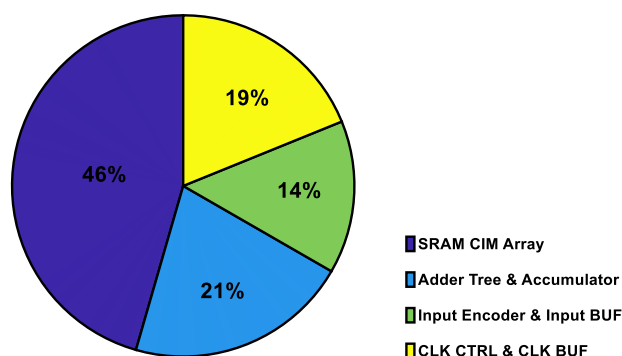


图 4-17 系统内各模块的计算功耗占比

综合考虑系统的计算功耗与计算速度，我们得出系统的吞吐率与能耗效率随电压的变化趋势。如图 4-18 所示，随着电压的增大，系统的计算速度变快，因此在单位时间内系统完成的计算次数更多，即系统的吞吐率提高。同时，由于电压增大，系统计算所消耗的功耗更高，并且功耗的增量大于算力的增量，因此随着电压的增大系统的能耗效率减小。从图中可以看出，电压小于 0.75V 时系统的能耗效率达到 75TOPS/W 以上，但是系统的算力性能低于 1TOPS，计算速度过慢。而当电压高于 0.85V 时系统的算力超过 2TOPS，但是其能耗效率不够理想，低于 60TOPS/W。综上所述，最终确定系统的最佳供电电压为 0.8V，让系统在处理 4-bit 输入和 4-bit 权重卷积运算时获得 1.82TOPS 的吞吐率与 61.745TOPS/W 的能耗效率。

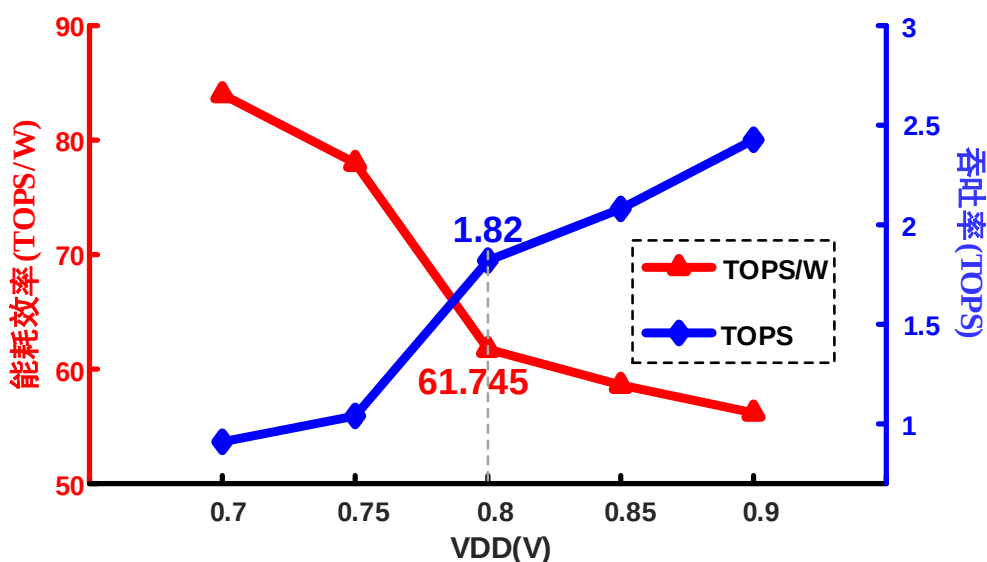


图 4-18 系统的吞吐率与能耗效率随电源电压的变化趋势

本章提出的架构可以灵活配置卷积计算组数，卷积计算组数可达 $64 \times n \times 64$ (n 最大为 4)。如图 4-19 所示，分别测量 $n=1$ 、 $n=2$ 、 $n=3$ 及 $n=4$ 时，系统的吞吐率及能耗效率的值。 $n=1$ 时，系统的算力为 1.365TOPS，能耗效率为 46.31TOPS/W； $n=4$ 时，系统的算力为 1.82TOPS，能耗效率为 61.74TOPS/W。可以看出随着计算组数增加，系统的吞吐率与能耗效率均增大，面对大规模神经网络算法时，架构的利用效益更高。

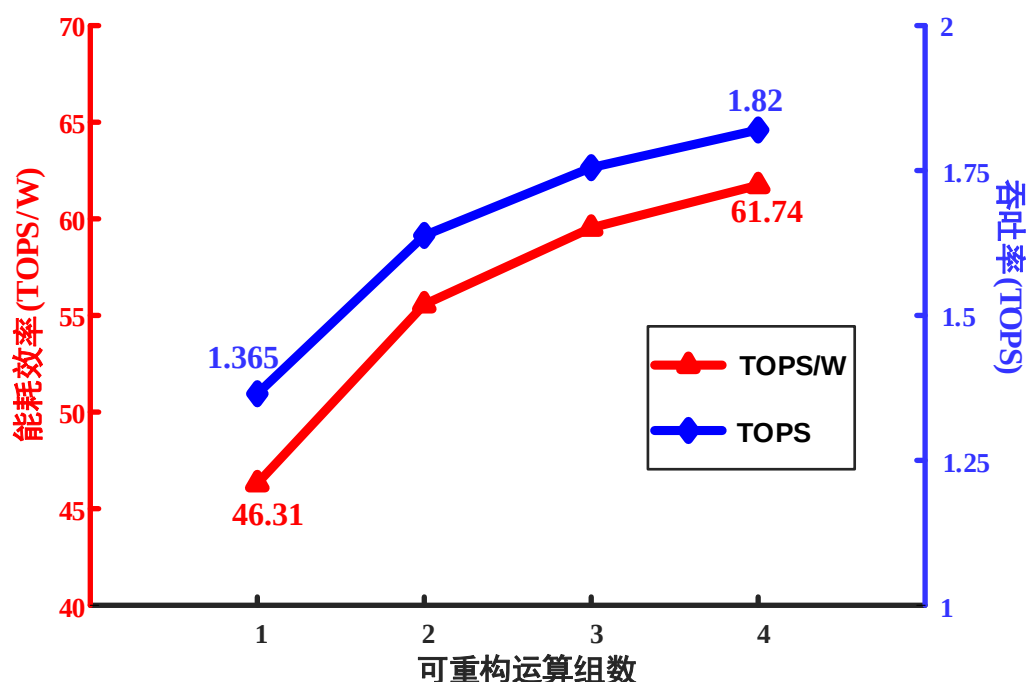


图 4-19 系统的吞吐率与能效效率随卷积计算组数的变化

综合以上仿真分析，对基于复用 Booth 计算单元和 6T SRAM 的可重构存内计算设计的各项性能指标进行总结，并与以往提出的存内计算架构及第三章提出的串行单比特存内计算架构进行对比。如表 4-1 所示，可以看出模拟存内计算设计的能效效率性能指标普遍优于数字存内计算设计，并且系统不需要高速时钟即可完成运算。但是模拟存内计算设计可支持的输入数据与权重数据的位宽较小，并且计算精度始终达不到 100%。而数字存内计算设计可以支持高数据位宽，并且支持数据位宽的灵活调整。本章的存内计算架构由于采用分时复用策略，在吞吐率性能上相较于串行单比特存内计算架构有所降低，但是显著降低了架构运算时消耗的能耗，最终系统的能效效率得到提升。

表 4-1 基于复用 Booth 计算单元和 6T SRAM 存内计算架构的性能总结与对比

	VLSI'21 ^[51]	JSSCC'22 ^[49]	CICC'22 ^[23]	ESSCIR C'19 ^[33]	第三章架构	本章架构
工艺制程	28nm	65nm	45nm	65nm	28nm	28nm
存内计算 类型	Analog CIM	Analog CIM	Analog CIM	Digital CIM	Digital CIM	Digital CIM
存内计算 阵列大小	3456Kb	16Kb	128Kb	16Kb	64Kb	64Kb
存内计算 单元类型	10T1C	8T	6T	6T	6T+1NOR	6T+BMCC
供电电压 (V)	1.0	1.05	0.8	0.8	0.9	0.8
时钟频率 (MHz)	40	50-100	25	N/A	500	500
输入精度 (bit)	1b/2b	2b/4b/6b/8b	6b	1b~16b	1b-8b	2b/4b/6b/8b
权重精度 (bit)	1b/2b	4b/8b	2b	4b/8b/12 b/16b	4b/8b/12b/16b	4b/8b
吞吐率 (GOPS)	4900(1b/1b)	2000(2b/4b)	102.4	567(1b/1 b)	3300(4b/4b) 917(8b/8b)	1820(4b/4b) 482(8b/8b)
能耗效率 (TOPS/W)	588(1b/1b)	158.7(2b/4b)	42	117.3(1b /1b)	56.39(4b/4b) 15.67(8b/8b)	61.745(4b/4b) 21.64(8b/8b)

4.5 本章小结

本章提出一种基于复用 Booth 计算单元和 6T SRAM 的存内计算架构，其在第三章提出的串行单比特存内计算架构的基础上，依旧采取流水线的方式实现卷积计算，并基于 Booth 乘法的原理，对输入数据进行预编码，从而架构支持单周期内串行输入 3-bit 数据。同时架构采取分时复用策略，每四行 SRAM 复用一個 Booth 计算单元，虽然减少了一个操作周期内并行运算的组数，但是降低了加法器树的面积与功耗开销，并且架构支持卷积计算组数的可重构，可实现并行计算的组数最少达到 64×64 组，最多达到 $64 \times 4 \times 64$ 组。架构支持输入数据位宽为 2-bit/4-bit/6-bit/8-bit，权重数据位宽为

4-bit/8-bit, 同时在 t_t 工艺角/0.8V 供电电压下, 架构实现了 482GOPS (8b/8b) 到 1820GOPS (4b/4b) 的吞吐率及 21.64TOPS/W (8b/8b) 到 61.745TOPS/W (4b/4b) 的能耗效率。

第5章 总结与展望

5.1 全文总结

随着人工智能和物联网等应用的快速发展及大规模普及，边缘端的嵌入式设备处理的计算数据量越来越庞大，导致人们对这些设备的算力及能效要求越来越高。然而传统的冯诺依曼架构由于存在“功耗墙”及“存储墙”的问题，导致智能化设备进行计算时产生了大量的数据迁移功耗且计算带宽受到存储器带宽的限制，因此严重限制了计算设备的吞吐率和能效效率的提升。存内计算架构是一种新型计算架构，它将部分计算单元嵌入到存储器内部，既保留了存储器原本的存储和读写功能，又让存储器具备一定的计算功能。存储器内部的数据可以不用传输到处理器中，直接在存储器内进行逻辑运算或乘累加运算等，将运算结果再传输到处理器中，从而很大程度地减少了数据的迁移量和迁移频率，并且计算带宽不受存储器带宽的限制，因此提高了系统的算力性能与能效效率。数字存内计算架构不仅可以实现高数据位宽的计算，并且计算精度始终为 100%，同时可以支持架构的灵活配置，从而应对不同数据位宽或不同运算规模的神经网络算法应用。本文基于数字存内计算架构的实现方式，提出了两种算力较高，能效效率良好的存内计算架构，可应用于神经网络中的多比特卷积运算加速场景。

本文首先提出一种基于 6T SRAM 的串行单比特输入存内计算架构，架构基于位串行单比特输入的数据输入方式，将卷积运算通过两级流水线的方式在多个操作周期内实现。第一级流水线负责将多组 1-bit 输入数据与多比特权重值相乘并将得到的中间积通过 11T 与 28T 交错运行的加法器树累加起来，累加得到的部分和暂存在寄存器中。第二级流水线负责对前几个操作周期得到的累加值向左移动一位，并与寄存器中存储的当前操作周期的部分和相加。由于采取流水线的方式，架构支持灵活配置输入数据的数据位宽与权重数据的数据位宽，输入数据的数据位宽最高可达 8-bit，权重数据的数据位宽最高可达 16-bit。该存内计算阵列的大小为 64Kb，一个操作周期内支持 256×64 组输入与权重数据并行进行运算，最高可同时并行输出 64 组卷积计算结果。该架构采用台积电标准 28nm CMOS 工艺进行了电路设计与后端版图设计，后仿结果表明架构在 4-bit 输入数据、4-bit 权重数据、16-bit 卷积运算结果的情况下获得 3300GOPS 的吞吐率与 56.39TOPS/W 的能效效率，在 8-bit 输入数据、8-bit 权重数据、24bit 卷积

运算结果的情况下获得 917GOPS 的吞吐率与 15.67TOPS/W 的能耗效率。

接着, 本文通过引入 Booth 乘法的原理并且依旧基于位串行输入的卷积计算结构, 提出一种基于复用 Booth 计算单元与 6T SRAM 的可重构存内计算架构。架构支持操作周期内串行输入 3-bit 数据, 通过 Booth 编码模块预先对输入数据进行 Booth 编码, 从而得到 Booth 操作的计算控制信号, 再由计算控制信号驱动 Booth 乘法计算单元 (BMCC) 与位扩展单元进行相应的操作, 从而得到部分积。架构仍采用两级流水线的方式实现多比特卷积运算, 同时架构采取分时复用 BMCC 计算单元的策略并且设计一种可配置移位周期的移位累加器, 令架构支持卷积计算组数的可重构。一个操作周期内支持最多支持 256×64 组输入与权重数据并行运算, 最少支持 64×64 组数据并行运算, 因此架构可灵活应用于不同规模的神经网络算法。架构可支持输入数据的数据位宽为 2-bit/4-bit/6-bit/8-bit, 权重数据的数据位宽为 4-bit/8-bit。使用台积电 28nm 搭建整个存内计算设计电路, 通过完备的仿真分析得出该架构在 4-bit 输入、4-bit 权重、14bit 卷积运算结果的情况下实现 1365GOPS 的吞吐率与 46.31TOPS/W 的能耗效率, 在 4-bit 输入、4-bit 权重、16-bit 卷积运算结果的情况下实现 1820GOPS 的吞吐率与 61.74TOPS/W 的能耗效率, 在 8-bit 输入、8-bit 权重、24-bit 卷积运算结果的情况下实现 482GOPS 的吞吐率与 21.64TOPS/W 的能耗效率。

5.2 未来研究展望

本文重点研究了高吞吐率、良好能耗效率的全精度数字存内计算电路的设计, 提出的两种架构分别通过高并行度计算与分时复用计算的策略, 在算力性能与能耗效率上表现良好。但是两种存内计算架构仅支持多比特卷积运算操作, 并且仅支持整数型数据的乘累加运算, 此外架构中加法器树的面积占比仍然较大, 限制了架构能效的进一步提升, 因此可以从以下方向展开进一步研究:

1. 充分利用数字存内计算架构的可配置性特点, 研究可配置运算操作类型的数字存内计算架构, 例如同时支持乘累加和逻辑累加操作, 让存内计算架构不仅可以应用于卷积神经网络, 还可以应用于二值神经网络等。

2. 研究支持浮点数运算的数字存内计算架构, 从而应对更复杂的神经网络算法。

3. 加法器树是限制数字存内计算能耗效率的主要因素, 考虑如何在保证算力性能的前提下减小加法器树的面积与功耗开销。

参考文献

- [1] Hinton G, Deng L, Yu D, et al. Deep Neural Networks for Acoustic Modeling in Speech Recognition: The Shared Views of Four Research Groups[J]. IEEE Signal Processing Magazine, 2012, 29(6): 82-97.
- [2] Chen Y H, Krishna T, Emer J, et al. 14.5 Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks[C]. IEEE International Solid-State Circuits Conference (ISSCC). 2016: 262-263.
- [3] Moons B, Verhelst M. An Energy-Efficient Precision-Scalable ConvNet Processor in 40-nm CMOS[J]. IEEE Journal of Solid-State Circuits, 2017, 52(4): 903-914.
- [4] 郭昕婕, 王光耀, 王绍迪. 存内计算芯片研究进展及应用[J]. 电子与信息学报, 2023, 45(5): 1888-1898.
- [5] 蔺智挺, 徐田, 童忠瑛, 等. 基于静态随机存取存储器的存内计算研究进展[J]. 电子与信息学报, 2022, 44(11): 4041-4057.
- [6] Chen C L P, Zhang C Y. Data-intensive applications, challenges, techniques and technologies: A survey on Big Data[J]. Information Sciences, 2014, 275.
- [7] Akin B, Franchetti F, Hoe J C. Data reorganization in memory using 3D-stacked DRAM[C]. ACM/IEEE 42nd Annual International Symposium on Computer Architecture (ISCA). 2015: 131-143.
- [8] Gao M, Ayers G, Kozyrakis C. Practical Near-Data Processing for In-Memory Analytics Frameworks[C]. International Conference on Parallel Architecture and Compilation (PACT). 2015: 113-124.
- [9] Farmahini-Farahani A, Ahn J H, Morrow K, et al. NDA: Near-DRAM acceleration architecture leveraging commodity DRAM devices and standard memory modules[C]. IEEE 21st International Symposium on High Performance Computer Architecture (HPCA). 2015: 283-295.
- [10] Horowitz M. 1.1 Computing's energy problem (and what we can do about it)[C]. IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC). 2014: 10-14.
- [11] Sze V, Chen Y H, Emer J, et al. Hardware for machine learning: Challenges and opportunities[C]. IEEE Custom Integrated Circuits Conference (CICC). 2018: 1-8.

- [12] Zidan M A, Strachan J P, Lu W D. The future of electronics based on memristive systems[J]. Nature Electronics, 2018, 1(1).
- [13] Chih Y D, Lee P H, Fujiwara H, et al. 16.4 An 89TOPS/W and 16.3TOPS/mm² All-Digital SRAM-Based Full-Precision Compute-In Memory Macro in 22nm for Machine-Learning Edge Applications[C]. IEEE International Solid- State Circuits Conference (ISSCC). San Francisco, CA, USA: IEEE, 2021: 252-254.
- [14] Si X, Chen J J, Tu Y N, et al. 24.5 A Twin-8T SRAM Computation-In-Memory Macro for Multiple-Bit CNN-Based Machine Learning[C]. IEEE International Solid- State Circuits Conference - (ISSCC). San Francisco, CA, USA: IEEE, 2019: 396-398.
- [15] Mori H, Zhao W C, Lee C E, et al. A 4nm 6163-TOPS/W/b $\mathbf{4790-TOPS/mm^2/b}$ SRAM Based Digital-Computing-in-Memory Macro Supporting Bit-Width Flexibility and Simultaneous MAC and Weight Update[C]. IEEE International Solid-State Circuits Conference (ISSCC). 2023: 132-134.
- [16] Fujiwara H, Mori H, Zhao W C, et al. A 5-nm 254-TOPS/W 221-TOPS/mm² Fully-Digital Computing-in-Memory Macro Supporting Wide-Range Dynamic-Voltage-Frequency Scaling and Simultaneous MAC and Write Operations[C]. IEEE International Solid- State Circuits Conference (ISSCC). San Francisco, CA, USA: IEEE, 2022: 1-3.
- [17] Lee C F, Lu C H, Lee C E, et al. A 12nm 121-TOPS/W 41.6-TOPS/mm² All Digital Full Precision SRAM-based Compute-in-Memory with Configurable Bit-width For AI Edge Applications[C]. IEEE Symposium on VLSI Technology and Circuits (VLSI Technology and Circuits). Honolulu, HI, USA: IEEE, 2022: 24-25.
- [18] Wang J, Wang X, Eckert C, et al. A 28-nm Compute SRAM With Bit-Serial Logic/Arithmetic Operations for Programmable In-Memory Vector Computing[J]. IEEE Journal of Solid-State Circuits, 2020, 55(1): 76-86.
- [19] Khwa W S, Chen J J, Li J F, et al. A 65nm 4Kb algorithm-dependent computing-in-memory SRAM unit-macro with 2.3ns and 55.8TOPS/W fully parallel product-sum operation for binary DNN edge processors[C]. IEEE International Solid-State Circuits Conference - (ISSCC). 2018: 496-498.
- [20] Si X, Chang M F, Khwa W S, et al. A Dual-Split 6T SRAM-Based Computing-in-Memory Unit-Macro With Fully Parallel Product-Sum Operation for Binarized DNN Edge Processors[J]. IEEE Transactions on Circuits and Systems I: Regular Papers, 2019, 66(11): 4172-4185.
- [21] Si X, Tu Y N, Huang W H, et al. A Local Computing Cell and 6T SRAM-Based

- Computing-in-Memory Macro With 8-b MAC Operation for Edge AI Chips[J]. IEEE Journal of Solid-State Circuits, 2021, 56(9): 2817-2831.
- [22] Si X, Chen J J, Tu Y N, et al. A Twin-8T SRAM Computation-in-Memory Unit-Macro for Multibit CNN-Based AI Edge Processors[J]. IEEE Journal of Solid-State Circuits, 2020, 55(1): 189-202.
- [23] Biswas A, Sanghvi H, Mehendale M, et al. An area-efficient 6T-SRAM based Compute-In-Memory architecture with reconfigurable SAR ADCs for energy-efficient deep neural networks in edge ML applications[C]. IEEE Custom Integrated Circuits Conference (CICC). 2022: 1-2.
- [24] Agrawal A, Jaiswal A, Roy D, et al. Xcel-RAM: Accelerating Binary Neural Networks in High-Throughput SRAM Compute Arrays[J]. IEEE Transactions on Circuits and Systems I: Regular Papers, 2019, 66(8): 3064-3076.
- [25] Biswas A, Chandrakasan A P. CONV-SRAM: An Energy-Efficient SRAM With In-Memory Dot-Product Computation for Low-Power Convolutional Neural Networks[J]. IEEE Journal of Solid-State Circuits, 2019, 54(1): 217-230.
- [26] Zhang J, Wang Z, Verma N. In-Memory Computation of a Machine-Learning Classifier in a Standard 6T SRAM Array[J]. IEEE Journal of Solid-State Circuits, 2017, 52(4): 915-924.
- [27] Jiang Z, Yin S, Seok M, et al. XNOR-SRAM: In-Memory Computing SRAM Macro for Binary/Ternary Deep Neural Networks[C]. IEEE Symposium on VLSI Technology. 2018: 173-174.
- [28] Yin S, Jiang Z, Seo J S, et al. XNOR-SRAM: In-Memory Computing SRAM Macro for Binary/Ternary Deep Neural Networks[J]. IEEE Journal of Solid-State Circuits, 2020, 55(6): 1733-1743.
- [29] Liu Q, Gao B, Yao P, et al. 33.2 A Fully Integrated Analog ReRAM Based 78.4TOPS/W Compute-In-Memory Chip with Fully Parallel MAC Computing[C]. IEEE International Solid-State Circuits Conference - (ISSCC). 2020: 500-502.
- [30] Wan W, Kubendran R, Gao B, et al. A Voltage-Mode Sensing Scheme with Differential-Row Weight Mapping for Energy-Efficient RRAM-Based In-Memory Computing[C]. IEEE Symposium on VLSI Technology. 2020: 1-2.
- [31] Wu T F, Li H, Huang P C, et al. Brain-inspired computing exploiting carbon nanotube FETs and resistive RAM: Hyperdimensional computing case study[C]. IEEE International Solid-State Circuits Conference - (ISSCC). 2018: 492-494.

- [32] Judd P, Albericio J, Moshovos A. Stripes: Bit-Serial Deep Neural Network Computing[J]. IEEE Computer Architecture Letters, 2017, 16(1): 80-83.
- [33] Kim H, Chen Q, Yoo T, et al. A 1-16b Precision Reconfigurable Digital In-Memory Computing Macro Featuring Column-MAC Architecture and Bit-Serial Computation[C]. IEEE 45th European Solid State Circuits Conference (ESSCIRC). Cracow, Poland: IEEE, 2019: 345-348.
- [34] Sharma H, Park J, Suda N, et al. Bit Fusion: Bit-Level Dynamically Composable Architecture for Accelerating Deep Neural Network[C]. ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA). Los Angeles, CA, USA: IEEE, 2018: 764-775.
- [35] 陈涛. 基于 FPGA 的 DDR3 内存控制器的研究与设计[D]. 成都理工大学, 2016.
- [36] Kautz W H. Cellular Logic-in-Memory Arrays[J]. IEEE Transactions on Computers, 1969, C-18(8): 719-727.
- [37] Stone H S. A Logic-in-Memory Computer[J]. IEEE Transactions on Computers, 1970, C-19(1): 73-78.
- [38] Fujiki D, Mahlke S, Das R. Duality Cache for Data Parallel Acceleration[C]. ACM/IEEE 46th Annual International Symposium on Computer Architecture (ISCA). 2019: 1-14.
- [39] Lin Z, Zhan H, Li X, et al. In-Memory Computing With Double Word Lines and Three Read Ports for Four Operands[J]. IEEE Transactions on Very Large Scale Integration (VLSI) Systems, 2020, 28(5): 1316-1320.
- [40] Kim J H, Kang S haeng, Lee S, et al. Aquabolt-XL: Samsung HBM2-PIM with in-memory processing for ML accelerators and beyond[C]. IEEE Hot Chips 33 Symposium (HCS). 2021: 1-26.
- [41] 王梓霖. 面向低功耗嵌入式系统的存内计算电路设计[D]. 电子科技大学, 2023.
- [42] Si X, Xue C X, Su J W, et al. Circuit Design Challenges in Computing-in-Memory for AI Edge Devices[C]. IEEE 13th International Conference on ASIC (ASICON). 2019: 1-4.
- [43] Kneip A, Bol D. Impact of Analog Non-Idealities on the Design Space of 6T-SRAM Current-Domain Dot-Product Operators for In-Memory Computing[J]. IEEE Transactions on Circuits and Systems I: Regular Papers, 2021, 68(5): 1931-1944.
- [44] 高世凡. 基于片上非易失存储器的模拟计算架构研究[D]. 浙江大学, 2023.
- [45] Tu F, Wang Y, Wu Z, et al. A 28nm 29.2TFLOPS/W BF16 and 36.5TOPS/W INT8 Reconfigurable Digital CIM Processor with Unified FP/INT Pipeline and Bitwise In-Memory Booth Multiplication for Cloud Deep Learning Acceleration[C]. IEEE

- International Solid- State Circuits Conference (ISSCC). San Francisco, CA, USA: IEEE, 2022: 1-3.
- [46] Gonugondla S K, Kang M, Shanbhag N. A 42pJ/decision 3.12TOPS/W robust in-memory machine learning classifier with on-chip training[C]. IEEE International Solid-State Circuits Conference - (ISSCC). 2018: 490-492.
- [47] Jeloka S, Akesh N B, Sylvester D, et al. A 28 nm Configurable Memory (TCAM/BCAM/SRAM) Using Push-Rule 6T Bit Cell Enabling Logic-in-Memory[J]. IEEE Journal of Solid-State Circuits, 2016, 51(4): 1009-1021.
- [48] 田喜贺. 纳米级 CMOS 高速低功耗加法器设计研究[D]. 西安电子科技大学, 2010.
- [49] Yue J, Liu Y, Yuan Z, et al. STICKER-IM: A 65 nm Computing-in-Memory NN Processor Using Block-Wise Sparsity Optimization and Inter/Intra-Macro Data Reuse[J]. IEEE Journal of Solid-State Circuits, 2022, 57(8): 2560-2573.
- [50] 张振伟. 应用于神经网络的存内计算电路设计[D]. 电子科技大学, 2023.
- [51] Yin S, Zhang B, Kim M, et al. PIMCA: A 3.4-Mb Programmable In-Memory Computing Accelerator in 28nm for On-Chip DNN Inference[C]. Symposium on VLSI Circuits. 2021: 1-2.
- [52] 司鑫. 面向人工智能的嵌入式存储器及存内计算电路设计[D]. 电子科技大学, 2021.
- [53] 陈炳欣. “存内计算”照进现实[N]. 中国电子报, 2022-03-04: 008.
- [54] 北京大学集成电路学院院长蔡一茂:存算一体助推 AI 新突破[N]. 中国计算机报, 2023-04-17: 003.
- [55] 张晔. 秀出科技范!南京这些企业跑出创新“加速度” [N]. 科技日报, 2021-06-25: 007.
- [56] Aga S, Jeloka S, Subramaniyan A, et al. Compute Caches[C]. 2017 IEEE International Symposium on High Performance Computer Architecture (HPCA). 2017: 481-492.
- [57] Eckert C, Wang X, Wang J, et al. Neural Cache: Bit-Serial In-Cache Acceleration of Deep Neural Networks[C]. 2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA). 2018: 383-396.